Hello and welcome to 3D Graphics and Animation. If you've watched a Hollywood movie recently or played a video game, you'll have realized the power of modern 3D graphics to create incredibly realistic effects, or to create very fantastical, cartoony or magical experiences. All of which it feel so really could reach out and touch them. 3D Graphics is a really exciting, and important part of our world and technology right now. This course will introduce you to it. We'll start with some of the basics of creating and setting out a 3D scene. We'll move on to look at how to make things move with animation techniques in physics simulation. In the second half of the course, we'll look a bit deeper into what goes on as you go from 3D models onto the pixels on the screen, and how graphics hardware works. The GPUs that already powering the modern video games in modern films and seeing how we can program those GPUs to create amazing graphical effects. This course is going to be a mix of theory and practice. Sylvia is going to work with me to teach you some of the mathematical side of it. Maths is really important for computer graphics. It's really about representing everything in our 3D world as numbers and as maths. It's vital to understand that there's mathematical side and how to implement them in code. You also have Tom coming in to give you an introduction to unity, which is the game engine that we're using for this course. If you've already done a course that uses unity, then you might have seen his Introduction already, or you might be doing two courses at the same time, which I'd highly recommend so that you could share that introduction work between those two courses. It's really important that you learn unity as part of this. Tyler and Rob will be coming in and teaching you some specific 3D graphics aspects of unity, that's again really important how you put a lot of this work into practice. As part of this course, you'll be both understanding that theory and creating some really compelling 3D graphics worlds. I hope you're enjoying it. Looking forward to it, like I'm looking forward to it. Let's get started creating some 3D graphics.

Creating a 3D environment is about getting some nice 3D meshes and assembling them into environments by positioning them around the world with transforms. But where do we get those meshes from? There are two basic ways of making meshes. The first is to create them using some 3D modeling software. As we've seen, creating a simple object is pretty easy. You can create stuff like cubes and spheres. You can combine them together to create more complex objects and you can create something reasonable looking just like that, all in an engine like Unity. But to go a little bit further, you need a more professional 3D modeling package and there are many around like Autodesk Maya or open source versions like Blender. You can create some amazing looking models in these packages, but it does need a lot of skill and experience to do so. I certainly think it's a good idea to try doing some 3D modeling, but I wouldn't say that everybody in this course needs to become a professional 3D modeler. The other important way of getting 3D models is to capture them from real life. By this, I mean

having a real physical object and digitizing it into a 3D model. There are a number of approaches to doing this. One is laser scanning, but a popular one recently is called photogrammetry. Which basically means taking lots and lots of photos of an object to an environment and using sophisticated software to combine these together into a 3D model. Because you're taking lots of photos from different directions, it can capture the mesh, the geometry of the objects, but also the surface texture and color that you can apply to it. It can create some really nice realistic 3D models. Having said that, probably the best approach for someone coming in as a beginner is not to create your own 3D models. There are lots of sites where you can purchase or even get for free, high-quality 3D models created by experts, including the Asset Store, which is built into Unity. That is a very good starting point for looking for models to adapt and put into your VR experience.

What applications of 3D graphics and animation did you find? I'm sure you found loads between you, there was so many ways we can use 3D graphics in animation. Some of the most obvious ones when entertainment particularly film, and that's probably where the first 3D graphics animations became visible. I personally remember when I was starting out working in 3D graphics when the first Pixar movie came out. Toy Story, it was so exciting to us that you could make an entire feature film just from 3D graphics. But of course, now almost every animated film is made using 3D graphics. It's no longer particularly an exciting thing but to aspheric graphics that should be in even more exciting because it's so ubiquitous and so important. 3D graphics can be so expressive, and if you think of the quality and excitement of Pixar movies, Disney movies, and other animated movies now they really fantastic, and the techniques that we'll be learning about this course underpin nice. It's not just about we traditionally think of as animated movies, the massive and fantastic special effects you can see in the lightest action blockbusters a justice important parts of 3D graphics, and the realism and the quality of them is really astounding now. I've been watching films going back from the '80s to the '90s, to the 2000s, I can see that the special effects in the '90s which I thought were fantastic, you can still tell that computer graphics. But if you look at modern films, you really don't know where you've got some 3D graphics, where it's real-life which is amazingly impressive. That's why I think some of the most subtle graphics effects a way more impressive than the big explosions in the magical stuff. One of the things I've noticed recently is how much you're now seeing what is called Digital Aging where you're taking actors at a certain age and creating 3D graphics versions of them when they were younger, whether this is Carrie Fisher and Princess Stallion as the young princess layer in Waegwan or other actors such as Michelle Pfeiffer in the Ant-Man and the Wasp. They were saying really able to create hugely realistic people who are essentially not there, and I think what so impressive is that there people so we're very attuned to human beings. We can really easily spot when something is wrong, and they

look seamlessly real so that if we didn't know that this actor is actually 20 years older than they appear in this film we wouldn't notice anything different and so it's subtle effects, so fantastic. Another very important area of computer graphics obviously is games, and all modern video games of course generate by computer graphics because generate all computers. The really extremely realistic, so impressive, realistic novel games, a testament to how much computer graphics has progressed over the last 30, 40 years. But it's not just about realism again, so you've got these really fantastic slightly more cartoony games that are really fun and really rely on the same computer graphics techniques we're seeing everywhere. Graphics animation is really important for entertainment but there are other applications. A lot of film and games are increasingly used in education as you can see here, but simulations and 3D graphics demonstrations are really important in a lot of education in science, where being able to see 3D representations of molecules is really helpful to understand the chemistry, for example. That's an example of a broader class of data visualization, so how you can represent complex data in a way that's easy to understand. A lot of data visualization is 2D in terms of graphs, in quite traditional forms but data that's increasingly complex needs new techniques, and 3D is one of the frontiers of data visualization that allows us to really visualize complex data sets in this way. This also leads to medical applications, so some of the data that's really important to visualize is data about the human body, for example, if we go into a hospital and have a scan whether that's a CT scan, or MRI, or even a traditional x-ray. That's increasingly going to be represent in some 3D form using 3D graphics techniques that makes it easier to analyze and diagnose because it's closer to the actual 3D forms of the human body. If you've been lucky enough to have a baby recently, you might even have had your ultrasound scan transforming 3D. I remember we didn't have that for our first child but for our second child, we saw the fetus in three days. For a reconstruction of the Opsound son, which is really incredibly exciting for new parent, it's your first real sights of what will be your new baby, the new member of your family. That's a really emotionally important application in 3D graphics. Follow much more emotional than even the most intense film that you can imagine. They were really important areas of application of 3D graphics both in entertainment and my more serious work that can really change people's lives, and a deeply important. If you choose to go into your career in 3D graphics, there are many different areas to work in and I hope you will enjoy them and [inaudible].

We need to learn maths in computer graphics because maths is used to describe and manipulate objects and also for the purpose of illumination. In this video, we'll be talking about one of the most fundamental building blocks in 3D graphics maths, vectors. You have probably dealt with vectors in your 2D graphics course. In 3D, they're pretty much the same but they exist and describe objects and relationships within a three-dimensional space which makes things a little bit more complicated. We'll have to

revisit some of the fundamentals. Feel free to skip this as long as you can answer all the in-video quizzes. Let's start with vectors in 2D. While we have understanding vectors is that they can be defined as a pair of x and y values which are the projections of the vector on the x and y-axis so that any point in this coordination system or your location can be expressed in the format of a vector. Vector a can be expressed as x_a and y_a. In this case, the norm of the vector or the magnitude which is a way we very often use to describe a vector can be calculated using the square root of a square plus y square.

Play video starting at :1:41 and follow transcript

1:41

Obviously, in 3D graphics, the norm of magnitude of vector a equals the square root of x_a square plus y_a square plus z_a square. Now you know how to calculate the magnitude of a vector which basically in plain English means the size of the vector, is a way to describe how long a vector really is. Knowing the math of how to calculate the magnitude of a vector is important as I'm now going to teach you a second way of representing a vector. Let's stick to 2D for now. As previously mentioned a vector can be described with its projections onto the x and y-axis. Vector a equals x_a, y_a. Here we're using two numbers to describe a vector. But sometimes you'll need to use another way to describe a vector and that is using its direction and its length. The length basically its magnitude which we just learned how to calculate. But how do we know its direction? The direction of a vector is essentially another vector. This might sound a bit pointless, but what makes it meaningful is that here we're using a special kind of vectors, they are called unit vectors because the length of these vectors are exactly one. Basically, if a vector is a unit vector now we know that its magnitude is set to one. That is the definition of unit vector and this is a concept you will come across a lot. Unit vectors are these vectors always exact the same length, one, but pointing in different directions. By fixing the length of these vectors leaving us only the direction to play with, we're able to then reduce the degree of freedom mathematically. This way we're able to turn complicated problems into simpler ones. Directions in a 2D or 3D space are essentially expressed by these unit vectors. They're vectors with a fixed length of one which allows for simplification of a lot of maths operations. In fact, very often in graphics programming, we have to set a vector length to one before we can do any other maths operation with it. This operation is so common, it actually has a name. Setting the length of a vector length to one is called normalizing it. The way we do it is by dividing each component of the vector by its magnitude. If you're familiar with the math behind vector normalization, feel free to skip this part as long as you can answer the quizzes.

Play video starting at :4:43 and follow transcript

4:43

Now we have a vector a which can be expressed as x_a, y_a. How do we find a unit vector which defines the direction of this vector? Let's say the unit vector of a is x_u, y_u, where we know that vector u has a magnitude of one. The square root of the sum

of x_u square and y_u square is one. Also, we know that as vector u and vector a point towards the same direction, x_u over y_u equals x_a over y_a. By rearranging equation 1 and 2, you can easily get equation 3. As we also know that the square root of x_a square plus y_a square is actually the magnitude of a so we can replace the bottom of equation 3 with the magnitude of a. We can express x_u as x_a over the magnitude of a and the same applies to y_u, so y_u is y_a over the magnitude of a. Basically, the unit vector of a equals x_a, y_a, can be expressed as u which is x_a over the magnitude of a and y_a over the magnitude of a, which can be rewritten as u equals to vector a over its magnitude. In summary, when we divide any vector by its own magnitude, this gives us a new vector which mathematically will have a magnitude of one. This new vector also maintains the relationship between its x and y component which means that it points at the same direction as the original vector. That is basically how we find the unit vector of any vectors.

Play video starting at :6:45 and follow transcript

6:45

Vectors can be represented by its magnitude and its direction. It's magnitude is a number and its direction is a vector, basically, it's unit vector. We often label unit vector of a as a prime, a can be expressed as its unit vector, a prime scaled by its magnitude. For vector a, x_a, y_a, a prime can be expressed as a over its magnitude and a can be expressed as a prime scaled by its magnitude. Here we use the star sign as scaling, which is a simple multiplication. But remember that vectors are not numbers, they are two or three-dimensional. We can have different kinds of multiplications as well. Later on, we will also learn dot and cross product of vectors. We need to be careful not to confuse these three multiplication operations, scaling, dot product, and cross product. These three all have different but important meanings mathematically.

Lets take a look at different operations with vectors. First, Scaling. If I'm scaling vector a with a scaler 1.5, which will give me a vector b, then b will be pointing at the same direction as a, but with a different magnitude. So if the magnitude of a is 2, then 1.5 × 2 is 3, the magnitude of b would be 3. We can say that b is 1.5 × a or a scaled by 1.5. The scaling operation basically applies the same scaler on each of the vectors components. So, for the new vector b, xb is 1.5 × xa and yb is 1.5 × you. Of course, scaling factor could also be negative. If you use a negative scaler, then you ended up with a vector pointing at the opposite direction of the original vector. So that's how scaling works. There are two other types of multiplication operations with vectors, dot and cross products. Before we move on to them, we need to go over to other simple operations, addition and subtraction. So first, Vector addition. It's really straightforward when you add two vectors together, you're simply adding the two x components to get a new x component and the two y components to get a new y component. But what's worth noticing is that by adding the two vectors together, you're getting a new vector, which

not only has a new magnitude but also a new direction. As an example, vector a, 5, 1 + vector b, 2, 4 would result in a new vector, 7, 5 as shown on this graph.

Play video starting at :2:7 and follow transcript

2:07

Geometrically, the sum of two vectors is found by placing the tail of one vector against the head of the other. And the new vector goes from the beginning of one vector to the end of the other. As a result, the new vector will have a new length and will be pointing towards a new direction. So here's a question for you. Is a vector addition commutative? As in, does a + b give the same result as b + a?

Play video starting at :2:40 and follow transcript

2:40

Yes, similar to addition with simple numbers, vector addition is also commutative. We can look at it both algebraically and geometrically. Both a + b and b + a give the same new x and y component, which is xa + xb and you + yb. And geometrically, we can move the tail of b to the head of a, or move the tail of a to the head of b. These two displacements would result in exactly the same new vector in blue. Now moving on to Vector subtraction. Similarly, we just do subtraction of each of the two components separately in order to form a new vector. So, if for a 5, 1, b 2, 4, a- b is the new vector, 3- 3. Geometrically, to represent a- b, you just draw a line from the head of one vector to the other, so you have your new vector.

Play video starting at :3:54 and follow transcript

3:54

So the question I have here is, is a- b the same as b- a?

Play video starting at :4: and follow transcript

4:00

Most of you must have figured out, they're not the same. With the same example for instance, here, a- b is 3, -3, but b- a is -3, 3. In fact, geometrically, when we draw the vector of a- b, the vector should always start from the subtrahend and point towards the minuend. So basically, a- b is not the same as b- a. In fact, a- b gives you a vector that has the same length as b- a, but pointing towards the opposite direction.

Another important vector operation is the Dot product. You remember, the scaling operation is the multiplication between a scalar and a vector and the result is a vector. Here, the Dot product is actually between two vectors and the result will be a scalar which is just one number. For a and b which are both two vectors, the way we calculate the Dot product between them is to multiply the X component of each vector and then we multiply the Y components and then we sum these two numbers up to get this one number which is the Dot product. For vector a; (1)(1) and b(4-2), a dot b will be an X component; one times four, we have four and for Y component, one times minus two,

we have minus two. Then we have, four plus minus two which give us two. This number or this scalar in vector math two is the result of a dot b.

Play video starting at :1:19 and follow transcript

1:19

Particularly, when we have two unit vectors, the Dot product of these two unit vectors will actually be the cosine value of the angle between them. This extends to vectors in general as well. We just need to scale them to a value between minus one and one using their Magnitudes. The cosine value of the angle between two vectors equals the Dot product scaled by their Magnitude. The relationship between two unit vectors can be viewed as a special case of this equation where the Magnitude of both vectors are one. Basically, the Dot product of a unit vector of itself is one. When we have two unit vectors pointing at almost the same direction, the Dot product is very close to one. As the angle between the two vector increases, the Dot product of them actually goes down. When the angle between them is 90 degrees or Pi over two where the two vectors are perpendicular with each other, the Dot product goes all the way down to zero.

Play video starting at :2:40 and follow transcript

2:40

So far we have learned a few vector operations including normalization, additions, subtraction, and Dot product. I'm mainly used 2D representations to teach these operations. You can easily guess that for 3D is just a matter of adding a third dimension onto the equation.

Play video starting at :3:5 and follow transcript

3:05

We have one final vector operation to go through, Cross product. Cross product is a bit special as it does not really have a 2D equivalent. In order to understand Cross product, you really need to think in the three dimensional space. One way to do it is to use your hands. You might remember that Dot product of two vectors is a simple number. But here, the cross product of two vectors ||a|| and ||b|| is another vector and this new vector is actually perpendicular to both ||a|| and ||b|| with the direction defined by the right hand rule. With your right hand, if the thumb is a, your index finger is b, then your middle finger defines the direction of the Cross product of ||a|| and ||b||.

Play video starting at :4:5 and follow transcript

4:05

The Magnitude of this new vector is the Magnitude of a and b times the sine value of the angle between a and b.

Play video starting at :4:19 and follow transcript

4:19

This way, the Cross product of a and b can be expressed as the Magnitude of a and b times the sine value of the angle between them. These varied together gives us a scalar

or the Magnitude of this new vector. We then need to also multiply by the direction of it which is a unit vector and perpendicular to both a and b and defined by the right hand rule.

Play video starting at :4:50 and follow transcript

4:50

The Cross to Cross product of two vectors is a vector. We can also express it with it's three components x, y and z, but the calculation is a little bit complicated. The x component is calculated with the y and z components of a and b which is ya times zb minus za time yb. The Y component is calculated in a similar way but this time with the set at X component of a and b and so as for the Z component which is calculated by the x and y part of a and b. If you remember, the Dot product is linked to the cosine wave where the result of the Dot product of two vectors goes down when the angle between two vectors goes up from zero to 90 degrees. With Cross product, it's the opposite. Here, the Magnitude of the Cross product actually goes up as the angle between two vectors goes up from zero to 90 degrees and then goes down as it goes from 90 to 180 degrees following this graph.

Play video starting at :6:5 and follow transcript

6:05

Here the question is; is Cross product commutative?

Play video starting at :6:13 and follow transcript

6:13

The cross product of a and b is not the same as b and a, so Cross product is not commutative. Here, the Magnitude of two operations are actually the same but the direction of this new vector are the opposite. You can try it with the right hand rule.

In this video, we're going to have a quick look at installing Unity. The first thing you're going to do, you're going to come up to unity.com. So unity.com here, if you just search for Unity 3D, you can search that then you will still find it on any search engine, but unity.com will dump you here at this initial page. Fairly clear, you can see all the things it's used for. Unity originally is a game's engine, but since cannot be used in all things. Explore this as you fill lots of details, lots of really interesting stuff to be had there. Explore that at your leisure. For now, let's just get cracking and get this installing. For all the creators play wherever let's click, "Get Started." It comes to this plans and pricing this is to say, you ain't paying that. Let's go to individual. Because you're student on the program, you should all have a University of London email address, so let's go to the student group. Let's give you some extra tips, which I don't know where this is. We've gone over this previously in the program or not, so click on, "sign up." If you're a student, you get a student license for Unity you get access to a few more teamwork options and you'll also get access to a few asset packs with some models that you can use to play around with foster learning Unity, et cetera. The way you get there, so just

click on "click get access." You can see there's three ways to access Unity here. I'm just going to leave my cursor on now so you can see where I'm a bit more. Different ways to do it. What you can do if you have been using GitHub with your previous assignments in previous modules, you can just connect Unity to your GitHub credentials. If you click, "Authorize," it will take you to a page that will link, ask you to log into GitHub and then it'll ask you GitHub, can Unity log into the GitHub to authorize that you are student. If you have the GitHub student status, the student Package, with GitHub education, then you can automatically get Unity with that. If you don't know what I'm talking about, that's fine. Look, I have another tab up here ready to click on,

Play video starting at :2:30 and follow transcript

2:30

@education.github.com/students, you'll find this page. The thing that we're interested here is this GitHub Student developer packs or anyone who's got a university email address will be able to apply for this. What do you get within that? Just pretty a lot. You get tons of stuff. There's lots of things here and again, you can go and look at that to your heart's content. But what I care about, and what we care about is this bit right here. We care about Unity so there you go used to create half of the world's games yet, pretty much right. You get a student plan free while you are student. That's how you get into that. Let's go back to Unity. We have got this page. We're going to download Unity now, and it'll bring you to this option here first-time users. The only difference between these two different things is that the first-time users just eases you into starting your own little mini-project there's some down here, so it will give you a choice of these kinds of games. It just dumps into pre-populated level where you can click learn and things work and there's some tutorials there about personalizing those games. If you want to do those, that's absolutely fine. It's a good fun way to just learn Unity in a different way. I'm just going to go to returning users and go for the clean install. Let's go here, you have to agree that you're not going to load some money you're a student essentially, and download Unity Hub.

Play video starting at :4:5 and follow transcript

4:05

This is where I'm going to leave this video for right now, and so download UnityHubSetup.exe, run that exe, it's very obvious steps. I'm not going to take you through all of it. But what that's going to do is it's going download the latest stable version of Unity and this is a long-term support additional, talk about that in a second. It's going to download and install Visual Studio with all the plug-ins that make it talk and work very nicely with Unity. The reason, a lot of point, you can't change if you really want to. Unity is known to work well with other things like Visual Studio code and there are pages in the Visual Studio Code website to talk about that. My suggestion would be make your life easy and just download the default version of Unity, which I know it's 2019.4. It will do that automatically and it will automatically download Visual Studio and

the plug-ins that make Unity and Visual Studio talk well to each other. I would just do that. I can't see any reason why you wouldn't do that?

Play video starting at :5:10 and follow transcript

5:10

Here we are at the Unity Hub window. A few things we spoke about the student license before, and the way you would get that working here is that you're going to log in. You can see that I've logged into I'm not going to click on this because it reveals personal details. I'm not going to click that. But if you click on the cog here and look at general, that shows me your Unity is installed as very little reason to put it in anywhere else other than where it belongs. You got licensed management. The license management is where you are going to deal with the licensing. You should have had an email from Unity to your email address giving you the license and there should be instructions within that email, and this is where you can access all of that. Let's go back to the main screen. Four types to look at, projects is the projects that you're working on in Unity. Just be where they got name, which version you're opening it with. That's because you can have multiple versions of Unity on your computer at the same time, we'll talk about that later. Target platform, self-explanatory, you making it for switch, you making it for iOS, you making it for PC. Last time you touched it, you can see these are old ones. I've got in the Cloud and the learn tab is just a quick way to access those things I spoke about before, these pre-made worlds for you to play around in. VR, one down the bottom here for those of you who are interested in that. But Tau defense and the Lego market game in FPS shooter, cutting mini-game. They give you various light bits of assets to play around move, or expose various ways to manipulate the engine and to get started. It's just like a nice little playground, and you can see they've got a link here to more resources on learn unity.com. Then tutorials is similar. These are tutorials directly from unity.com. It's just augment what you're learning here. Community, just some quick, things. The main place you're going to be is going to be going to unity answers and unity forums, where you'll be finding specific questions to your game, your defendant work. The ever-important tab is this one. On my computer I happen to have two versions installed. You've got 2020, which is one of the more recent ones, not the most recently at Tommy recording, but one of the more recent ones. You've also got this version. This version has got LTS by long-term support, and if indeed you didn't install 2019.4 When you install this and you've got something different here. If you end up here and click on "Add" then it's a selective version of Unity. Now if you want to download an old version of Unity for whatever reason, and they can't really think of any reasons why you would, because you can always important an old project made in an older version of unity and upgrade it. If it's not able to do that, you need an older version, then you can go to the download archive here in order to go and find it. But currently, unity Germany have three major branches available. They have to Tech branch, and they have the long-term support branch. They've got the beta program which is little bit

different to the tech branch. Tech branches there, mainstream branch, and it is these official releases, as you can see, 2020.2 is the latest one buffer then 0.1, which I have. The recommended release is 2019.4 long-term sport dish? Now, why would they recommend that one over more recent versions here, or indeed, why would they recommend that one over the Beta? In terms of the Beta, the Beta isn't stable. They won't be production ready. You don't want to use the Beta for anything that matters. You'll going to project matters. You don't want to use it. Don't use the Beta. In terms of the other versions, 2019.4 versus 2020 point, wherever long-term support version means that this version is not going to change for the next two to four years. If you are already in production on a game, or you're looking to lock into the production cycle of a game. You do not want to change the version of your unity. You do not want to upgrade from 2019.4 to 2020.1. You don't even really want to upgrade from 2019.4.16 to anything else in 2019.4. Long-term support addition means that this version is locked in and the only thing that's being released for it now is bug fixes. It's just like polishing it. There's going to be no substantial new features. It's just bug fixing, and that means you can be safe and secure if your game works here and it exports and does all the things you want it to, it's going to be absolutely fine. Was if you moved out a new branch, you'd be upgrading. Whenever you upgrade unity, you count, you run the risk of breaking your game. Though this happened to, I'm a development company one or two years ago. As a company was making a game for the switch, there were four weeks out from release and they decided to upgrade their unity installations and that went through an upgrade to do their projects and all their game code broke. They would not, and they had to not back release the game by six mumps. It was needed catastrophic and nearly sunk the company because I had to go and fix all their game. They had to go back to the beginning of quality assurance with Nintendo, who had to go through their internal quality testing, etc. Six months, that is a lot of money in marketing where they missed the apogee of their marking, all the things they time to get it out the door. Don't be that guy, don't be that person. It's also important when it comes to your projects. If you're working with people on a project, you all want to be, as we say in English, singing from the same hymn sheet, it means you all want to be on the same page in the same way that your game design document will get your singing from the same page. You also want to have exactly the same piece of software. To do this, you just agree on the version. The easy way to do this is to use the long-term sport addition. It means that you won't ever be in a situation where one member of your team has done work in a new version of unity and someone who's trying to bring that work into an older version of unity. It's trying to access that work. It can't happen. You can upgrade and a project may from an older version, so say 2019 to 2020, but you cannot take a project from 2020 and retrospectively Crimea into 2019.4. If you're using different versions of unity, you're storing up a world of pain and difficulty for yourselves and you have quite enough to deal with a ready. Just trust me, use the long-term support dish and you don't need the

latest one. I promise you.You've got Unity Hub open, looking at the Projects folder. Here, probably looks a bit more like this. That's too that. I've spoken about all these different tabs too etc. About having different installs and what have you. Let's go and look. I'm making a new project now, a word of warning to you, which isn't particularly obvious. If you just click on "New" here, then Unity will create a blank project using the most recent install that you have on your computer. Now, if you are like me and you were interested in some of the function differences to in 2019 and 2020, that would mean on my computer here right now, it would create a new project in 2020.1 which is going to play Mary Halvin with any group work that I do because you will need to be working on 2019.4.16 so that you know, you're using the same version. To avoid this, you click on the down arrow here and then it will give you a choice of which install to use. Go ahead and use the 2019 one. Then you have some templates. If you scroll down, you'll see, the once taking some reason, retrieving a list of variable templates. Here you go. You can see the some of those games that we spoke about before, they become test ones. But human interest in these top two is it going to be a 2D game? Is going to be 3D game? We're going to stick with 3D. Here's the project name, so I'm just going to call it Tester for now. Nice and inventive. Also just a word of warning. When unity creates new project, it will create the project in a folder with the name that you put in here. This location doesn't need a \tester folder to go underneath it. With this configuration or put it in my Documents folder \unity\tester, and then it'll populate it with the default project structure. We're going to do that and then just click "Create". We're going to let Unity do its thing. It'll come up with a splash screen. By the magic of video editing is made, you can see from your own machine experience that loading sequence can take quite awhile to establish depends on faster computer is. Before I start looking at some of the functions of the window that we've got going on here, we can look at the layout. Now, yours won't look like that, it will look like this. I've got a different layout which I'm using Forever tutorial videos later on in the module. This is the default layer. Before we talk about what each window is, I'll just talk you through what you can do the window layout to change it how you prefer. Just be aware that it's in the top right-hand corner up here. There's this tab here and the moment it says default, but this is where your window arrangements are. You can see there's a 2-by-3, there's a four split. For those who are more used to 3D programs like Maya and 3D Studio Max or Blender even. I've got one named after myself. This is actually my favorite way of using this thing. It's going to come out looking like this by default, but just be aware that you can drag and drop and play with these videos. You can drag them off and make them into floating windows. You've got multiple monitors. You can throw it to the other monitor and make use of that. You can organize them behind each other if that's helpful. That's come out. I wanted to get rid of the asset window anyway. You can drag them so that beside each other and then rearranged them fairly standard like you do. Lots of other things we can dock and redock these things. You want the console up here beyond

hierarchy. Go for it. Unusual, but you can do that. Then when you have made. Let's just do something unusual. Let me put my console up here and let me put my game window maybe down in the corner and you need to see a little preview of my game window each time it actually say I actually like that. Then you can go up here in the top right corner. Go to "Save Layout" something went wrong there. My highlights didn't work very well there. Go to "Save Layout" and then give it a name. This can be, I'll just going to call it Test. This is test layout. You can see that this is work because if I go back to default, that's what the default is. If I get drop-down and select test, they got a test layer with my weird console B on the scene and my game view down here. I'm going to go back to default for the time being. Yes. Actually what I will do is I'll go to my other thing I'm going to use in the future here, which is going to be the baton, that one. When I make videos later on, you will notice that this part of the screen is going to be blocked out. That's to allow some space for subtitles to come up for those of you who benefit from that. My head, I'm hoping will be in the corner here and this will be blocked out. Don't worry about those parts of the screen because I've used some windows I don't use. We're not going to be using it during this course, during that. But now I will go back to default. That's how I get to have my head in there. Let's look at what we have here. We have the same window. The same window is where you edit things, is where you drag and drop objects if you're placing things into your game. The game window is what someone would see if they're playing your game. Yes, they're playing your game. It's the view from the camera. Now, if I go into seen and then I do a drop-down here, then you can see that in this scene and the scene is basically a level, as it were within unity. There are actually two things that come as default as a directional light. We'll talk about lights and other time. There's a main camera. If you look at the main camera where the camera is pointing, you can see that is the same as the game view. The game view is the preview of what you see with the active camera. Sticking with arsine, I'm going to stay here. I'm going to right-click in the hierarchy window. Explain what is the mountain's going to do 3D object cube. I have made a cube. Whenever you have an object in Unity, you can press F to Zoom in on it. We can see in press F, I did that. If you could move around and you sort of lose where the object is. If you press F on your keyboard, always comes back and it will refocus on there so you can move around it. Now, the hierarchy I've renamed it, Buxton. Yeah, I have renamed it back. The hierarchy is a list of everything that is in your scene right now that is different to the project for the project window. The project window is what's available over the entire game. This is where all your scripts, your textures, your scenes, your assets, etc, live on your hard drive for the game project basically. It's not the same as what's in the hierarchy. The hierarchy is only showing you the game objects that are in the current scene, the current level. If you don't understand that now, that's fine. It will definitely become a lot clearer as we work with unity further. One of the other major windows is this one. Over on the right-hand side is the inspector window. The inspector window gives you all the details. You can

see this quite a lot of details to be had, just for a simple cube. I'm not going into those right now. But you can't collapse those whenever some of those are getting in your way. Down the bottom. In this configuration, at least we have the console window. The Console window is where areas and you can either write in code to have some areas given to you. Or they'll be areas that even that you don't write, that unity will give to you when it's looking for reference, it's missing something, etc. All that data and the errors will come up in the console. Then to move around the scene window. You saw me moving around this cube like this. I can do this and I could do this. Now if any of you have used 3D modeling software before and I have. Then this will be second nature to you is the usual holding the alt key on your keyboard. I'm not entirely sure it's going to be for Mac, I would imagine the Apple key. It's going to be an alt key on a Windows PC, and you can drag it around the focal point. With your main mouse button, your left mouse button, I'm left-handed, so I have to reverse this. With your main males button, I'm going to say, you can rotate around. With your middle mouse button, you can pan in the same plane. With your right mouse button, you can use that to Zoom in and out. You can also achieve the same thing, but pressing W, E, and R. That pertains to the objects. If I wanted to move my object, I press W. If I want to rotate it or press E. You can see I can rotate that. Then this r is to scale the object. You can scale it in different ways and were heavy. If you ever get lost with all this and you just want to reset it, you can go up to this part in the inspector over here. Then you can click on these three dots and do reset. Now reset it back to how the object was born into the scene essentially. It resets what we call the transform. Transform is the position, rotation and scale of the object and we'll talk about that more in a future video. Another nice little one key that I like to say, so I just press there to re-highlight if I create a second object, so I create a second cube. You can't see it. Why is that? It's because they're directly over each other. If I just press W to move, click on those arrows. If it's yellow, it shows. Highlight it, and pull it. You can see, it's moved like that. Now if I wanted to snap these objects together, I can hold V. Now you can see once I'm holding V, I will be able to snap by vertex. If you wanted those two objects to snap exactly together, that can be very useful. Let's just drop that in.

Play video starting at :11:27 and follow transcript11:27

In terms of other things on the window, in terms of these icons at the top. These are what you call gizmos and there's lots of different options there. You can turn the lights on and off. You can make this a 2D views. If you're working on a 2D game or 2D part of the game, then it would reset to what the default 2D view is. Again, people use the 3D. You can go for Wireframe mode or Shaded and Wireframe, which can be helpful. Default is just plain shaded view. Have a little play, have a little mess around with that.

Play video starting at :12:7 and follow transcript12:07

Like I said, all these things that you see in the scene view are called gizmos. They're the icons essentially and you can turn these on and off. Go around, have a little play, see

what is there. The other main thing to point out to you in terms of the UI here is the Play, Pause and the Frame Advance button. Now, when you play a game, what you do is use Control P or you can click this button up here. If I click there, this is now running my game. Remember I said that the game will show you the view from the camera so you can see it's totally different from the scene view. This is what the camera sees, my first cube and then my other cube just in behind it there. If I have to pause it, that pauses the game play but it still allows me to stop, pause and diagnose what's going on in my scene. I can go to my scene view once the game is paused. This is like pausing the action in real time. This is really good for debugging and working out what's going on, where your objects are going, if you can't see them and things like that. If you have paused, played, and paused like this, you pause the action. You can then use this button to step, and it will just go through, frame-by-frame. There's nothing to move here so you're not going to see any action. But when you do have things moving on the screen, then you can use that to just advance frame-by-frame.
Play video starting at :13:49 and follow transcript13:49
I'm just going to hide my taskbar, there we go. We've got a bit more space. Unless you change this already, important thing to note. You'd see it has two modes. It's got editor mode, which is what we see here now, and it's got play mode, which is when the game is playing like this. They're two very different modes. You can change things in the inspector to play around the behavior of your game. But when you stop pressing "Play", then everything gets reset. There are ways to overcome that and we'll come back to that in another video. But you'd see that the box usually has a light gray cover. Now you've seen that I've got purple configured as my thinkers. Purple is one of my favorite colors as you can see. If you want to change the color that covers your unit window, and I strongly recommend you do so it's really clear to you when you're in play mode or when you are not. If you go to Edit
Play video starting at :14:54 and follow transcript14:54
and then go to Preferences, then in Colors. Colors is the play mode tint. Then use this, and then select whatever color you fancy. Knock yourself out as your editor. This is where you can change all the other colors. But the one thing I would definitely do, so if I use the defaults, you see it goes gray. If I click show you with the default, use Control P to start the play mode. That to me is not very clear though I'm in play mode. If I go to Edit, Preferences, and then somewhere there, it has been dark in there before play mode tint. It's been darker than I had it before this right one. Trust me, you'll thank me for that. Go and change your play mode tint. Now, aside from this, I'm going to leave it there for the main UI interface. I'm going to look at documentation interface. To access documentation, it's very easy. You go to Help. Then you've got two parts of documentation. You can see there's some links here. Actually form an [inaudible] , which I spoke about in the previous video. You've got Unity Manual, you've got Scripting Reference. Lets get to Unity Manual first.

Play video starting at :16:17 and follow transcript16:17
That has opened up a different screen unknowingly. So I'm just going to bring that back to my screen. There we go. Apology accept. So you need to use manual, tells you what version it's for, confirms what it is in the top left-hand corner here as well. You can see also, just want to highlight this, that once you can access all of this online from their website, this is often what will come up if you Google for things, you find yourself back in the menu or the scripting reference. This is actually coming from your hard drive. So all of this is available for you when you're offline as well. So it's important to note that. So this is the Unity manual, which is the full documentation about all the different parts of the program.
Play video starting at :17:9 and follow transcript17:09
So UI is a bit different. UI is not the UI that we're looking at, that's to do with making UI within the game itself. Working in to would be the introduction to the actual UI, et cetera, so if you wanted to look at something about what we've been talking about here, here's the stuff about Unity's interface right there. Now, the other thing I'm going to call your attention to is there's another code of documentation called the scripting API. The scripting API is all the libraries that come with Unity. So when you're using C-sharp in Unity, not really using just C-sharp, you're using the Unity flavored version of C-sharp. So if you wanted to find out about someone that's a collision, then it will start talking to you about all these things that might be to do with collision. You might be actually be looking for a rigid body. Might be looking for a collider, et cetera. You can search in the manual in the same way. So in most circumstances, certainly in the beginning, you'll be looking in the manual for the collision. It tells you about the comic collision. Just make sure that you've always got the correct version. If it's coming from your local hard drive, then you're going to have the right version. If you're looking at this online, then you might have a different version up in the top left here. So just make sure you're looking at the right version otherwise it'll be a bit disorientating. All the menus, all the options, et cetera, far more than I could ever teach you in these videos. But just a little thing to understand is that if you are wondering, so let's go back to our cubes. You're looking at all this stuff that's here. You think, "What is all this gum from the side? What does it mean?" To say what's the transform? What's a Box Collider? You see little question mark icon there. You can open reference with the Box Collider and that's just nice, convenient. Gives you preview the box and it tells you what a thing is and you've got links. It shows you where it fits in amongst the greater whole. It's a 3D physics component, which we'll talk about later. But this is a way for you to get used to navigate the documentation from the editor.
Play video starting at :19:26 and follow transcript19:26
So the final thing, I haven't got rid of it earlier, but I'm going to bring it back now, it's the Asset Store. Now, if you want to bring up any Windows, you're following a tutorial, or you're just playing around, you want to bring up any Windows, or say for example, you

drag off for Window,. You make it floating like that, and then you've canceled accidentally. "Oh my goodness, how do I get it back?" It's okay, go to Window, and then you have to find it. I know where it is. I know it's console is. It's so General, Console. Bring it back. I'm just going to drop it down here. It's where it usually lives. So final thing, the Asset Store. So if you go to Window, you'll see Asset Store there. It'll bring up the Asset Store, and it comes up like this. I'm going to tear that off. Then I'm just going to maximize that now. In many ways, the Asset Store is a little easier to navigate through a web browser to be perfectly honest. Run through Unity Direct. Just need to log into the web browser at their website of Unity, the same way that I have done into Unity here. So it's a short form for all things to help you make your game. There's everything you could wish for under the sun here, mostly, and in all price brackets, including many free assets as well. So you can see here, they've got staff picks, the top free assets, down here, top paid, et cetera. Lots and lots of things for you to create. So for you to explore and maybe using some of your creations and play around with. It's a fantastic way to help you get started with development. For example, you can get some characters working around in your prototype, gets some free character models working around your prototype. Again, once you're working at the gameplay and what you want your game play to actually do. You can use assets for sounds, for coding function A and C to save you time, making it yourself if it's a fairly big, large function. It's also great for learning, especially for the programming assets. You can download them. You'll get all those scripts for free. You can just go into the code and dig into the code a bit looking at an open source project really. You can go into the code and tinker around with it and really see what's going on. So we didn't see in the Asset Store, it's there, so you can say assets. You can look at the different ways of different departments where they have things. You can also search for assets, so say I wanted to look for sounds. I could do that, maybe I wanted to look for nature sounds, no particular reason, but nature sounds. I could go for using free assets. I could restrict it to my own specific version just in case there's something annoying. If I want to only really look at the best, I can shrink it down, but I always got no results now, wants it. Let's go for all unity versions, I think that's the problem.

Play video starting at :22:32 and follow transcript22:32

Sounds nature, they got nature sound effects doesn't seem to be a huge amount of, seem to have picked an unpopular way of shrinking things down. You're going to get recommendations for paid stuff at the top just because it is still a shot from, but you can use these things on the site too. Help you find what you want to. I'm going to get rid of the assets, don't know.

Play video starting at :22:58 and follow transcript22:58

The last thing to look at is the Package Manager. The Package Manager here is all the Unity in-house accessories that you could add for free. This is all stuff that is owned by Unity, so for example, ProBuilder used to be an asset that you bought for money in the

asset store, and then Unity bought them. They bought the people that may ProBuilder, and then ProBuilder became something you can have for free within Unity. It's used to provided some fairly rudimentary geometry manipulation. You'd mainly use it for blocking out level. You wouldn't really want to use ProBuilder for anything advanced, you'd use a dedicated 3D program like Blender or Maya for that, but for blocking out level, this can be really, really useful because they only have very basic objects available in Unity and you can't really manipulate them very much, but it's ProBuilder does allow you to move vertices and edges etc. That's an example of some of the things, and there is all kinds of function that needs to be heard there. It's a bit like an asset store, so you don't need to buy stuff that's just there already, but it's not important standard when you start a new project, because you've installed all of this, you can appreciate, you'd have a very bloated fat project, with lots of stuff that isn't needed.But what we do in order to install these things, so example TextMeshPro is your I don't date it, so I just click "Install" and then when it's installed, you will see in your project, some folders appear with all the assets and stuff that is required for that asset to do its thing. Last thing that you want to test if everything's okay. We're going to make it jump example script to show how the console of the visual studio integration works. Details aren't important here, but I'm just going to right-click in my Project folder, I'm going to create a C-sharp script. I'm going to call it testing. If you select a script, you'll see in the inspector, the text of the script, that is not really very useful. What we want to do is double-click, and then you open Visual Studio. Like I said, if you've installed your [inaudible] just follow the instructions, just click "Next" you'll end up with Visual Studio Community additions to the free version installed on your computer. It's opened up in one of my other monsters, and I'm just going to bring it over here and I want to type some things.

Play video starting at :25:44 and follow transcript25:44

All I'm going to do is in the Start method. I'm going to go into the Start method such is in a moment, I do Debug.org.

Play video starting at :26: and follow transcript26:00

I've done that, I've saved it Control S. I'm just going to minimize Visual Studio. Just simplify the scenes, delete one. I just clicked and press Delete there. Order to attach that, to make this script here within the scene, I need to actually attach it. I'm going to drag and drop onto the cube, check on my cubed, I have a script there. I'm going to go through what I've just done and what this means, etc, in a future video. I'm going to press play. That's running. My console has a message saying hello, so that's what console input is going to look like. Now, the final thing I'll say is I cannot teach you everything there is to know about Unity. I'm sure you can see just from looking at it, is a massive program. We've lots of features. Everyone's needs are going to be different, everyone's games are going to be different. But I hope this gets you started with playing around and experimenting, and don't forget that there's the documents available here.

Don't forget you've got the manual here, you've got the scripting reference there. Again, just mentioned from the help menu here, you've got the forums and the answer's, etc to go and look at, and you've also got some extra tutorials on YouTube website to help you. I hope this helps you get started playing around and experimenting and getting into grips with the Unity interface.

In this video we're going to talk about unities three different coordinate systems.

Play video starting at ::16 and follow transcript

0:16

Okay, so there's a few different ways to talk about coordinates in unity but your three main ones that are going to be converting term from are your world system, your view system and your screen system. So, world coordinates are kind of as you might expect, there where the object actually is in 3D space. Screen view is where that object appears on the screen. So this is the 3D world that is rendered in a 2D plane. So these two sets of coordinates are different because you're using a perspective view of the world relating that to 2D plane. And similar to a view, but with the view you can have multiple views on screen at once. Here you see an example of a demo of a four player multiplayer game, and you can have multiple views there but that is still a 2D view. So, screening view taking a slice like a snapshot of a certain point of the 3D view. And a worldview, a world coordinates are expressed as infected Three formats, your x, your y and your z position as a Vector3. But be aware that screen and picks and view coordinates are expressed as Vector2s. So this will be screening pixels, but also the views from 0 to 1. So they actually use slightly different ways of referring as well. So I'm just going to illustrate this. So with a screen for example, because you're counting in pixels. In a standard widescreen fill HD monitor, you go from 0 to 1920 on the x, and 0 to 1080 on the y axis. In a view, because you can have multiple views, 0 to 1 is where you're going to from and to in both axes. So, for example, here, just one of those views from 0 to 1 would go from the far left to the far right and from the bottom to the top of the top left, which means that for each of those views, actually, it's 0 to 14 times over. Because you can have, as I said in a previous lecture, you can have as many cameras as you like. Now, when you're working with the coordinate system, you're working with objects moving in certain dimensions and where they're moving in terms of relative to other things in 3D space, it's worth remembering what your directions are. As I've referred to in previous videos, your positive z is forward. Your positive x is right and you're positive y is up. And if you find it difficult to remember, then think about the colors that the words are in the slide here. And then think about the gizmo that exists in the top right hand corner of the scene view, which shows you your x in red, your z in blue, and your y in green. The most important thing is if you're used to other 3D systems, just remember that y is up. And also be aware of the fine print with the documentation. It can be quite easy to miss, but can cost you a lot of time and pulled out hair if you indeed

have any hair left. So, for example, when you're translating Viewport to WorldPoint, you're trying to say, translate where you're clicking on the screen and translating that to a world coordinate, you can see here we have underlined in red. I'm just going to hover my cursor over. Viewport spaces normalized relative to the camera. The bottom left you put 0, 0, top right is 1, 1. Z position because you're going from a 2d will Viewport to WorldPoint in 3d. Z position is in world units from the camera. And I just say, be careful about that because you may not necessarily be using the z axis. Now, if you're building your world in terms of how it might look in reality, so you build your world along the X and Z axis, the forward and right axis. Then if you want it to be top down and say like I have in this example, then you'd actually orient your camera. So it's pointing down the y axis, not along the z axis. So this is just a design choice that you choose when you start building at the level. Are you going to build the level flat in front of you like this? Are you going to build it as you kind of more extinctively feel? Just take account for that. So, for example, in this example, I would actually have to calculate and you can see here with my arrows. You can see from there, I'm looking at the y axis here, so I'd have to subtract the position, the Y position of the camera from the y position of the object I want it to be looking at in order to give that z coordinate for the previous set of documentation. So just be aware of which access that you're using and where things are pointing. And if they are actually pointing along the z axis and maybe you have to change your code every now and again for that, that's just a quick culture that can easily catch you out.

The virtual worlds that we want to create are essentially made out of objects, they're made out of things. Each of these objects has to be represented in a computer in order to be rendered in 3D graphics. We've seen how spaces, and positions, and points can be represented as numbers. Any point in 3D space can be represented as an X, which is across, a Y, which is up and down, and Z, which is in and out. We can make use of this fact build up objects. Objects in 3D graphics are built out of points. A whole bunch of points that are represented as an X, a Y, and a Z. These points are joined together to make lines called edges. These lines are then joined together to make 2D shapes, called polygons. Lots of these polygons are joined together to make a 3D surface, called meshes. Let me show you an example. This is a 3D object. You can see the little pink dots. These are points that define the shape. We call these points vertices. They're the corners of the object. You can see, on the side, one of these vertices, there's three arrows coming out of it. These three arrows represent the X, Y, and Z positions of the vertex. If you pull the green arrow, you will change the X position, and if you pull along the blue, you will change the Z position of the vertex. The lines in black join the vertices together, and they form the edges of the polygon. A polygon is just a flat shape made out of straight lines, just like I'm showing you now. This object is made out of four-sided polygons, called quadrilaterals. Each of these polygons has four corners, four vertices which are joined together to make four lines, the edges of the polygon. These polygons

are essentially squares that I messed around a little way bit to change the shape. These flat shapes, the squares, are put together to create a 3D shape. Each quadrilateral is one side of this 3D shape, which was once a cube, and I've messed it around a little bit, so it's a deformed cube now. We call all the vertices and polygons the geometry of the object because it is the geometric shape. We'll come to other things as well as the shape in future lectures. The full shape is built up of polygons. This can seem too simple. Real-world objects aren't made out of flat shapes, they're curved, or jagged, or often uneven. But actually building objects out of polygons can work really well. Because they're simple and they can be handled well on a computer. Your graphics card can render millions, or even billions of polygons every second. Because you can have lots of polygons, you can use them to make up really complex, and rich, and smooth curved surfaces.

Creating a 3D environment is about getting some nice 3D measures and assembling them into environments by position around the world with transforms. But where do we get those measures from?

Play video starting at ::23 and follow transcript

0:23

There are two basic ways of making measures. The first is to create them using some 3 D modeling software. As we've seen, creating a simple object is pretty easy. You can create stuff like cubes and spheres, you can combine them together to create more complex objects. And you can create something reasonable looking just like that, all in an engine like unity. But to go a little bit further, you need a more professional 3D modeling package. And there are many around like Autodesk, Maya or open source versions like Blender. You can create some amazing looking models in these packages, but it does need a lot of skill and experience to do so. I certainly think it's a good idea to try doing some 3D modeling, but I wouldn't say that everybody in this course needs to become a professional 3D modeler. The other important way of getting 3D models is to capture them from real life. By this I mean having a real physical object and digitizing it into a 3D model. There are a number of approaches to doing this, one is laser scanning, but a popular one recently is called Photograph Matori. Which basically means taking lots and lots and lots of photos of an object or an environment, and using sophisticated software to combine these together into a 3D model. Because you're taking lots of photos from different directions, it can capture the mesh, the geometry of the object, but also the surface, texture and color that you can apply to it. It can create some really nice, realistic 3D models. Having said that, probably the best approach for someone coming in as a beginner is not to create your own 3D models. There are lots of sites where you can purchase or even get for free, high quality 3D models created by experts, including the asset store, which is built into unity. That is a very good starting point for looking for models to adapt and put into your VR experience.

We're going to get into some more unity stuff now. This is going to be our setup for a bit. You've got myself here and then over that side, down here, I've blocked out the bottom of the screen. That's so that if you're using subtitles I'm not going to put anything on screen that's going to block out subtitles for you. This is my layout just so I can make videos. You can obviously have a Windows layout, as you wish [inaudible] previous Window. I'm just going to tuck up the assets there, so we're not distracted by that and the projects, so I'm not talking about that yet. This is essentially what a new scene looks like. I'm going to talk to you about a couple of things. But the main thing I want to talk to you about is the basic model that underpins unity, which is the GameObject and component model.

Play video starting at ::55 and follow transcript

0:55

Let's go to "GameObject" and let's go "3D objects" and let's click "Cube", fantastic acute, press "F" to focus. Use alternate and the three mouse buttons to move around and tumble, and zoom, etc. A cube, this is a GameObject with several things attached to it. Each of these things here, like the box collider, we've it's options, the Mesh Renderer with its options, the cube mesh filter with its options. All of these things are called components and the transform it's called a component. All these things can be tucked under and hidden if you don't want to see all of them at the same time, you just want to focus on one thing. At the bottom of this material is linked to the Mesh Renderer, what's been used in the Mesh Renderer. All these things are called components. But what is it that holds these components then? Where are these components being held? They're components that bolt onto something. What is that thing? Well, let's go to GameObject, and then create "Empty". You can see in a hierarchy got my cube, GameObject, I can't see it. Let's zoom out. Let's just drag it over there. The GameObject is the smallest unit really of anything in unity. The only thing that an empty game object has is a Transform here. That is what the GameObject is. It's Container or place for components to bolt onto.

Play video starting at :2:41 and follow transcript

2:41

If you see add components even inferred, there isn't it. If I get rid of the search box, you can see there's actually a lot of things that I can add here. Indeed if I started looking for one of the things my cube as a box collider. I can add a box collider. Actually I could add a mesh filter and I can add a mesh renderer, I believe I had as well. Mesh filter and I can set the properties there to make it a cube. Then at the moment it's bright pink, that means it hasn't got any proper textures on it. This is what happens with an object doesn't have textures and unity. But scratch the Mesh Renderer and let's pick the default material. I've essentially just made a copy of that cube. There's no difference, the components in slightly different order, but it's exactly the same. In that way, in fact, a lot of the objects from the GameObject menu of things you can put in are GameObjects

with various components attached. Let's look at a lightness look at spotlight. A spotlight is a GameObject. We have a transform and it has a light component attached, a light component of type spot. That's what spotlight is, its game object with a certain combination components. In fact, everything is a game object with certain combinations of components attached. If I wanted to rename this, I press "F2" there, my special cube. If I wanted to rename it a different way, I can go up here and go "MyNotsoSpecialCube" if I wanted to, if you felt so inclined. Let's just have a look at how this works out. Just use the diagram here. I'm just going to turn this on. What I've just told you is everything attaches to the GameObject. This is the broad thing that everything attaches to it. There is a transform, which I told you was the only thing that an object needs. In this example, I had had a box collider and I've got a mesh filter. I've also got a Mesh.

Play video starting at :5:15 and follow transcript

5:15

Get my ink there, Renderer. The only thing that absolutely has to be there is a transform. That's the minimum you can get away with. All the rest of these things start completely separate from each other. They are individual components. Now, it so happens that these two linked, but they still stand completely separate to each other. But at the same time, they are actually connected to each other through the GameObject. But a GameObject, if you like, is a container for all these. Another way you may want to think about it is here's you GameObject, then here's your transform. The first thing that goes into that bucket. Then here's your box collider. Here is your Mesh Renderer and as you mesh filter. But they all go in this box, which is a game object. That's how the game object model works in unity. Let's go back to unity.

Play video starting at :6:29 and follow transcript

6:29

I've added my bit, so I've showed you, we can build up this same thing. The other thing to remember here actually is at Scripps that you write in code are components in enough themselves. Now if I come over to my not so special key and add a component. I've scripted it. It's just a blank script actually called Hello World, which is here under the facets, Hello world. Now actually, what's a good practice? Get into, it's got to "Assets". I'm going to right-click in my project window, not hierarchy to Project create a folder. Before you get to you first it's good create scripts folder. I'm just going to drag and drop that in scripts.

Play video starting at :7:16 and follow transcript

7:16

Let me bring up Hello World in my editor. This is what an empty script looks like. For example, say when the game runs, I wanted to do some really simple. I want to print something in the console log window. We can use that using a command called Debug.log. I'm just going to go Debug.log. Hello World. How are you? C-sharp, so it needs terminates at the end. It can control and save that. When you create a new script,

every script comes from these two methods for mono behavior. I'll talk about what mono behaviors in the second. There's always going to have start method and update method. Start method is what happens when the object becomes active and update is what happens every single frame. If you get favor on your programming and you've ever finds you don't need the update method or you're not going to put anything in there is actually technically best to remove it because that method still fires every frame. It's not a major thing at this level, but later on that could save some performance for yourself. It's not big thing. Let's go back to Unity. At the moment, if I run, nothing's happening. In fact, I've got an objects because I've got script attached to my camera, which I forgot to remove. I'm just going to remove that component. At the moment, there's nothing happening, so where's my Debug.Log? Well, I forgot to attach it. I'm going to go here to my project, drag and drop Hello World script onto Mynotsospecial key. Now look, it turns up looking like a component. In fact, if I remove this component, I can add it like I did. Only have components by typing in the search and his Hello World. If I play again, here you go, Hello World, how are you, and if I look in the Console window, you can see where it's coming from from his Hello World script.

Play video starting at :9:41 and follow transcript

9:41

Showing you that the script is a component in of itself, but it's also connected to the other parts of the game object by being attached to the game object. Now I can show you this. By going back to my testing here. If I wanted to say print out the transform, I could say "Position," and then concatenate, this concatenation operator there.

Play video starting at :10:20 and follow transcript

10:20

Transform.rotation.

Play video starting at :10:26 and follow transcript

10:26

Let's just go and check that that runs. Into its combining. "Press Play". There you go. Hello World, how are you, and a position there.

Play video starting at :10:41 and follow transcript

10:41

What is going on here. Well, let me just go back to the code. What did I saw. I did Transform.rotation. I did wonder whether it was a problem. There you go position. They came up with four numbers there for rotations because rotations happening quaternions rather than what you're used to X or what we all used to XYZ as in Euler's. That's a different lesson. Transform position, apologies there. Sorry. Now we will shall see three figures. That makes more sense. Position is that.

Play video starting at :11:21 and follow transcript

11:21

How was I able to get that. Because this component is attached to a game object, I can see all the other components that are attached to it. You can see here there is a Transform component of my not so special key. I can still access the Transform of the game objects. I don't have to make a reference to this actual game objects. I can just say the Transform, which is the Transform attached to this game object. Here you can see I've used the tooltip there. Visual Studio is really good for helping you learn C-sharp and particularly the into flavor of C-Sharp. Get used to using the IntelliSense the code suggestion, and just look at the menus of all the functions you can do and it helps you find out if you're in the right place or not.

Play video starting at :12:5 and follow transcript

12:05

I talked about how these functions are Parchment behavior. That's the thing up here. There you go. Model behaves as the base class from which every Unity Script derives. Now, if you've used Unity a lot before. You'll know that there are some things that are not Mono Behaviors such as scriptable objects, but [inaudible] purposes for these lessons, everything derives from Mono Behavior and all the functions that you use from the base class of Mono Behavior and everything that you read in the scripting docs, majority of it is going to be derived from Mono Behavior. Mono Behavior is just the script class and you can see at the top here, Hello World inherits from Mono Behavior. The things that you need to remember here are that the game object is the fundamental container within Unity. Everything else that attaches to it is a component and all objects and Unity are essentially game objects of transform, with another collection of components attached to them and that's why every object in Unity is.

Okay, so here we are in a clean unity environment. I've got some my priest house from previous videos, but this tool tips purposes. This is a blank project. So, first thing I'm going to do it. I've got scripts folder I've already made. I'm going to make three scripts in this project. It's just going to be super simple. Make three scripts. I'm going to have one script that keeps track of the score. I'm going to have one script that prints out to the console here in the bottom of the screen. I'm going to have one script that talks to both for those as a kind of game controller might do in a real situation. Okay, so first things first, let's go up to the top here to game object. I've got three d object. We're going to make a cube. The Mighty Cube. Okay, so, this Mighty Cube is going to be called the grandiose name of game controller. Okay, welcome to game controller. And then we're going to look at the script we going to make, so I said I'm going to make three scripts, so create, one is going to be called it Variable track.

Play video starting at :1:13 and follow transcript

1:13

Okay, so I'm going to double click.

Play video starting at :1:20 and follow transcript

1:20
Yeah, open up in visual studio.
Play video starting at :1:25 and follow transcript

1:25
Okay?
Play video starting at :1:28 and follow transcript

1:28
Say all I want my variable tracker is a variable to track. I'm going to make it score and then I'm going to have a function in there to update that score. So all I'm going to have on top is going to have public because it's going to be an integer, score equals to zero have there and then below, I'm going to have public void add to score, which takes no parameters. And it's just going to be score plus equals to one. That's it. Because I'm not going to use update, I'm going to delete it because it gets called every frame, and I'd rather not. So it's good to get into the habit deleting that if you don't need things like that. So that's one script, then I'm going to create another one. So right click on scripts. Unity is just compiling. Yeah, great, C sharp script. Okay, so the next one I'm going to make is going to be the one I'm going to use to print out to the console, so I'm going to call that console script.
Play video starting at :2:38 and follow transcript

2:38
Okay, it's been my co window back, so I got console script here, so I'm going to have at the top. We're going to have a hint. I'm going to call this temp score.
Play video starting at :2:54 and follow transcript

2:54
So we need, yeah, I think that's all I need. And then I'm just going to have a couple of functions here. I'm going to have one public void say hello. That's going to be debug dot log, hello, world. Just to check that, we have to speak to it and it makes sense and everything, and then I'm just going to have another one public void print score. Okay, so, temp, I'm going to make my temp score, and I'm going to make that equal to get components variable tracker top score and debug dot blog score. So you're coming on their concatenation temp score.
Play video starting at :3:57 and follow transcript

3:57
Okay, so I could just do, score like plus temps plus get competitive variable dot score could put all in one line. But wanted to show you here that I haven't made a reference here to trade thing yet, so this is going to have to go on the same thing, all right. So I'm going to intend to put my console script and my gain control the script on the variable tracking script on the same controller on the same object. So that get component means get a component on the same object to which this component is attached. I don't have to go and search for where is the game controller? Where is this? And then get the

game component from the game controller. If I'm already attached to the game controller, then I can do that. Okay, so that's it, I've saved that and T will recompile and I can drag those two dragon drop onto the game controller and then the game controller. I'm just going to remember in the top, right corner. I've got collapsible components here. We can see that both of those are there and there is indeed a public variable their score that I can use, Kate. So then I'm going to have a third script. We're going to call that testing. It's inventive, isn't it? Let's open that. We will be adding this to the main camera when I'm done. So, Let's start off with one way. I didn't say. First of all, I need to make a reference to the game controller.

Play video starting at :5:53 and follow transcript

5:53

So I'm going to do that, and I'm not going to do the public aside method because I want to show you a different way of making a reference, you can use the public. I could make this public game controller game object game controller, and then drag and drop the game controller. That's absolutely fine. I'm just going to show you a different way. So game on game controller, so game controller equals, and then it's going to be game objects with a capital small G means just for this instance but I actually want to access a function from the entire class. So capital G for game object find, and then use this two. Find the object that is called game controller. So whichever object has got the name of game controller, I'm going to sign that to game controller. So that's the first thing that we're doing. Making references is expensive in terms of process of time. And if you make it doesn't matter really, at this level, with the number of things we're talking about about your game increases. Yeah, and as your projects increase in the future, good practices make your references once in the start method. Okay, so we've done that, and then in update we're going to put some input here now. So if input. So I move my curse out the way input, get a key down, and I just tend to default to use in space bar quite frankly, then so here's one way of doing it. What we could do is we could say game controller dot get component. Open that variable tracker, variable tracker dot add to score. That's absolutely fine if I want it. So what I want to do is every time I press space on the head, one to the score or add some bits to the score and then print it out to the screen, it's okay to do this. This is perfectly valid code, and it works. But as I said before, if I am doing this multiple times, it means that every time I come to the update method, I'm going to remake this reference. I'm going to remake this reference, and this reference is lost when update when the update method method finishes because it goes out of scope. So again, here, what I actually want to do in order to be for kind of good code in unity, whoops is not do that. Yeah, Okay. Sorry, public variable tracker VT, okay. And then what's better to do isn't here. I'm going to do VT equals game controller dot Get component variable tracker. Okay, now, just make sure that when you get to the end of here, you got variable track in between the angle brackets, which shows the tightly looking for and you got, like, the double things just show it's a function of our

parameters. So that's made my reference one time, and that will cash that and I don't have to keep on remaking that reference all the time. I know what that means is that actually down here I don't need this. I could just say VT, I'll show you. I'll type it all out to say I can show you that it works. Add to score okay, that's a much better way of doing it. So I'm going to add to my score, and then I want to increase it. So that says that I actually want to have a reference to my console script as well, which I'm going to call CS. Just for this and CS equals game controller get component console script, okay. Remember that each script that you write each original script you write is a component in of itself. But you may instance, across several instances, several game objects in your world so you can treat those just like any other component, whether it's built by unity, whether it's built by you, they're all components. This is part of that game object component model that I spoke about in the previous lecture and in here we're going to do CS and we've already done our function of Prince score. It was a public void function. So just go see us Prince score. And that is good. So we should have a thing that we want to do, actually, something I forgot. CS Say hello, okay. Now final thing to remember is, I want to touch my testing script. Two there haven't actually done that yet, although there's a missing script there from a previous demo that I forgot to remove capsule components. Okay, I can drag and drop onto their I can drag and drop over here. It doesn't really matter, says testing script there. It's going to show none there in the inspector and that's okay, because I'm going to make, well, I'll tell you what actually, I think what I did when I originally made this is I was going to drag and drop. So what I can do is I can drag and drop the game controller over there is a variable tracking. Like he did the same for the console script. I'm not going to do that. Let's make that none. Well, what that actually means is interesting good programming don't need public anymore. It doesn't need to be accessible outside of any script. I've got my get components that you're here. However, what I can do if I wanted to is I can do something called serialized field for both those or for anything actually, Syria list field in the unity in the unity engine means that this variable is private. But you can still see it in the Inspector. And this is a way to get around the fact that, if you make something public, it's automatically viable. And, Inspector, as you can see here already over there because you don't want nothing to be public all the time T, that's bad coding and you want to do defensive code and keep her thing encapsulated lockdown, etcetera. If we restrict his, say the object good object oriented stuff. So unity is going to compile and you can save it to make sure, like thank you.

Play video starting at :12:35 and follow transcript

12:35

So we should have to press play and then you can see over here that has made those references. They're not there. If I exit play mode, they're not there because I don't assign them in the editor I assign them at runtime using start method. To do this, click in the window press space. Hello World Score one Hello World Score to press it twice. So

there you go. So we have the input script, my testing script, speaking to my console script and my variable script. And that's just a very quick example to show you how you can get components talking to each other in unity.

In this topic, you learn about transforms, their properties of 3D objects that allow you to move them around, rotate them and change their size. In 3D graphics you use transforms all the time to lay out objects into a complex, or animate their movements in real time. You will also learn how they are represented as matrices, a mathematical technique that underlies a lot of the graphics techniques. At the same time, Tom will show you how to use them in unity and go about creating your first graphic scene.

We've seen how we can make shapes out of verses and polygons. These polygon measures the objects that populate our virtual worlds and then we can see in virtual reality. But simply having objects is not enough to create a scene. We want to create worlds which have many objects, but importantly, these objects need to be positioned likely. It's no good having all the furniture in a big pile when actually what we want is a nicely laid out room with the sofas in the right places, the coffee tables in the right place and the lighting in the right place. So we need to be able to position our objects where we want them in our environment and, as we'll see later, would be like to be able to move them around as we've seen the positions in our environment to represented by an x a y and a z. And these will be fundamental to how we move things around. The most basic thing we'd want to do to an object is move it left or right up or down or in and out. Those are the three directions that we've learned about already X and Y, and Z that is what we call translation. It's what we think about first when we think about positioning and moving an object, moving it left or right up or down or in and out. But it's no good having your T V in the right place in your room if it's facing the wall as well as your object having your objects in the right place, they need to be facing in the right direction. So we need to be able to turn objects what we call rotating objects and here again, X and YZ They're quite useful so we can rotate an object about X. Think about a big line or a pole coming going through the middle of an object in the X direction. And if you twist that pole, imagine what happens to the whole objects. That's what I mean by rotating about X. We can also do the same about Y and also about Z, So we can break down the rotations into rotation about X rotation about Y and a rotation about Z. Actually, behind the scenes, it's represented in a slightly more complicated way called Kryptonians. I won't go into the details. It's quite mathematical, but there are several reasons why it's a good representation

Transforms, Let us position objects in the world. But they also let us do something else. Objects have their translation rotation, which can put them anywhere in the world. But we can also give them a shared transform. So that we can put them together to form a

single, more complex object. We can have loads of objects with their own transforms. But we can also transform the whole object. So I can move it up or down, left or right, and I can rotate it. I can do whatever I like. Its very much like a group in a drawing package like illustrator PowerPoint. So each of these objects has their own transform, so they can have their own positions rotations inside the more complex objects. But there's this big transform that controls all of these objects and allows them to move together. How we normally do this is to create a new game object, which is empty but contains all the other game objects. This has allowed us to create complex objects easily by putting simple objects together. For example, you can create a table by having a top of the table and four legs. They are each a game object with a mesh and transform of their own. But we can combine them together to form a bigger, more complex game object, that has its own transform. So we can move the whole table around without having to think about how to move each leg.

In this video, we're going to talk quickly about the transform component of GameObjects. Some of the ways that you can manipulate them using the mouse and the keyboard and how you might go about accessing that basic information encode. In the previous video, we talked about GameObjects and components, etc. We talked about how the transform is indeed the only component that you need to have on an object. Just to play around some of the things in the Transform, show some stuff. I'm going to create a new 3D objects. GameObject 3D objects. I'm going to create a cylinder because it's just a little bit different from before. Those are just going to make my scene window a little bit bigger here. As I said before, don't worry about how your window look like, how your layout is. I've got this layout here so that my hair can sit here in the corner. Don't worry about about this window where my head is, but sprite packer, That's just to stop important information being where my head is. We've got a cylinder here.

Play video starting at :1:17 and follow transcript

1:17

Before I zoom in to the right-hand corner actual line inspector than I can fold down everything that's in the inspector collapse all components, and let's just bring up the transform. I'll transform if you remember, has got a position, rotation and scale. Say, if you're going to look at this on our object here, then the way you would interact with this or get this to work is you can press "Q", "W", and "R".

Play video starting at :1:53 and follow transcript

1:53

We have r, w, that's our move patents. The move see the arrows. You can see there's an axis for the y-axis, the up axis. How do I know that's the case? Because the y, x and z are here. That can always be a nice reminder for yourself. If I click and hold that, it turns yellow and I can drag my cylinder up and down the y-axis forward and back on the

z-axis, left and right on the x-axis. Just to bear in mind that these axes may be different to ones that you're used to if you've used up a 3D engines or other 3D software. But y is up in Unity, said is forward x's right, actually positive x is moving right? That's how to drag it by just one axes. If you want to move the object by two axes, then you go in the middle here. But one of these planes that are highlighted in yellow right now. If I click and drag this, this is like dragging it across the floor. It's not going to move up and down on the y-axis at all. Then the same with my, that's not going to move forward and back some z-axis and so on, so forth. If I press "E" to access the rotation, you can see that that is making it into a sphere. Again, we've got green, blue, and red things to drag it around by. Now. Nice little tip here. If you want to make something rotate by, say 45 degrees, 90 degrees and snap it to cardinal directions. If you click and holds, it turns yellow on one of those axes and then you press control. Then you can turn and snap it using increments of 15. Again, let's move to our scale. Just reset those things little bit easier to see our scale if it's the right way up. I press article, scale. Gone w for move, e for rotation, [inaudible] , and I can scale and one way, that way, this way and altogether. That's fairly straightforward. These to say, of course, you can do this from your inspector menu up in the corner here, and you can just input values if you so desire. That's resettle that back to 0. That is s if to refocus. We are back focused on item. Now, in the previous video, I had a little script called Hello World, etc. We're just going to use some of that. I'm going to open that in Visual Studio,

Play video starting at :4:34 and follow transcript

4:34

so-called open Visual Studio now sits bring that in. Here we go. That's our Debug Log that we had from last time. You saw we do this little bit before. But [inaudible] are getting a debug log? Like I said, all there components are connected to the transform. You can access this information in the GameObject. If I just want to access, I could do transform position. It's going to print the text of transform position. Then there's a concatenation there and then transform position. I can do the same for rotation, transform.rotation, Debug Log. Case-sensitive by the way. Not in the text obviously, but for the functions. Now, for scale, you want to use local scale. It's good reasons for that. There should not going to go into now be funny, actually access the scale functions that going on there. Then just use local scope. I'm going to just always make sure got share to with people start in Unity. I mean, you've done this lots never programming, but if you file needs something to save, there's a little star up there. I'm sure this happens in another ID in the upper IDs that you use as well. Just make sure that you have the Control Shift S to save everything. Let's drag, oh sorry that my unities there, I'm just actually wanted to drag my future Studio off the screen. Let's go back to you in to its thought about it. A little shortcut that you can use to enter play mode is rather than having to go up to the corner here and use the play button there, you can just press "Control" and pay. No. That was being impatient. It doesn't work. Should know by now. Why hasn't it worked? I

haven't added that script as a component and my cylinder just going to collapse everything again. But I can now see that my script is there on Hello World. Then now let's go and do it. Let's zoom in and have a look here. We can see that we've got a hello world originally got the position here, so 0 and x, y and z. Then I'm just going to skip ahead to the scale. It's just 1,1,1, which is how it comes out default. Rotation you'll notice that all four numbers, so there's a good reason for that. Rotations can come in two forms in unity, the actual native way of using rotations is something called quaternions. Begins with q quaternions, and it translates from Euler angles into quaternions. Now, Euler's, which is actually spout E-U-L-E-R. Euler angles are x, y and z and really Eulers is, are similar. The essentially these angles that we see when we want to go and focus around our object here. It's the x, y, and z rotation. Y is there for, in this rotation thing here won't quaternions take four coordinates. The reason they do that, there's a four-factor axis there to prevent something called gimbal lock. Gimbal lock is something, is a phenomenon that occurs with Euler angles where you, if there's a certain sequence or rotating in one plane and rotating in this in another plane it can render the third plane completely null and void, so no rotation. The third plane doesn't actually have any meaningful value. Best way to see that is to look up gimble lock on the Wikipedia page. It's got a very nice diagram that illustrates exactly how that works. It's far easier to see the needs for me to explain it. That's all the information you can access in transform.

In this video, we're going to look at the cameras and how it can control cameras in our scene view. I'm back here in my scene, my empty scene from last time, we've got to sit into here now still actually got my hello-world script hanging around here, if you see down in the corner, just here. I could remove that component and that's absolutely fine, but if there's ever a time where you want to just disable something that you can simply just uncheck that box there. If I now go and play,

Play video starting at ::43 and follow transcript

0:43

you'll see that script is referring. Just to give you another option to disable something, but you don't necessarily want to remove it and lose quite a lot. Maybe you've made references in that component. We're going to talk about references later on. But if you set lot of options there and you don't lose those options, they evolved and remove it, you can just disable it. I've disabled that script so it's not firing. I don't need it for this thing. Here's a cylinder and I'm going to also add a plane for it to sit on. I'm going to bring my cylinder up, so holding v here at this point around my cylinder. Taking one of the vertices of the cylinder, just snapping it, just says absolutely flat on that plane and a case is No. I just wanted to give some ideas, just to put some more stuff in there for the scene to frame. On my directional, I'm just going to add some shadows, so now I've got shadows. It's something for us to observe and just take a look at. Cameras in the scene.

We have our camera here represented by that. In the scene view, whenever you select the camera, you will see a preview for that camera. Now, every scene in unity must have at least one camera. You can have as many cameras as you want. If you had a local multiplayer game, and you had four players in this local multiplayer game, then you'd have at least four cameras because you'd have four cameras showing in each corner of the screen to show the players viewpoint. You may have more cameras in there. It depends what you're doing with your game. You can have as many as you want. You could also think about cameras as viewing another part of the game area. Maybe you've played games before where you throw a switch and the little inset shows that a door is opened or a creature is being let out in another part of the level, and it's communicating to the player was happening with that switch, that would be a second camera. You can have loads of cameras, but you must have at least one. That's to be something to render to the screen. When you select a camera, you will see it in the preview down here in the corner, so you can see this wonderful preview I've got at the moment. You can control the camera in the same way that we controlled all things before. Naturally, if I zoom out this bit, so you zoom in a little bit, say I can do my W, my E, and, my R to go from my movement rotation and scale. If I zoom out, you can see in that camera preview that is updating in real time which is nice. If you want to, you can rotate your camera, if you're just trying to line up that perfect shot, etc. Now sometimes you'll want to move the camera, so for example, I want to tip this down a little bit to get a bit more of a vantage going to put it up, I want to take this down a little bit, but my rotation angles do not let me do that. All I can do is like tilt the camera, but none of it's in the right way. How would I go about that? Well, in the top off the screen up here, I'm just going to zoom in for you, you can see that it's center [inaudible] global, if I click that, and so it changes to local, you can see now that the handles on my camera have actually changed. If I do that here, just zoom in a little bit, so you can see if I click global, local, can you see that? Might make more sense if I do the moves. This is the global move and the local move. The difference is that what it says global up here, this is the axes for the entire world, and when I've got local, and that's just the axes just for that object, if I click on my cylinder, you can see it's got different axes. But if I click on global is quite exactly the same. All three objects, the plane, the cylinder, the camera, they've all got the same. So that's something else to bear in mind when you want to arrange your camera or did any objects and you'll see. Now, I can just tilt down and I've got my nice scene that I wanted in the first place. You can also use something called a fly-through method. For those of you who used a first-person viewpoint games, if you hold down the right mouse button in the scene view, like this, you don't see the purple bit. That's just something I've added to my cursor. Let me turn that off for a moment. But you can see here in the middle that it's got an eye with four keys there. What that's showing you is that you can now use W, A, S, D to fly-through. I've lost myself that way [inaudible]. If you find this more useful, you can use this first-person shooter, first-person flyer

viewpoint. Easy to use a W, A, S, D to move around. I'm still holding onto the right mouse button whilst I do this. Q to just go down, E to just go up. To line up that nice, shot for yourself. Now, I'm here, but you can see my camera preview that my preview is still very different. If I did line something up using the fly-through view, how do I set my camera to this? This was nice little shortcut for that. If you click on "Main Camera" and then you go to "GameObject" and "Align With View", there you go. Instantly. So you can line up your view, fine tune it and the main camera would just align itself perfectly to how you've set up the scene camera. Just to double-check this, click to the "Game View" and there's our game view.

Play video starting at :6:58 and follow transcript

6:58

Another thing to talk to you about is the perspective versus the orthogonal camera. Now, I'm going to go up in the corner here and look at this bit here. You can click on this bit "Perspective to Isometric". Now, what difference does that make? Quite a lot. There's your perspective. Perspective means that things will get smaller as they recede from the camera. But then for some game views, you don't want that. Sometimes for a map view or you're trying to make a game in a isometric old school fashion.

Play video starting at :7:31 and follow transcript

7:31

Just to show you what's going on here, if I click on the "Isometric", you can see that it's not getting any smaller as it goes further away from the camera, but it is still moving in three dimensions. Which, again, depending on what your game needs, that's useful. I'll just show you the same thing again. Perspective, it gets small as it goes away, etc. If you want the isometric or the orthogonal camera, perspective camera, then you can do that. You can also check it. Let me zoom that again. That's my camera and that is changed here. Under Projection it's got how the camera renders perspective. That's another thing that's worth bearing in mind. You can hover over each of these and it will give you a little tool tip. But perspective versus orthographic. You won't see in the scene view, but you do see that in the game view. Sometimes you want to keep them separate because you want to navigate a bit easier. You're trying to select something. This is the switchbacks perspective.

Play video starting at :8:46 and follow transcript

8:46

That works there. Sometimes it's useful to click on these arrows here, and then you can click in the middle.

Play video starting at :9: and follow transcript

9:00

If you click on these directions, you can see it changes here, says top and a view from the right. Then you can go back to perspective, but this will automatically. Now if you click the middle toggle between perspective isometric. That can be useful for viewing

things from one direction or another. If you want to break that hold down your turner and then you'll be back to your view. Just bear in mind that up here in the corner, that will rotate with you as you move around. You see that all the axes moving around. If I click to z and view straight on you can actually see that these two are not perfectly bang on. Hold down your turner and back to my 3D tumbling panning and zooming.

Play video starting at :9:55 and follow transcript

9:55

Just worth mentioning as well up here, the 2D option, so toggled scenes in 2D view when it's toggled off scenes in 3D view. Now, if you're making a 3D game, you're not going to use this quite as much. If you're making a 2D game that is enabled by default. Plane doesn't have any mass, so it's not rendered here. That's rendered by default. You'll also notice that there is no gizmo up in the corner here where that was before. If I click off 2D, my gizmo has come back. If I go back to 2D, now gizmo has disappeared. This is how your game would render if you making a 2D game. Why would you want to turn off 2D? Sometimes you're just trying to find an object, you're trying to work out where they are in the z-axis versus the camera. Let's just give you another option. You can turn lights on and off as well in camera view. If you're working on particularly dark scene, that can be useful because you turn off the lights, and if it's dark, then everything becomes flat-shaded like this and that, again, helps you to navigate around.

Play video starting at :11:9 and follow transcript

11:09

Let's look at the actual inspector and what's going on in inspector so it's got transform like everything else does. The camera here has got lots and lots of properties. Some things I'm going to mention in particular that come up earlier on. Your field of view is changing essentially how much information the camera can receive. Now, we can fit in our camera preview just down here, and if I change that field of view, it was 60, like that, before I changed that and broadened it out. Those who play first-person shooters might be used to doing this, by changing your field of view to about 90 so you can see more of your environment and therefore react a little bit quicker. So that's all that does. It's almost a bit like a zoom in your eye. What else do we have? Oh, yes, I want to talk about the Clear Flags. You can see in the scene of view that all you have in the background is a gray box. But you can see in the game view you have this funny horizon thing going on here. This is what the Clear Flags over here means. Let me just move that so I keep my camera preview in there. What this [inaudible] is a skybox. If I click and go solid color, you can see it's a solid color and then actually, you'd be able to change this using the dropper tool. I'm going to have to go out here to find the dropper tool. Let's just click on that. My color palette is here, let's pick red. Now you can see that. Oh, I guess it's a bit hot. Let's just save our eyes, shall we? So don't burn our retinas. There I guess is a bit more dimmer.

Play video starting at :12:53 and follow transcript

12:53
What is this skybox anyway? A skybox is very simple. It is a cube with textures that face inside, within which your game world sits. So the cube sits outside of your game world. Next time you're playing a 3D game, look up at the sky. You're not looking at the sky, you're looking at a flat texture. It's a flat texture and a very useful, very efficient in terms of resources, to give an idea of cohesiveness to the world and certainly, an idea of distance, and that something is far away. You can change the skybox to show things in the distance without having to render all the geometry. You can just show a flat image of that, so a city on the horizon or similar to that. At the moment, that where the blanks are filled in, all these blank bits, they're outside of my cylinder and my plane. They are being filled in by the skybox. You may want that, or you may want a solid color, in which case that's what you see when you run your game. It is up to you. Skyboxes by themselves involve a lot of options, I'm not going to go into those now, but hopefully, this has just given you a quick rundown of how to manipulate your camera, and how to get it to move around.

In this video, we're going to look at lights within Unity. I'm back here in my basic scene with just the defaults that come up with Unity. You've got the main camera and you've got directional light and these are the two things that always come in if you create a new scene from scratch. I made a little object earlier just to help us illustrate how some of these lights work. Called room is going to drag and drop that in there. We can see our room comes up in the middle there. There's our Game View, which isn't what we see here. Main camera, there you go, says, the Camera Preview. Remember last time we can align the camera with the view in order to set up a new view for the camera, then you can do the same the other way around. If I wanted to go to how the camera's right now, I can select "Main Camera" and then up and top-left corner here I can go GameObject. Align View, we've selected, so before, we were doing Align View, but if you do Align View to select it, then here we are, I'm in the same place as my main camera. I don't particularly need to see my main camera at the moment. Again, actually, up in the corner here, what I can do under my Hierarchy, you see there's an extra two columns to left. This is to show height and this is to lock that item. This is when you've got lots of items in a hierarchy and if you want to not accidentally click on something and drag it when you're trying to click and drag something else, you get those things in Photoshop, etc. all the time with your layers. Then you can lock them there. But at the moment, I just want to hide it. I'm going to hide my camera. Let's have a look at this first thing. We've got my directional light here. Directional lights can best be thought of as like the sun. They shine light from everywhere in the scene down at a certain angle. Now, position doesn't matter, scale doesn't matter, but rotation does. Let me show you. I'm just going to make my view a bit bigger for this video because I don't actually need that console. I tell you what, if you have one, get rid of window. Just right click. If I'm here on my console, I can right click here. I can just do close tab. There you go, I've got

a bit more window space now. Let's have a look at our direction window. Now, if I drag that in, at the moment, actually, there's not a lot to show you because I'm not seeing any shadows. Now, directional lights may not come with any shadows on by default, so what you do is you select your directional light up here in the Hierarchy and then you're going to go to the Inspector. Then here, now, inspect for directional light. See Shadow Type. Now it shows you can do hard or soft shadows. If you have lots of shadows going on in your scene, hard shadow's a bit less expensive to display on screen, but we haven't exactly got huge amount going on there, so I'll go soft shadows. Now we've got soft shadows.

Play video starting at :3:9 and follow transcript

3:09

Back to our directional light. Directional light, if I drag and move that around, you can see it makes no difference at all to the shadows. If I press off my scale and I scale it, etc, makes no difference whatsoever. But if I go to grab and rotate it, there you go. Move and scale don't matter, rotation does. As I said before, imagine this is like the sun is the only present source of light. You can do all the usual things like changing color. If I click over here, so I'll zoom out again, it gets your color wheel. Just be sure that you selected the blue. You think you've selected a blue. Darn, why isn't it blue? Because you didn't selected blue. Just be aware that some of these color wheels, you can change how this works to your liking. This is the color you'll be choosing, but you're still choosing white here. There you go. Window will change in real-time. There you go isn't it nice? Fetching pastel blue. There's a directional light for us. Holding my right-click and using my fly through the WASD keys, let's go into this room that I built before. We can see that there's some ambient lights going on here. Now, what I might want to do is I'm going to turn off the directional light. I can't see a directional light, but it's still there. If I untick disable the directional light, that's not viewing anymore. There's still some ambient light coming it, and that's because I've got a lot of ambient light coming in from my skybox. You can turn that off. We'll turn it off later, and to get some real examples of that. But for now, I'm just going to carry on showing you some lights. Point lights, in GameObject and then Light, Point Light. There's a point light.

Play video starting at :5:14 and follow transcript

5:14

It's just there, can't really see huge amount because you have to be careful with these things. But we'll show you where it is, but you can't always be entirely clear where those things are. Now if I hold V to snap it, so bring it somewhere up to where we are. Here we go. You see I snapped its one of the vertices on our plane. Just focus on it so we are orienting, round it a little bit more. You can see that that point light is making a difference and just to underline, then make it red. Great. Here, rotation doesn't make any difference. Scale doesn't make any difference, but location does. It's literally just this point here in the middle. Just that point there in the middle which is illuminating. You can

play around this. You can increase or decrease the range. If I increase the range like that, then you can see that light would actually spread out quite far. You can see it's just going on the ceiling there before bringing it down to one. Can't see shining on anything unless I start to bring that closer to the wall. Now, you can see that starting to shine off of the wall. If I wanted to make that brighter, you up the intensity. There you go, actually hasn't got much range, but if I up the range you see, the net becomes the range in intensity. You play around with those to find the effect that you want to have with your light. I put some extra objects in there so you can see is starting to bounce off some of those other objects. Once we're in this place of darkness, let's bring in one of our other kinds of lights. Again, I'm going to go back up to "GameObject", use "Light", and then the "Spotlight". Where's my spotlight? It's there. I'm also going to use the same thing that I always do. Hold V and then snippet on these vertices then just bring it up there. Now, let's just put it about here maybe flowing through that wall. Maybe could've made the room bigger, I'm sorry. There's our spotlight. You can see it's got a cone to it. Again, you can play around the range. It makes it shorter. You can see it's actually dimmed now because I made it shorter, it's because the intensity will decrease over the length of the spotlight. If I wanted to put it there, which seems pretty good. If I wanted to make that brighter underneath it, then I would wake up the intensity. You can see that drops off faster as it goes away. Again, it's a whole things for you to, the realm width leave that at 50, I'm going to make that less warm fetching orange. Here, scale doesn't matter,

Play video starting at :8:44 and follow transcript

8:44

but position as you might expect, does and so does rotation, but beware of this. You can see that, that light is rotating width fixed. It's rotating I from one surface to another. Look, why can't we see it going up the wall? Well, it's too short. Use this as a guide to guide you. You can see if I move that in, then now I'm rotating it, it will actually go on both of those.

Play video starting at :9:17 and follow transcript

9:17

It's still missing out that corner. Still a little bit too short to hit that corner. Here you go. Now, you can see that light, it won't reflect there. See you get the idea. Spotlights, different utility so only goes in one direction for a fixed code. You got one version, one light left. "Light", "Area Lights". That has come up right in the middle of our room. Interesting. Now an area light is usually a rectangle and you can see that we can change the shape of it can be rectangle, it can be disk. You can change the radius of it. If you got a rectangle, you can change the width. I'll have do that so you can do that. If you watch these numbers here, got some dragging on the outside of this rectangle you can see there's a change in C, can change in the inspect to change it in by I, depending what works. Now, we're not seeing anything here. Maybe I haven't put intensity up enough.

Play video starting at :10:29 and follow transcript

10:29

Is something a bit wrong here? We're going to give it a color so that when it works, you can see that light up, bright green. Why am I not seeing any bright green? Is emitting from the right, from that side. No. What happens with an area light.? An area light is a flat plane which emits light from one side. It's the thing that would be analogous to lock a panel light. It's got sunlight redirection. We've got a light bulb effect with a point-like, we've got a spotlight which is a spotlight, and then you've got an area light which would be lucky when those LED panel or maybe those lights that you see at football pitches. It emits from a whole area. Why isn't this working when it comes down to real-time versus bait lights? In Unity, there are two kinds of lighting to be had. There's real-time lights, which change in real-time as the game progresses and then there is baked lights. The reason why we need these two things is because light is very costly for a computer to calculate. With baked lights, if you wanted to have some very nice realistic lighting, you would arrange your lights at just so and then you essentially click a button and then the computer will spend time baking in those lights. What that will do is work out how the light bounces off all the objects in your scene and it then will save that information as something called a lightmap. The lightmap is really a bit like a texture that goes over the other textures you already have in there that describes how to display the light to the renderer. But that has to be done one time. Those are the things that don't move around. It's good for the panel lights I was talking about static things that don't move. But if you wanted something to be moving in your game, say the siren light on the police car or the flashlight on the torch, then that would have to be real time. It's up to you where you make this [inaudible] between the two things. How did we get it to bake? In the very bottom right-hand corner of your window, you can see you've got this auto-generate lighting off. Now, if you click that, let me zoom out, what you actually do is bring up the lighting settings panel. What you can do here at the bottom, you can have auto-generate on but I highly advice you don't have that, because what that means if you have auto-generate on, it means that the computer will be re-baking your lights every time you shift something in your scene. That could take quite a long time. But if you just want to place a box and then generate it, you can bring this up to you can just do generate lighting. You can see it's in the bottom right-hand corner. It's doing some jobs and now it's worked out. It is now. That hasn't given us much effect there and it's not going to update. I'm just going to make sure that is properly there. I'm just clicking generate lighting off screen here.

Play video starting at :13:42 and follow transcript

13:42

With area lights, there's two things I said, that it's got to be baked lights, and they are about baking in objects that aren't going to move so you can have nice realistic lighting around your scene. Of course, the issue here is that Unity needs to be told what objects

are not going to move. That's why we're not seeing any lights here at the moment. If we go into the top right-hand corner of your inspect, you see this little box on every object. This is static. You click static. Then we go and rebuild our lights again. Just go click on generate lighting and our little bar is working down the bottom here. One final thing.

Play video starting at :14:29 and follow transcript

14:29

I've deleted my area light. If the only light I've got now is my spotlight and my point light and my directional light is outside of the room. You can't see anything from my direction light, if you remember, that was blue. Was it blue? Yes, it was blue. Then why is it I'm still seeing a load of light in here and even if I deleted my directional light, I'm still seeing a load of light in the room. What's happening there? Let me bring my lighting panel back that we've been using to play you some options. I'm just going to doc it next door to the inspector for the sake of convenience.

Play video starting at :15:7 and follow transcript

15:07

By default there will be a lot of ambient light in your Unity scene. The ways you can turn this off if you really want to, is at the very top here, you can go to the environmental lighting. Let's just zoom in. You can go to Skybox then you have to choose something from here. Leave it on Skybox , but bring this intensity multiplier down. Already we can see and in fact, let's go up here and switch to the game view because we don't need to see the outlines, we don't need to edit anything. We just want to see how it looks. We've doubled down this intensity multiply here from one down to zero. You can already see that this is making a difference in our lights that we [inaudible] stand out a bit more. If you want to take that even further, you don't want to see the edges of the [inaudible]. You really just want to see the lights that you've put in. Then what you can do, you've got environmental reflections here, you can drag that down to zero as well, or if you're leaving it as one you can just click on Custom, just fit it with nothing, there's no keep map there. That's not even skybox and it's drag it down to zero. Now we can see that the only light that we can see our spotlight and our point-like that we put in earlier. Fun thing. If you're in the scene view and you want this look at your game, but you're trying to move around the editor and you can't see a difference between the wall and the floor here and you want to be able to placing something, you can quite easily go zooming up here and then just in these case at the top you see this light bulb. Click that. That will give you an unshaded view. It just means it'll show you flat textures and flat materials without any light play on them. That just makes everything looks notes globally evenly lit. No character, but really good for editing. If you need to do that, then you can zoom too , your game's looking like, then you can do that. We're back now. You can see that this has completed. This took about three minutes of time. I'll locally that and get it for you, it's about three minutes at a time and you can see this was a green area light. If you remember, in my color over here, for the area light, this is what I did. You zoom out.

This is so intense here because I went, I'd wake the intensity up to a 100 because I forgotten to make it static. Then it's green around the here. This is the area that it's happening here.

Play video starting at :17:42 and follow transcript

17:42

I'm going to check making a difference that every bait. There's the area light for you. You've got variety, different lights. Be aware of the demonstration, bait lights and real-time lights a lot of the time, essentially in your scene can have a mixture. Those are the three modes that unity can running. You've got bait lights, real-time lights, and the mixture of them. Be judicious and careful with your baking lights because it takes a long time to render those. But it is the way to get really nice lighting without really harming your frame rate during the actual running of the game itself. If you're having lots of objects that got light on them, you need to make a choice between performance and how long it's going to take to bake those in.

In this video, we're going to look at prefabs, one of the most important concepts in unity. Okay, so I'm here in the blank scene and for the time being in the top left-hand corner I have hidden my main camera in my direction, like because I don't need to see him at the moment for this scene. That's not what I care about at the moment. So let's go ahead and go and game object in fact, another way, I don't have to go to the menu I can just go in the hierarchy and right click and then do 3D object. Let's make a capsule. So a capsule nice shape to use for box standard enemy. So if I was making a prototype enemy just to move around just to get some behavior going, I'd make a capsule. And then I'd probably just make another capsule, because when you make an enemy, it's a good idea to know where the enemy is facing. So what I tend to do is I will give my enemy a nose. Now I'm going to go, so here my hierarchy click on Capsule, Press F2 go enemy and then the second one that I've made smaller call that enemy nose. What I can do is I can drag keep holding down your left mouse button, drag and drop onto the enemy, and you can see that now enemy nose is a child of enemy. This is one way to not only organizes your hierarchy, but it's useful for grouping objects together for movement, etc. If I now click and the enemy nose and if I now go to the top of the transform here and click reset, you can see, actually, if I go to WIREFRAME.

Play video starting at :2:2 and follow transcript

2:02

Go to enemy, my enemy nose is actually oop no, did that work? What? So you don't do reset apologies. If you do reset, then what I've just done is I've reset all the transforms, the rotation and the scale. So actually, the best way to do is what I wanted to do is I wanted to zero in on the parents, so it's easy to rotate and get that object in place. So do it by hand with position. So now I've done that. Let's go back to shaded. So it's there.

And if I go to rotate, click and then hold control well, I can rotate by 90 degrees exactly. Cool. And let's just move that up.

Play video starting at :2:52 and follow transcript

2:52

Move that in, there you go. Now he's got a little nose. It's in the say. The thing I would also say is be careful of your directions. The reason why I just moved this is because the blue arrow on my enemy is worth the enemy's forward. So if I'm using this as an indicator of where the enemy is facing, I want this.

Play video starting at :3:23 and follow transcript

3:23

I get it out there, it's about level. That's good enough. I want that to be along the blue forward facing positive blue axis, which is forward for my enemy, okay? All right, so here's our enemy with a little nose sticking out. Great. Now the trouble is what if I wanted several of those enemies? What if I wanted to instantiate or copy, make loads of copies of that enemy? Well, we use a concept called prefabs. So often what you would do immunities in your project folder. You'll come here and you will create a folder. You don't have to do this but it's generally good to stay organized. Good idea to make protocol prefabs. And then from my hierarchy up here in the corner, I'm going to drag and drop enemy onto prefabs, okay? So now you can see that my enemy is in this prefabs folder in my project folder. And then notice also, I wish it was clearer, but it's not, and you can't change this. But the text has gone blue in the hierarchy window. That means that the enemy is now a prefab. And you can also confirm this because there's an extra line in the inspector here saying there is linked to a prefab. What is a prefab? A prefab is like a cookie cutter, and this particular enemy is like a cookie from that cookie cutter. It is like classes and instances, okay? So it now means I can delete that enemy, and I haven't lost it for my project, the enemy is in my project, even if it's not in the level right now. But I could also now just drag out as many enemies as I want, if I wanted to have lots of them and they'd all be the same. Now, other things are also very nice. So you can click on one of these enemies, any of these enemies. And if you look up in the inspector, you can see it says open. So we open the prefab. So this is your prefab editor window. Now maybe, just maybe, I thought well, actually, I didn't want to give him a wonky nose, and it is off a little bit. So I can go up here to position, and I'll just make his nose a bit more central. There you go. Let me zoom out my window. Zoom in here. What I can do not I say so I can look down from above. You can see that's absolutely bang on now. So just play to my camera, using the square in the top right hand corner here, okay? We should go back to this. I don't want to see a camera. Okay, so I can do that. And let's just make it a little bit clearer. What's happened? Let's just move that nose in a little bit. And I can go back just by pressing in the top left hand corner the arrow here, go back to my scene and do you see hell, all of my I'll tell you what might made me to speak there. If I bring back my directional light, where's my directional light?

There we go. If I rotate this around, and just illuminate the front of my, right I can hide it again. Just illuminate the front of my enemies so you can see that those changes have gone across all of those prefabs. Now you don't have to go into prefab mode not to do that. If you clicked on a prefab, so I happen to selected this one, then what I can do is if I made any edits to the nose. So I'll push that in a bit too far, actually going to bring that out a bit more so it's a bit clear, so I'm not going to bring it out even further. So if I'm looking from the top down view, I can see where this enemy is facing. Then I've moved that nose there. If I click on enemy, I can go to overrides and then I can go to it will show what has changed, so even says click on individual items to review, revert and apply. So this is showing me all the changes that have happened to the prefab so far that haven't been made a permanent part of prefab. So it says I've changed the transform of the enemy nose. If I click on it, I can get all these details, okay? But it was before and what it is now if I could apply all there you go and again on all of my prefabs here the properties have changed. Okay, so let's delete all of those. So something else you might want to use prefabs for a lot is if you want to instantiate something from a script. So, for example, you wanted to create a bullet that fires from a gun or you want to create an enemy that spawns from a certain position. So let's just make a quick script to show how we would use prefabs in this context. So I'm going to create a C sharp script. I'm going to call it Spawner, and I'm just going to drag and drop into my scripts folder.

Play video starting at :8:42 and follow transcript

8:42

My computer is going to be a bit slow. [INAUDIBLE] Let's double click on Spawner and let's bring in my script, okay? I'm not going to put any logic in here, I just want to show you something. So if I was to make a public variable, so I can make a public game object.

Play video starting at :9: and follow transcript

9:00

Hold on. That is a classic mistake that people make. I'll just put my cursor in the wrong place. Right within the class, obviously. Public game object again, using intellisense to see if you're not getting something you should expect then that's why game object, and I'm going to give it a name. This can be enemy Klein. Okay, so that's my Spawner script. Just take it back over there. I'm going to drag and just wait for unit2 to sort itself out. I'm going to drag and drop this on my main camera. Okay, let me just so like we said before, let's just collapse all components and look at this one thing. So now you can see here that enemy clone, and it automatically puts in the space for you when you use camel case by the way. The enemy clone is a game object but nothing has been assigned to it. Now, here's a common area that you'll see, and I'm going to bring up the console window again. And to do that we go to window at the top and it's in General console. There's my console because I removed in my last video I didn't put it back. So let's just run this.

Play video starting at :10:15 and follow transcript

10:15

Okay, so actually I expected there to be an area there, but I'm not calling on that enemy Spawner, so there's no assignment there. We'll get into that another time, actually. So with my Spawner on the enemy camera, I wanted to say I want to have something, I'm going to press space. I'd write the logic to say when I press space, then Spawn Enemy Clone needs to know what enemy clone is. Okay, so in my prefabs here in the bottom left of my project, drag and drop that into there and now it has made this link. You can only do this for prefabs you can't do it from the hierarchy. It's got to be from the project window with a prefab. Then you can drag and drop that. And now every time I press space, if I program that logic, I will create an enemy that will be that same enemy prefab. Often you'll see errors in your console window saying a variable is unassigned. What that means is you created a public variable in script and you've got it here. And actually it hasn't been assigned yet, like that. The other way you can do this most people drag and drop. You can click on the radio button there, and that will bring up the select game object thing here. You can go to assets. So there's stuff in your scene or you can go to assets and your enemy, okay? So I've made one before, so that's why I've got, [LAUGH] two there. I forgot to delete the one I did for another video. Okay, so there is prefabs, really useful way to make your coding efficient and to organize yourself.

Matrices
Transcript missing

You can also multiply two matrices together, and the result will be another matrix. However, there is something special about matrix multiplication. You cannot just grab randomly two matrices, and multiply them. There are rules about the dimension of the matrices that can be multiplied together, and also in which other that's the multiplication applied. Basically the number of columns in the first matrix, must be the same as the number of roles in the second matrix. For example, here it can multiply a two by three matrix with a three by one matrix. And resulting matrix, which is called the product matrix will have the number of rows from the first and the number of columns from the second matrix. So the matrix product of this operation will be a two x 1 matrix. Let us see another example here. You can multiply a two x 3 matrix With a three x 2 matrix. And the product matrix will have the dimension of two x 2. In order to determine each element of the product matrix, we multiply rows from the first matrix with columns from the 2nd Matrix. So, for instance, the very first element of the product matrix C 00 Is calculated with the following steps. First, we have to grab the first roll of the first matrix. In this case we have three numbers, A 00 A 01 and 02 and then we grab the first column from the second matrix. Which, because of the rules of matrix multiplication, is

guaranteed to also have three numbers, B 00 B 10 and B 20, CO is then calculated as the sum of a old times B 00 and A A one times B one and a 02 times B 20 In order to calculate the 2nd element in this product Matrix CO one. You use the first roll of the first matrix, and then the second column of the second matrix. Here, C 01 is calculated as the sum of a. O times B 01 and a 01 times B 11 and a 02 times B 21 The 3rd element of the product matrix C10, is calculated using the second row of the first matrix, and then the first Column of the 2nd Matrix. Can you try to write down the equation for c 10. How is CO1 one calculated using the second role of matrix one and the second column of matrix to see 11 is calculated as the sum of 81 old times B 01 and a 11 times between one and 812 times B 22

Introduction to Transformation Matrix
Transcript missing

Let's take a look at how transformation matrices work with scaling. For scaling, we just need to scale the x, y and z value separately. To calculate x prime, we can scale x with M_00, and y with M_11, and z with M_22. Basically, here we only need the diagonal values of the transformation matrix, to achieve the scaling operation. We can easily just set the rest of the elements in a matrix to zero to achieve this. This type of matrix, where its entries outside of the main diagonal are all set to zero, is called a diagonal matrix. We can use a diagonal matrix to perform the scale transformation. If you remember, there are two different kinds of scaling, uniform scaling and non-uniform scaling. For uniform scaling, only one scalar is needed. The diagonal line should have the same value. When you apply this transformation matrix to each of the vertices of a cube, you will get another cube, which could be a bigger or smaller one. You can, of course, also use a matrix with different values, in its diagonal line. This way you're performing a non-uniform scaling, which means that after you apply this to a cube, it will become a rectangular cuboid. Here, the changes apply to both the size and the shape of the original object.

You can use matrices to describe the rotations as well, so we can use matrix to rotate vectors. And in fact we will have different matrices for rotations around different axes. The matrix rotation around z axis has a first row, of cosine theta minus sign theta zero. The second row is sine theta, cosine theta zero and the last row is 0 0 and one. This way the components in the new vector are x prime equals cosine theta times X minus sine theta times Y. Y prime equals sine theta times X plus cosine theta times Y. And as we're rotating around z axis, the z value remains the same. So we have z prime equals z. for the rotation matrix around Y axis. The first row is cosine theta zero sine theta. The second row is 0 1 0 and the last row is minus sine theta zero and cosine theta. The components in the new vector will be X prime equals cosine theta times X plus sine

theta times Z. And as we're rotating around the Y axis here, why prime equals Y? And Z prime equals minus sine theta times X plus cosine theta times Z. Finally for rotations around X axis, the first row of the rotation matrix will be 1 0 0 the second row will be zero cosine theta minus sine theta. And the last row is zero sine theta and cosine theta. The components in the new vector will be X prime equals X. As we're rotating around the X axis. Y prime equals cosine theta times Y minus sine theta times Z and Z prime equals sine theta times Y plus cosine theta times Z.

Play video starting at :2:27 and follow transcript

2:27

Very often when you rotate an object you will be rotating it not only around one axis, but two or even three. In these cases it is important to know that for rotation order matters. For instance, here aren't rotated arrow 50 degrees around X axis 120 around y axis and 80 around z axis.

Play video starting at :2:53 and follow transcript

2:53

So if you first rotate it around x axis by 50 degrees and then y access by 120 and finally z access by 80, you Will have Arrow one. But if you were rotate it first around y axis by 120 and then z axis by 80 and then finally x axis 50, you will end up with arrow two.

Play video starting at :3:17 and follow transcript

3:17

And if you rotate it first around Z axis by 80 and then X axis 50 and then finally Y axis 120 you have an arrow like the third one basically. Although the rotations you applied around X, Y and Z axis are the same value, but the order in which you apply them will change the result of the rotation.

Play video starting at :3:41 and follow transcript

3:41

We can apply a set of rotation matrices to one object. So if we want to achieve the same effect as we're seeing in arrow 1, then we'll need to first apply the x rotation matrix, which means the x rotation matrix has to be immediately next to the vector we're operating on. Then we apply the y rotation matrix so the Y rotation matrix is just in front of the x rotation matrix. And finally we apply the Z rotation matrix in front of the Y rotation matrix.

Play video starting at :4:15 and follow transcript

4:15

So from left to right we have that Z, Y and X rotation matrix. This is because we want the X matrix to be the first one applied on the vector followed by Y, and finally Z. And in reality when we conduct this operation, we will use one single matrix to present a combination of X and Y and Z rotation. And this rotation matrix, which does the rotation around the X, and then Y. And then Z will be the multiplication of the three matrices Z

times Y, times X, which is in the opposite order of the rotations you want to apply to the original vector.

Now we have learned scaling and rotation. There is one final transformation left, translation. How do we do translation using matrices? For translation, we're actually just shifting each component of a vector of a certain value. All we need to do is to update each component with a constant. In order for this to work with our transformation matrix, we need to expand it to a four by four matrix by adding an extra column and extra row in the end of the three by three matrix. The additional column basically stores the translation value for X, Y and Z. The result of the transformation will be a 3D vector with X prime equals X plus t_x Y prime equals Y plus t_y and Z prime equals Z plus t_z. Now I know how to do each of the three operations translate, rotate and scaling with matrices, but here is one more thing we need to talk about.
Play video starting at :1:21 and follow transcript1:21
That is the order of transformations, similar to how rotation around different axis worked, the order of transforms also matters. Rotating first and then translating will give you one result while translating first and then rotating will give you a very different one. We can rotate 90 degrees around Y axis and then translated along the X axis, you will end up at a position very different from if you do the translation first, followed by rotation. It is usually more intuitive to rotate first before translation. Typically, you would have the rotation matrix right in front of the vector, and then the translation matrix further away. This also means that if you want to use one matrix to represent these two operations together, then this new matrix should be calculated as a translation matrix times rotation matrix. In fact, the most intuitive and common way is to first scale the object, then rotate and then finally translate it. The order of operations you should apply to an object is scale, rotate and translate or S,R,T. However, as we have learned previously with rotation, you might want to use a single transformation matrix for all three operations. In this case to preserve the order of this transformation, this matrix will be calculated as T times R times S. That's all we have about matrices. Now I have a question for you, why do you think we need matrices for 3-D graphics?
Play video starting at :3:25 and follow transcript3:25
Matrices are a neater way of representing transforms. It uses one single format to express different transformations, and you can concatenate multiple matrices to represent your final transformation. The introduction to matrices might seem to be making things more complicated than it already is. However, keep in mind that most of the examples I use here is based on operations over a single vector. In 3D computer graphics, normally you're dealing with thousands and millions of vertices and your 3D scene. If we're talking about real time applications such as gaming or VR, we need to update everything, 30-90 times per second. That is why you often need a top of the

range graphics card and also called GPU to support this. The use of matrices turns complex calculations into many independent but simple operations which can be dealt with very quickly in parallel, which is what GPU's are designed for.

We've got here a basic, plane and a play that I bought from Novice Edges already premade, they only had twice as high on doing that, I just had a thought based on what was talked about before. For some of you who are more experienced with 3D programs. If you are confident with 3D programs and you're happy building some basic level geometry within you rather than bringing it from outside. If that's your thing, then I will just remind you that there's a package called Pro builder that was bought by Unity a few years ago. That's give you a lot of pretty solid 3D manipulation and sculpting tools. If you want to do that, then what you do is you go up to window in the top here, then you go to Package Manager. Then in Package Manager here, you just wait for it to load, it's going loading packages. Then you will find Pro builder here and click on Pro builder.
Play video starting at :1:11 and follow transcript1:11
It's just taking a bit of time to do it, and you click install. Pro builder is way outside the scope of this course. But it does have a lot of very well-made tools there. Peter, do the things you normally do in 3D Studio Max or microwave or blend away if you're used to in terms of faces, vertices, edges, edge loops, etc, extrusions, and more heavy. It's just a little thing to tell you what's there. Back to the main thing here. I'm just going to be rocking with very basic primitive things. I have my player and I have a plane that it moves on. Although I'm interested in and I have moved my camera so you can see what my camera is doing there. I'm going to make my console window bit bigger so don't need it at the moment. We're going to go ahead and open up my played movement script. I just made a new script ready. Double-click and open up in Visual Studio. I'm going to use a couple of variables for this. I am going to have go and click in there. Lets start, I'm going to use serialize field. Pretend to have a habit of doing that just so we can say again, synchronize field is so that you can see the variable in the Inspector, but it's not actually publicly accessible from outside the script. You get the best of both worlds. Serialized field. Got one float for movement speed. Then I'm going to have one float for rotate speed. Now, usage will become apparent for you to see. I'm going to skip over Start. Let's go to update here.
Play video starting at :2:51 and follow transcript2:51
I'm just going to go to update their and so that Start here. So floats movement, I'm just going to create a local variable here. It's just a temporary thing and that's going to Input.GetAxis(vertical). Now let's just type it in, then we can see and we can go through what this is turn equals Input.GetAxis(Horizontal). What these two things are, I'm setting up a float called [inaudible] and turn. The first one here, this first float movement it goes Input.GetAxis(vertical). What this is, it's looking at the input class of unity and it's saying, Is there any player input coming in at the moment? Now, how do I know that we've got

this axis called vertical, that I can call? Good question. Let's go back to Unity. If we go up to it's just compiling that edit. Then go to Project Settings, then we can go to Input Manager. We'll zoom out here so you can see everything. Input Manager and then here you can say this is where you can configure the controls and you've got axes, you drop down that, and here you go, these are all the things that unity has, and this is default here. There are 18 different axes and he go for horizontal and you have vertical. I'm not going into all the different things here, but some of those are fairly self-explanatory. But what you can't see, he's got negative and positive button, it's left and right or A and D. Then for vertical, it's up and down, and then S and W. So you got WASD and the arrow keys afford backwards left and right movement. The vertical s the forward and backwards, and horizontal is left and right. I am using them for moving forward and backwards and rotating rounds left around to the right. In those two statements, I'm grabbing those values as they're being input into unity. What we going to do with those? We are going to use a command called translate. Transform translate is a command in the transform library that will move an object. By the way, this has nothing to do physics at the moment. We are just moving the position of an object without using physics. That takes a vector. We are going to have to do new Vector3. I'm going to make that 0, 0. Then that's going to be movement, because that's a value. Then times movementSpeed times Time.DeltaTime. I'm going to type out the over statement, and then I'll come back and look at these.

Play video starting at :6:21 and follow transcript6:21

What is going on here? We transform this line here. We transform.Translate. What I'm doing here is, I'm describing how the forward and backwards motion of the objects is going to work. It takes a vector. This is a new Vector3, so this is the x, y, and z axis. Remember that positive z is forward, so for pressing up, we're going to move forward. If we press down, we're going to move back. This movement which is just a single float value, goes in here as part of this vector. Vector3 times movementSpeed, which is what we defined up here earlier, times Time.deltaTime. What is that there? If I zoom out a little bit, you can see the completion time in seconds since the last frame. Whenever you use movement, you want to use Time.deltaTime. What this does is, it makes the game frame rate independent. Now, if you didn't have Time.deltaTime, what could happen is that if a computer has a faster frame rate than another computer, that object will travel further in the same amount of time than the slower computer. If you have a computer that runs at 30 frames a second, and a computer that's running game at 15 frames a second, and you only had the movement times movementSpeed, it means that if there's 30 frames happening, and update happens every frame, then the object and first computer game will move twice the distance as it would do in the second computer game. You want to have these frame rate independent. If you use the Time.deltaTime value, what it does is it will look at the time between this current frame and the last frame, and it will multiply that. What you would get is, if you have the computers run 30

frames and computers running at 15 frames, the computer would smooth out and calculate from A to B the 15 points for that objects move as opposed to the 30. But in both examples, the object would move the same distance. That's frame rate independence. You always want to have Time.deltaTime for that thing. Then we've got similar for rotates. Rotations just rotates around an axis. In this example we're rotating around the y-axis, so y is pointing up. You're rotating around the y-axis. It's taking the turn value that we saw here. Then I've just got a different value for rotateSpeed, and then we've got Time.deltaTime. Control Shift S to save that. Let's go back to Unity. You're just going to compile, drag and drop our player movement onto play out. I'm just going to check that I haven't got duplicate things going on there. Don't actually want the rigid body. Let's say I'm using the prefab for the moments actually, because I can't delete it. Going to remove that. It'll remove the box collider because you really shouldn't be using Override. It's just a player, it's fun. Really shouldn't be having any physical component on something we're using Transform and translate. Just got that one script left, drop down. Maybe with speed, we got nothing going on there, so I'm going to make that say 50 rotate. Let's do the same, and let's play. Forward and backwards, I'm rotating Left and Right. Now here's a little trick for you. These values don't feel very good to me. So you may want to decrease the movement speed, I want to increase my rotate speed. This is what you often do in UT. That's a little bit better, it seems a bit more realistic, that's nice. Remember, if you change things here and you've got your play mode on, when you click play mode off, look down in this corner, when you play it, there you go, it's gone to 50 and 50 again. Is there a way to maintain those values, when I click "Play Off" again? Well, yeah there is actually little tip for you. Remember, when we did 20 and rotate we did 100, I think. Here's our live preview. What you can do, is in the script value, this goes for any component by the way, you can click the Menu there. To copy component, we go out of play mode, you go back there, and then you click "Paste Component Values." There you go. Now we've got a 20 and 100 and our values are maintaining, and this feels the same. I know you're not controlling it, but it feels the same. But you can see, that my values are still there. That can be useful to prevent frustration if you try and remember to do that.

Play video starting at :11:30 and follow transcript11:30

That's our basic movement there, let's flip back to our script here. I'm just going to show how you might do a novice to do something different. So steadying in our outdoor function, input dot get button down, remember get button down. It's going to do jump. So say you want it to jump, then transform.translate.

Play video starting at :11:59 and follow transcript11:59

You see everyone can press enter too quickly. So new vector three zero, make it one f actually. I was going to make it 0.0F. Now this, here's when I press whatever button's denoted as jump, then it's going to be transferred to translates, it's going to make it

move up a little bit. Now, it's not our job strictly speaking. Because I've got no code there at all to make it come down again.
Play video starting at :12:34 and follow transcript12:34
So you can see if I press that, then it jumps up and down. I've got no forces on this item at all. I could've left the rigid body and the [inaudible] and it would work. But that's really asking for trouble later on down the line, it's going to get a habit of mixing the two. How do I know which button to use? Well, again, as we did before, if I go to get to "Edit, " and I go to "Project Settings," I'm in my input Manager here. Where is the jump button? That's folder, horizontal, vertical. Where's the jump button? There it is. Jump button space, at the moment. So that's how I knew to press the Spacebar, then you've got file 1, with left control file 2, left alternate file 3, left shift, these are your default controls in case that's useful to know, but you can change them.
Play video starting at :13:26 and follow transcript13:26
We've done that. Some other things to show you. Let's just run this again.
Play video starting at :13:37 and follow transcript13:37
Now, if I tell you when I let go of my key, letting go now, let go now. Can see how my object is actually continuing to move after I've let go? It's got this easing in and easing out motion to it. What if I don't want that? Well, if I want something a bit sharper, a bit snappier, well that's easy. We go back into our update function, and you're going to do GetAxisRaw. Now, the reason this works is because when you do GetAxis, in the input manager that I showed you, there is the awesome factors there that show you how quickly, essentially your file, if you're pressing left on the stick or left on the key, how quickly does that value return to center? GetAxis is of value between minus one and plus one. We've got get an it can be any point on that scale. With GetAxisRaw, that is either minus one or 0 or one. There we edit snaps between those three values. There's no easing in, and easing out. It is best done if you try this yourself, to be honest, it's quite difficult to convey on camera. Before I press this now, it may or may not be of how big he see or it come to very sudden stop now.
Play video starting at :15:9 and follow transcript15:09
Stop now. Now. Now. Now. Now. There's no easing in and easing out. It just depends on what you want in your game. This is the thing because you actually element of control. If we wanted to follow our player, then you may be tempted to just child the camera, and then the camera is just going to follow along with everything. Well, you get this weirdness going on. You'd get that weirdness going on where the whole world is rotating, and that can make you feel a bit ill after to be perfectly honest. Let's not do that. Let's unchild it. Better way to go around that. If you want your camera to follow your player, let's just use simple script. We're going to go right-click Create new scripts, C-sharp script, going to quit cameraPosition.
Play video starting at :16:17 and follow transcript16:17

CameraPosition, easy thing to do. We're going to do vector3 offset. That's going to make an offset. Public GameObject player can lead to be public. Picked three offsets. I mean, you can't make it serialized field, but these are ways to do it. Actually, if you were to position your camera and get it all sorted first of all, this is a better way to do that. Rather than put in your offset manually, why don't you position your camera in the editor and then you can get their program to work out what you'll say is, it's easier to do the difficult work. We've got vector3 offset. In the Start method, which only gets called once, when the object goes active for the first time and say the offset equals transform position minus player.transform.position. This script is going to attach to the main camera. Here I'm going to say the offset is wherever this camera is now, minus where the player is and that's how far away, in what angle, etc I want to be for the player. Then every update cycle transform position, it's going to be set to player.transform.position plus the offset.
Play video starting at :17:53 and follow transcript17:53
Missing in I. Those seems to be fine. Save that. Go back to entry, drag-and-drop my camera position onto the main camera and then we can just check that it's there. We do need to drag-and-drop a player into there to assign it.
Play video starting at :18:20 and follow transcript18:20
Then here you can see the player moves around and it's not rotating with the player. Just very simple. That stops you having that terrible constantly rotating with the player which isn't how are you going to want your convention movement in the game, with cameras in the game to work that would be the hideous quite frankly. The last thing I'm going to show you is its just a very common error. Often you'll run it. I'm going to remove that reference there. You will see this error quite lot. You can see down in the console it is going like Bilio. Let's just pause it here. I'll let you sign reference exception. The variable player is not being assigned to. In the interest of getting you used to read in these error messages, just means you haven't defined it. You haven't said what player is and you know that because there is a none over here and this, if you get to really high number, it denotes that there's something going on in your update loop, but also you could just double-click on it and it'll take you to the error anyway. It can't find what that player bit is. There's some basic movement and some basic camera following for you to use use in your games.

In this video, we're going to start building examples seen together, implementing some of the things that we've learned in our videos so far. Here we are with my blank screen. I just started a new 3D project from the menu. In here, you can see I've got my main camera, and my directional light right there. I'm just going to check. Fine. We have nothing here in the scene. I'm going to start by throwing down a 3D object in the plane. By doing this, it's just going to provide a flow. I want to build is a Cafe. I'll give myself a scale of five and five. I must make that one. Our plane is going to be flat regardless of

what you do. Let's zoom out a bit. I'll see if I can fix that. I'm just going to reset this X2 column.
Play video starting at :1:24 and follow transcript1:24
Zero, zero, zero. There you go. This is zero, scaled it up to five. This is going to be our floor. In case [inaudible] It's got a cube. There's our cube right in there. For some reason, it's spawning in. Not a particularly useful value. Scale seems to be pretty massive. I'm going to bring down the scale with my plane a little bit. There you go. Now seems to be a little bit more realistic. Haven't gator? Yes, can be something a bit more doable with particular slot 34, my scale, that's crazy. What I'm going to do here is I'm just going to go from the top and then just rotate round a bit. I'm just going to move this into position, I'm going to get this roughly. This is where you really want to do a lot of modeling per [inaudible] and you can see. Because it can be a little bit fiddly using the basic things if you do that. You're going to use it's fee to snap,
Play video starting at :2:57 and follow transcript2:57
yeah, just a little bit more. I'm just going to make that a little bit bigger, a little bit shorter, and then move, snap to the corner. That'll be good enough. We're going to do, clearly doing a little price top here, it doesn't have to be perfect. I'm going to put some walls there. There you go, so v. It's not that there. That's the wall. Lovely. They'll do just slightly of the hanging. It'll be a complete wall. I just press Control D. You'll see here in my hierarchy that there are two these things. Now, I was going to drag that across to snap that there, actually, I'm going to snap it to the other wall so it's completely perpendicular then what we can do is hold v.
Play video starting at :4:9 and follow transcript4:09
That will do.
Play video starting at :4:13 and follow transcript4:13
Let's just rename this and pull that wall.
Play video starting at :4:20 and follow transcript4:20
It could be wall, and then this next one can also be wall. We'll make it wall one. I'm going to duplicate the pair of these, and I've still got these teeny ones selected, and then I'm going to make it very tight. I'm going to grab an axis, hold control, flip those walls around, there you go, and so that's just a quick way to build a room. What I will do is make this one a little bit smaller. They'll now be in and out into the room that we're building here. That was my direction way, is there. Just drag it out the way remember the direction doesn't make a little difference, but rotation does. Tell you what I'm going to turn off lights, so that things a little bit easier to work with elsewhere just laying things out. I want to have a cafe with some entrances and some exits, etc. I think we can work with these sizes. First thing, I would like to put in here is I am going to put some chairs or some benches and some tables, let's create, I'm going to make a cube. I'm just going to snap that a little thank you. Very good. Snap that down there. This can be the back of a bench. Let's just make it go up, I'm going to go out.

Play video starting at :6:21 and follow transcript6:21
I think that would be cool.
Play video starting at :6:28 and follow transcript6:28
If I do that, then I can go through the object there. So once more just duplicate that again. Then pull it out. I'll tell you what before, so you can see already here, everything's getting a little bit cluttered. Good way to start organizing hierarchies, create an empty object. I'm going to call this, room. Then I can take my plane screen name my plane to floor. Floor, walls, drag into room. Just tuck that under. Now, we don't have a lot of clutter or as much clutter. Going on so far, shrink that, just going to bring that out down a bit. What do we think of that? If I snap that like that, that seems pretty good so that we'll do as a standard bench seat. Again, I begin to create empty and say bench seat.
Play video starting at :7:42 and follow transcript7:42
I guess, I'm going to drag and drop these two cubes into there. Now at this point, I want to use that bench seat again. I want to replicate that across my cafe. It's a good idea to make this bench seat a prefab so if you remember, what we are going to do is I'm going to grab the bench seat. I'm going to drag and drop it from the hierarchy down here into the project, so drag, drop here.
Play video starting at :8:13 and follow transcript8:13
Drop there. You've got bench seat is gone blue in my hierarchy, in my project I'm going to create a folder because you want to keep things nice and neat if you can.
Play video starting at :8:30 and follow transcript8:30
Just really keeping things super simple.
Play video starting at :8:41 and follow transcript8:41
Let's go for another cube. Snap it over here, till there. I'm going to stretch that up, stretch in a little bit like this. How does that look from the side?
Play video starting at :9:6 and follow transcript9:06
I'm going to just use a fetus snap up to height again where I can bring it lower, quite a bit lower actually. I'm going to put a table on top of it. That'll do. Just roughly the same. We can adjust that in a second. Then what I might do is try and see if I can grab that cube, duplicate that, bring it out here. But let's bring that outside the prefab because it's, let me just not do that. I'm duplicating that cube so I want to keep everything the same width, it just saves hassle. You can see there that it's got little cross on it, this means it's trying to add it to the prefab. I don't want it to do that. I'm not going to update my prefab, so it's going to drag it out here. You can see that that cube here now is free-floating just like this other cube, because I'm going to make this cube into a tabletop. Let's just squish this below that and it's just stretch it a bit like that. I'm just going from the top here, want to place it over that object. I'm just going to snap it, so I'm snapping it a little bit. There you go. Just helps me get it on top of this objects here.
Play video starting at :10:35 and follow transcript10:35
That will do. Let's go back to shade it. Let's rotate back round, thus is at level?

Play video starting at :10:49 and follow transcript10:49
We can have a look here. I'm going to shade it and then again to wire-frame margin. That's good enough for me. At the moment, so doesn't need to be insert this point and that's level, great. These two keys are going to create another empty, going to cause a table, and then I drag these two into the table. Then I'm going to drag the table into the prefabs folder. The table is now a prefab. Because the bench seat was made a prefab already, I can grab a bench seat and bring this here. I can actually go to my inspector over here and I can rotate it around the y-axis. I'm going in here. It's actually on the y-axis by a 180 degrees. Make sure you're selecting the whole bench seat not just the component. There we are.
Play video starting at :11:55 and follow transcript11:55
What am I want to do, just make sure that the table and the bench seat are the same.
Play video starting at :12:10 and follow transcript12:10
Just bring this table a little bit more in line and this bench seat making sure you're selecting the whole bench seat and not just a little bit or in line like this. But I want to be too exact because this thoughtless, boring feature of watch me and be, we don't really care who's trying. They you go, split that out little bit. Now for bench seat. Here, I can now just move this down here, the direction line layout coming at the top, so got the room there. If I bring this and we've got two bench seats. You can see here in my hierarchy two bench seats and a table. Why don't I actually pull them together, create empty and say that's a booth. Let me grab those, drag and drop them into the booth, and then make the booth a prefab. Shrink that down. Now I got booth in our scene. Here you go. Now we can move this one to the back roughly, here in roughly a stunt up like in. I could press control D, that's the easiest thing to do. I'm just going to drag these out. This is how you can use prefabs to really make your workflow a lot quicker. Now I am going to slip both of them control D, drag them over here. Not only that, but also the changes that we're going to make in a bit are going to be an important thing. This is a really badly laid out car-fi because but I think it'd have a counter here. But that's just what we're going to do for now. We're just going to have one door going in now. I am going to put my wall there. It's a big entrance. Then maybe our counter is over here or something to happen there.
Play video starting at :14:43 and follow transcript14:43
Just come to the counter in a bit. This makes some lights, so again, with our lights, I'm going to turn on lighting now. I am going to find my direction light. It going to rotate this cafe things a little bit, better lit. There they go, that's a bit nicer. We do need something to be lighting this cafe a little bit more, so let's put some point-like scene. I'm going to put in 3D object. Let's do a capsule. I'm going to retype this [inaudible] , hold control so it's exact. Okay, it's 90 degrees. I'm going to scale that down a little bit. But a make-out that these little wall lights. Maybe make them even a bit longer.
Play video starting at :15:53 and follow transcript15:53

See where I've duplicated all these booths? Again, create an object can say booths, plural, shift and select, drag and drop into booths. Tuck that up [inaudible] up. Keep hierarchy. We 're nice and neat, there's a capsule. Click on that. F T, call it a light. These are a default color at the moment, we're going to change that in a bit. There's our light. Now let's GameObjects, Light, and I am going to do a point light.
Play video starting at :16:39 and follow transcript16:39
What I'm going to do is I'm going to attach the point light a little bit unto there.
Play video starting at :16:54 and follow transcript16:54
Let me just turn off, dismiss. In case you didn't see what I did there, went out to the top right here and just turn it gizmos on and off. It's because that light bulb was getting in the way.
Play video starting at :17:11 and follow transcript17:11
There's our point light. I'm turning off the gizmo. Next, I'd like to see how it actually looks at the current big fat icon getting in the way. Getting in the way. If you scroll wheel is too quick, then use your right mouse button and alternate it. Let's just bring that down underneath. That's going to do for us. As my kit, some ridiculous color, like orange, don't know why. I'm going to drag and drop that point-like underneath the light, and you can see that the point-light now is [inaudible] to the light. Let me just Control Undo, going to zoom in and show you that again. Drag-and-drop point-light underneath the light. We'll do that. I'm going to tuck that underneath. Now I'm going to drag and drop my light in C prefabs, so now I've got set prefabs going on here. Go back to here. I've already got that in position so might as well, Control D, drag it along, Control D, drag it along, Control D, drag it along. That's one wall. What I'm going to do is I'm going to grab all of them at the same time. Do Control D. I'm going to bring them over here that are a tight them using my inspector. I've got all of them selected. It's showing different x and y because they have slightly different x and y folder. I'm going to do rotation, punch in 80, that takes the lot of them.
Play video starting at :19:2 and follow transcript19:02
Just want to check, hold on. If that's my line, where my point-light? Yeah, it's the right side.
Play video starting at :19:11 and follow transcript19:11
I wanted to check that the point lights was the right side of these loads. Now we've got some lights underneath. It's just that these look different because the direction light's not shining there. But you can see that if I take one of these lights, then that's my light and then my point-light. Let's just focus in. Sometimes the F doesn't work. Particularly great stats that light and the point light is just in front of it.
Play video starting at :19:49 and follow transcript19:49
Right-click, going to Create Empty lights. Draw and drop then. What's going on there? Lights. What's the problem?
Play video starting at :20:12 and follow transcript20:12

Now, yeah. Got some lights there. I'll leave that back wall. There's some lights for us. Maybe I'll [inaudible] in later or some such. Not huge amount needed. [inaudible] what we can do in a minute.
Play video starting at :20:36 and follow transcript20:36
Let's do a big light in there, just in the corner there. It will make good thing to make a big light. Let's just use a cylinder for the sake of variation, hold that grab control, to rotate it by 90 degrees. Then what I'm going to do, is I'm going to embed it in the wall.
Play video starting at :20:57 and follow transcript20:57
I'll stretch it out to be some weird light doesn't it? Now the drop in our area lights, say GameObjects, light, area light. Now remember, there's a number of things to consider with our area lights. Oh let me just turn [inaudible] back on, otherwise you can't see it too. It's going to be emitting lights in z positive. Make sure that it's pointing the right way. I'm going to snap it little. You don't need that. Snap it a little bit to the front of our cylinder. Bring it down, bring it forward, so it's not behind.
Play video starting at :21:57 and follow transcript21:57
I mean, this isn't the best light to use it for this at the moment, but I'm using it because I want to prove a point. But this will do. I'm just moving and scaling, so you can see how this works. I'm going to rotate this down a little bit, tilt that down.
Play video starting at :22:21 and follow transcript22:21
I'm just going to do this just for the effects really. Let's make the color blue. Like that. Let's make the intensity, about 20. Now, you remember there are two things that need to happen before an area light gets shown on our screen. Two things, first of all those objects need to be marked as static because it needs to be made known to the Unity Editor which objects are not going to move and which objects therefore can take a lightmap, a bait in lightmap and then we need to do the baking of that lightmap. I'm going to do that in a second. Now before I do all that, we're going to just create some basic materials and textures. Now I haven't gone through this in detail. We're going to do a little bit more on this later on, but just to make this look a little bit nicer. What we can do here is in our project, were going to come down here, I'm going to right-click, I'm going to zoom in, that's going to make it easier and to create. Then here you can see is the phrase material. Let's do seat material. I'm going to right-click create a folder that's going to be called Materials. Didn't see that coming, did you?
Play video starting at :23:48 and follow transcript23:48
This now allows us to choose a different color, at least, I'm just going to stick to flat colors for this. Keep it simple. When you click on the material, you come up with inspector panel, looks like this. By default it's going to be an albedo material and we can choose the color. Let's just go and choose I don't know diner seats like reddish, something like that. Let's do that. Then if we go into our prefab, if we go into our bench seat, we don't click on the bench seat here we on that prefab view that we looked at before and I can drag and drop my seat material on the back and the bottom. Then

when we've done that, we can leave. We can double-click on "Scenes" there and all of our bench seats have updated. Fantastic. Didn't even have to go through and copy it across. Let's do the same for table. I need a different color, so I'm going to right-click on "Project" "Create" "Material" Table Material, and then I suppose it's gray, just to make sure it's not white. Would you offer a gray table in a cafe? I don't know. Maybe it says slightly nicer cafe. It's not Formica. It's not plastic tables, it's brown, it's wood. Let's just imagine it, shall we? We can dream. But the base could be I'll tell you what let's make a little bit of effort, we could say metal material.
Play video starting at :25:34 and follow transcript25:34
Let's, click the "Gray." Then I can click on scenes or you can always, if you want to click back in the hierarchy here, side effect. You can see that all of our stuff, if I Zoom in, you can't see those particularly well. But it is there. Hold on, turn off the lights. You can see that there's a brown tabletop and the great top. That's nice. That's what prefabs will do for you. I suppose really, we should do something to those lights themselves. Let's right-click "Create Material." Lets call it LightMaterial, just a standard thing.
Play video starting at :26:26 and follow transcript26:26
White is pretty good, actually. White would be fine. We can click on our "Light", pressed the wrong button, just a minute.
Play video starting at :26:44 and follow transcript26:44
Click it right there. Let's just drag and drop our LightMaterial onto that. Now, it's worth doing it still because we may want to change our LightMaterial later on because I'll show you in a second. We've got LightMaterial on Mesh Renderer. It's still worth doing that.
Play video starting at :27:10 and follow transcript27:10
Actually, I'm going to drag and drop my LightMaterial onto that object at the back there. You can see that this, which we're going to say is the, let's call it RoomLight. That's also called the LightMaterial. Say I changed my mind later on, I wanted the LightMaterials to actually be an awesome yellow or any other color that I imagined. If I took the LightMaterial and just change that now to a yellow. You can see it actually changing in real time in your scene and see how that turns out. Maybe we can make that slight yellow which keep it right there, just for the sake of demonstration. There we go.
Play video starting at :27:58 and follow transcript27:58
Very well and good.
Play video starting at :28:2 and follow transcript28:02
Then we've done all that so you can change your materials on the fly. It's the same with I'll see me turn out, so I'm just going to drag and drop these into my materials because in the seat material decided that wasn't the right red, then we can make that a little bit done if you want. You can see that I've changed that. Here's the old color. You can go, so the tooltip says you can go back to this by clicking on the color. Lets just make it a bit duller. It's not that broke cafe. All these things just gives you a bit modularity and helps you to set up.

Play video starting at :28:43 and follow transcript28:43
Welcome back here. We've just done our light bake. Now Mike and peters, it's not massively powerful, but it's not weak either. One airy light in, let's be honest, is quite a simple scene. These finisher is not exactly particularly complex, but this is just taken 45-minutes to get this single airy light to bake. Now, that's just as a warning. It doesn't mean don't use Airy lights, but factor that into your time flow and depending on the hardware that you have and the game that you're using. Sea wave, you can't achieve similar things by using other kinds of lighting within your game. Now, I can click, the other thing you can do here is, I'll bring up my lighting thing here and if Click on at the top, you can see there's a thing that says Bates lightmaps. If you click on that, you can now see that there's a lot of different lightmaps here. That's the result of us going through this process. You can open those, you can have a little preview and these are actually image files that you could open them up in Photoshop if you wanted and change them if you felt that was a good idea, often isn't. But it can be done.
Play video starting at :29:55 and follow transcript29:55
Good, but took 45 minutes to do the light bake. Beware, but it can be pretty good that switched my game view. I'll show you it's going to move out. If we move to something a bit dark and a bit more Moody, if I do that to Control Shift F to snap my camera, the Alignment view, let's go to our game as well. You guys see a nice Moody lighting, et cetera. There we go. We've got a basic scene ready there. We've thrown some fancier together. I've showing you how Prefabs can be used. We've used some lights, I've shown you how baked versus real-time lights work in actual real life. Let's have a look at how we can change some things within our scene using some input. At the moment, what I want to do is I'm going to change some lights, which are the moment pink. But I won't mess around with this pink. Let's go back to orange, where we were before. What I want to do is I want to press a key and change your wall lights, get wall lights to change color. [inaudible] I'm going to make some scripts. You can see here that I've made a scripts folder down here. I'm going to make two scripts, create C Sharp script. First one, it's going to be called input reader because I'm going to use one script to take the input. Then I'm going to create a second one. I'm going to call that light control, separate these two out, just one to read the keys, co-functions and over scripts and then a light control script to change the color of the lights. Let's open to pair these up in Visual Studio. That's going to drag them in here. It might input read across a chunk. This is my light control class. My light control class,
Play video starting at :31:56 and follow transcript31:56
what you might be tempted to think is that I could create a space for the light prefab and then get hold of the light component in the children there, and then change the color of it in script. If I do it to prefab, then I do it to everything, but that's not quite how Unity works. Remember that a prefab lives in the project and not in the hierarchy. The prefab is instantiated in the project. Now, if your project is running, then any changes that you

make to the prefab are not going to be reflected in the actual running level, in the hierarchy of the level that's running at the time, it'll be reflected in the project when you stop that project and go back, by then you would have actually changed the prefab. But that's not helpful if you're wanting to change the lights, right then. What we need to do is we need to click in the right, collect these objects together. Then we iterate through the array and then change the light points that way. To that end, I'm going to make a game object. I'm going to call it "side lights" because it's for the side light. I've put it at the wrong place. It's fine. Just clicked my cursor in the wrong space. Side lights. Now, before I do this, what I'm going to do is I'm going to use the "tag" function of game objects and nodes all together into a single array. Now if I go to my prefab, so double click my prefab here and I go up here to our right. I've actually done this before. Let me remove this. Come and excuse me it's by line. What you need to do is you need to add a tag here. I did this already, of course remove it. Up here in the "inspector" menu, click on "tag", and tag is just a way of grouping things together. Play is common tag. You can get a reference to the object that has player as a tag. Again, controller, common ones might be enemies, you have lots of air enemies, ground enemies. They might be a tags so you can get all those enemies together into a collection. If you don't see the tag that you want there and you're creating new one, we have to do is go to the tag, go at tag. Then you add another one here, type something, press save, and then it would come up as another tags. This is what happened here. But then in Unity you must go back to the light, again, bringing up this thing and then select the "tag" from the menu. It won't automatically tag that we fill new tag. Make sure you go through and select it. You create a tag. Side lights. Go back to the project. Let's go back to my code. In the start method, which is the method that automatically fires once when the object comes active, I'm going to make my reference, what we usually use to make references. So game objects.find game objects. We're tag the plural objects with tag, then open brackets, and then just the text. It's side light singular.

Play video starting at :35:21 and follow transcript35:21

We've got that. Then input reader. We're going to go our over script here, input reader, factor data that we want do that here. Let's create in light control, we can create a couple of functions. Do public, void set can be read from another script, purple lights.

Play video starting at :35:47 and follow transcript35:47

Then we're going to do a for each loop for the collection that we just made for each. Then open brackets can be GameObjects and give it a name. G0 is common in side lights is the name of the collection. Then we're going to do "go" of each GameObject there, we going to GetComponentInChildren or just type this out and then I would explain what's going on.

Play video starting at :36:17 and follow transcript36:17

What's happening here in this script is that for each GameObject, we're going to use this function, GetComponentInChildren. Why GetComponentInChildren? There's

GetComponents, GetComponentInChildren is a few things, GetComponentInParent, etc. If I go to "My Light" and I double-click, [inaudible] just compiling him out. If I double-click on My light, you'll remember that the light objects is actually a caption. Then the Point Light is a child of the light. I'm collecting these GameObject here. I'm collecting these GameObject, I'm referring to this Point Light below it, or more accurately, I'm referring to this component of the Point Light GameObject below it. Back to my code. We're going to do color magenta. Then we can just save yourselves a little bit of copying and pasting I'm going to have another one. Mark it Red, day-to-day each GameObject make it "color." Now, for the color function, you can see, there's a bunch of things you can do here. You can do convergence, you can do cycling from one pellet and overuse laps to go from one color to another smoothly or that kind of thing. There's also a lot of inbuilt colors there. You can type in your RGB values if you so desire for prototyping, etc. I quite often just use the built in colors because it's easy. I'm trying to show function like this. We got two public void methods there. Getting component children, changing the color to magenta or red as appropriate. Let's get our Input reader. Our Input reader at the top, I'm going to make a reference to our LightControl script. Remember that the script is a component in of itself, so we can form a reference to it. It's not just built-in things that we can make reference to. Light control scripts, that and I'm going to make that public because that's the other way of making a reference. In Light Control, I've used code here to make that reference. Input reader, I'm not going to put anything in stop because I'm actually going to do the drag and drop method to show you how that works as well. Bin update. Let's do if "Input.GetKeyDown." Input.GetKeyDown, what that does is it means that the button press any fires. Once it's true on the third, it fires in the first frame that is true. If you did get key, then this method would fire every single frame they held the key down. That means it will fire up to 60 frames a second. So normally you often want for an Input.GetKeyDown so any fires once. You've got space and then we're going to do "lc.PurpleLights." So that's fine. Then we'll do similar Input.GetKeyDown.

Play video starting at :39:33 and follow transcript39:33

By the way, notice that the space is lowercase before, by the way, "lc.RedLights" These are two scripts. Let's drag my code over there. We need to attach them somewhere. Let's go back to our Hierarchy, to our scence [inaudible] is compiling.

Play video starting at :39:59 and follow transcript39:59

What I wanted to [inaudible] folder. I'm going to drag and drop "Input Reader" onto the "Main Camera." I'm going to drag and drop "Light Control" onto the "Lights group." This empty container objects which I use just to organize all the lights. So that's there. [inaudible]. Main camera, I said I had to assign the Light Control. What I can do is I can drag and drop the lights object to over here, and it will find the lights control component from the GameObject lights. Just drag and drop that there, and then that should all be ready to go. Then we'll click ""Play." Look our game. Make sure you click on your game

window first of all, and press "Space." We've got purple lights. Press M, I've got red lights. Space M, Space M, Space M. Just some very simple input to show that you can change lots of things at the same time, we've got some basic interactivity in our scene [inaudible]. In this video, we built a basic scene out with some of the knowledge that we've learned from previous few videos.

In this video, I'm going to show you how to do quick web build, so you can easily share what you've made in Unity with other people across the Internet. Here I'm back in my built in Unity and I want to build something out of the editor so that someone who doesn't Unity installed on their machines can see what I've done. The way we're going to do this, is if you go up to the top left-hand corner, go to "File" and then "Build Settings". Let me just zoom in on the screen that we have here. This is our Build Settings.
Play video starting at ::49 and follow transcript0:49
By default, you'll see that the little Unity logo is standing by the PC, Mac & Linux Standalone build. As you can imagine, there's different things to pick here, x86 target platform can you build for Windows because, I'm on a Windows PC here etc. If you want to build as a standalone x86 then do Build and Run and then you would find the built folder in your directory folder, will be similar to what I'm about to show you. I'm going to show you what to do for WebGL. WebGL is little bit easier, what this will do, it will build out wherever you've gotten your Unity editor into an index.html file, plus all the resources to go with that, and it will essentially save that up like a web page. This option here, Development Build which does exist in the other ones as well, a development build is what is bigger, it's got less streamlining done in it. When you click the "Tick" for development build it will minify your scripts,
Play video starting at :1:55 and follow transcript1:55
which means that it shrinks the scripts down, scrambles them, and shrinks them down so it take up less space they are quicker to load, but also a lot more difficult to crack from someone if they're trying to get to the code. If you do development build, it will minify your scripts so that you can look at all the scripts that generated upon build. For now, you don't really need to development build so we just going to leave that. But what we do need to do is we need to switch platform. You switch platform and this takes a second.
Play video starting at :2:27 and follow transcript2:27
It's still thinking about it and here we are. You can see that the Unity search has took about 30 seconds or so, and you can see the unity logo has changed onto the WebGL. That's now the target platform, if I wanted to switch back to anything else, I click "Switch Platform". Let's just click "Build and Run". It's going to open up our explorer. It's going to ask us to give a place for that to happen. Good way to do that it's a new folder and I'm

going to call that something cryptic like WebBuild. WebBuild there, I'm going to select that folder.
Play video starting at :3:15 and follow transcript3:15
Let's be a bit more for us. Now, it's starting to build and it'll show you there, state it goes through. It does take a bit of time depending on size of your project to do this thing. Unfortunately, this doesn't really get much quicker.
Play video starting at :4:1 and follow transcript4:01
We're back from the web built now well, that took quite a while so we edited all of that out. There was a console thing here, say how long it took to clear by mistake. I don't want to go through all again, but it took six minutes, 359 seconds. You'll get a little message here that says how long that took. Now, the trouble with trying to test a web build within Unity is that you'll build it and it will start a local web server for you to run that built and into test it straight away, which is fine. It will do that and it'll come up into window and show you what's happening. You'll see that when you run your build. But what if you want to run your builds without having to go through the build step again. What if you wanted to run it and experience it in that WebGL format another point outside of the Unity editor. Well, the way to go about that is, so you've got your build that says, you remember that we built it to something called WebBuilt within the project folder. This is where we are, like I said before, here's all your data and here's our index.html. Now you want to reach your command line first in order to do this. On Windows I'm going to hold "Shift" and then "Right-click" and it has to be shift and right-click because when you get the open PowerShell window here on Mac, you're going to use the terminal Linux the same wherever you need to do to get to your command line. First of all, check your version of Python. It's really difficult to imagine that you haven't got Python 3.X in some form on your machine by this point. Check that you got Python installed. Then after that, just type in Python with the web directory, http.server.
Play video starting at :5:53 and follow transcript5:53
All that does is it will start up, listed web server for us. If I start my Firefox up, then I'm just going to do these windows over there so you can see things going on at the same time. Let's go to local host 8,000.
Play video starting at :6:14 and follow transcript6:14
They go and then it just runs our built. I'm just a word of caution you may have to turn off tracking protection. Tracking protection can cause some issues with the Unity web [inaudible] just if it won't load, you get an error, then come up here turn off track and protection or things like that for whichever browser you're using, things that can get in the way. You can see that it's built and served some stuff there, there's no much we can do there, but that's a quick easy way to run your game without having to build it from scratch.

Hi, In this video, I will show you how you can export a Unity project to WebGL and upload it to the Coursera platform. We have in fact, included a series of activities in your course which allows you to share your Unity projects with other students and get valuable feedback. Now, just to note that this demonstration is valued for all of your courses which use this methodology for sharing Unity projects. Now, the first thing you want to do is to head to your Unity editor. Now, here you can see my inspirational Red Cube amazing project, which essentially does nothing, but it is here for you as an example. So if you're happy with your project, the next thing that you want to do is you want to export a WebGL version of it. And you can do this by clicking on the File tab located on the top left hand side of your Unity editor and then you want to click on Build Settings. Now this will open up a pop up window and there's a few things that you need to put here. So, first of all, you need to select all the scenes that you would like to export as WebGL. Now, in my case, I only have one. This is called inspiration of Red Cube, so I just wannna make sure that this scene is ticked. Next, you want to choose the export platform. In our case, we're going to export to WebGL, so you you want to select WebGL and finally, you want to click on Build. Now when you click on build, a second pop up window appears and here you simply need to put the title of your WebGL export and the destination. Now, in my case, I put Red Cube as a title and I want to export to my desktop. Now when you click Save, then Unity would produce a WebGL export with this title and at this location, in my case, desktop again. Now, depending on the complexity and size of your project, this might take several minutes. So I have gained some time for you and did this ahead of time. So you go ahead and click on Save, but I've already done it .And you can see that the WebGL export sits in my desktop and it's called Red Cube. Now, if you open up this folder, this contains all the files and folders required for Unity to build and run this project on the Web, you can see that there is an index.html on the root of the folder. And there is also a build folder containing a lot of data in terms of assets and packages of your project, in the format of JSON and JavaScript files. Now, at this point, what you want to do in preparation for Coursera upload, is that you want to zip these files and folders. Now it's very important, you don't want to zip the folder itself. You want to zip all folders and files within the main folder altogether. Now, actually, what I mean by this, so you click on the WebGL export folder, then you click every folder and file like this, so you select them all. Then you right click with your mouse and you click Compress. So you want to use this methodology for zipping your project. And this would produce an archive.zip, you can you can rename it. I'm just going to call it game test like this and then I'm going to just drag it on my desktop for simplicity. At this point, you're ready to go for the platform applaud. So we go back to the Coursera platform and this is the Unity applauder activity. And this is how it looks like. So you're going to have several of them. But don't worry, because each of this activity will have some detailed instructions on exactly what you should be

uploading and sharing. Now, for the purpose of this tutorial, I'm just going to use this first 3D scene Unity applauder. So you just click on the Start button and what this one will do, it will open up the Unity applauder which looks like this. Now the only thing you need to do really is to drug your WebGL export into the rectangle and that will upload your project. Now this rectangle below with your project, you can see it already in there. This is just a confirmation that your project uploaded correctly, so you can't really do much here. But it's just for you to know that that was successful. Now, if you want to do some changes, or if you've made a mistake, you can always replace the file. So you click and replace file and then you drag in an updated version of your project or a complete different projects. But you can only upload one project at a time per activity. So once you've done this, all you need to do is, just need to close the tab essentially and you will go back to the Unity apploader dashboard. And at this point, you need to generate your shareable link, to be able to share your project on discussion forums and peer review assignments. So there is a blue button just at the very end of the activity. You just click on this and this will take a few second and it will generate the shareable link that you can use to share your projects with other students. Now, it is crucial that you test this before you post it on discussion forums or peer review assignments. You just want to make sure that your project works. Now just bear in mind, you need to select the link from the beginning, up until and excluding the upper hand. So just until the false statement. So you copy this.
Play video starting at :5:35 and follow transcript5:35
You open another browser tab and then you paste it in. And then you press Enter and this will open up your project. And you just want to make sure that this works. It might take a few seconds for it to load up.
Play video starting at :5:52 and follow transcript5:52
And there you go. That is your project running on Coursera for other students to review and give you feedback. You can expand it here and you can view it full screen. Or you can just go back and view it as so. So this is it. This is how you can generate your shareable link. So now you can take this link, remember, up until and excluding the apprehend path and you just paste this on discussion forums and peer review assignments.

In this topic, we are going to look at what I think is one of the most important, useful and really exciting aspects of 3D animation, which is physics. Everything in the world, in fact, everything in the universe obeys the laws of physics. One of the great things about the laws of physics is that they can all be expressed in mathematics. Why is that great? It's great because computers are really, really good at math. So we can just code up the equations of the laws of physics in a computer program and that will simulate all of those physical laws so that our objects in our program move realistically as if they were in the real world with real physics. That's exactly what game engines like Unity do. They

have a physics engine which implements not every single aspect of real world physics, but the most important ones. That led to a lot of our objects move realistically, but importantly also interact realistically. It's not just about stuff moving normally. It's the fact that we can pick an object up and throw it and whatever direction we throw it in, however fast, however slow, it'll respond in the correct way physically. That gives us a lot of dynamic fine-grained into activities, not just pressing a button, it always does the same thing. The fine details of how we throw a ball or how we kick it really deeply affects how it moves. You can see that with a lot of exciting physics case base games where so many the puzzles are to do with solving how objects move around from realistic classics like half-life and the more recent VR version of half-life Alix, which have physics throughout. Their game play to more casual 2D games like Angry Birds, still fundamentally based on physics. It's a really important animation technique and that's what we're going to learn about in this topic.

We know about graphics objects. They are made up of geometry, which are the polygons that make up the shape of the objects. They have materials which are colors and textures that make it look nice and make it look like real-world things. But if we're going to do physics, we also need to know about the physical properties, not just the graphical properties of an object. We also need a physics objects, and that consists of a number of things. We've got a rigid body. That's your main physical thing. The physics world is made out of these things called rigid bodies. A rigid body, it's a body which means an object and its rigid, that means it doesn't move in itself, it doesn't stretch, it doesn't bend. You can make stretching and bendy things out of rigid bodies, but each individual one doesn't stretch or bend. That rigid body is which has your basic properties like your mass and velocity. As well as a rigid body, you have a physic material which represents certain other properties like how bouncy and object is or how much friction like, how strong friction is on that object. That's also really important and really affects the way the body moves and how it interacts with other things. So it's very important to set that physics material. There is a last thing called the collision geometry. What's that? Well, that's a special set of geometry, this only for handling collisions. Why do we need that? Well, when objects collide, you see two objects moving and they move to each other and they interpenetrate and at that point you create a force. What happens is in the real world when two objects hit each other, they don't move into each other, they just stop. But there's nothing to stop that happening in graphics. So you actually have to go and check check every single polygon of every object against every single other polygon of every other objects, to make sure they haven't interpenetrated, one hasn't moved into the other and you don't need decent collision stuff. Actually that's really quite slow. You've got a lot of calculations to do. That's why when you've got an object, you want really details graphical geometry, so it looks really great. But actually, you want your physics geometry to be a lot simpler so the physics can be really fast. To be

honest, you can't really tell the difference between how a box bounces and how something that is a box with a little bit of extra detail bounces. It doesn't make much difference to your perception of the physics of something. You can simplify your geometry done quite a lot. If your object looks a lot like a box, make it a box. If your object looks a lot like a bowl, make it a sphere, sphere collisions are actually very fast to calculate. Or if it's a little bit more complicated, you can build it out of several different boxes. Or you can create custom geometry. But custom geometry there is a lot simpler than your graphical geometry. When you work with physics, all you need to do is create a bunch of rigid bodies with physic material and with collision geometries. When you do that, all you need to do is say, I want you to obey the laws of physics and it will just happen. It will work and it will look realistic. That's a really fantastic thing about physics engines and why they're a really great resource if you're creating veil environments.

Okay, so I'm starting here with just a very blank scene here. And what we're going to do in this video is we are going to, just create an empty game object that spawns a bullet and the bullets going to move. It's going to destroy after a certain length of time. Point brief and simple, but hopefully give you enough understanding Instantiation. Side, here, I'm going to create my bullet first. Prefab it, then create the thing that's going to spawn it. So we're just going to make it very big game object, then 3D object and a sphere seems a good thing, for a bullet, so press f and there's our sphere. Wonderful, I don't really need to a huge amount of that. That would be good to change the color, I suppose. So what I can do, to do that is I can do right click on materials here. You can right click anywhere, but I've made materials folder, material. Let's call it, bullet material, so I tend to call these things, I'm going to zoom out so well, my bullet material selected. I move here in my inspector, and I'm going to make it. A nice pink color, I think a pink bullet would be lovely. going to drag and drop it, I can drag it and drop it on the, drag and drop it on the sphere up there. I can drag and drop it in the scene view here, so I'm going to do that. So that is my sphere, are going to highlight it. And I could either, highlight it the hierarchy and click F2. Or I can go up here and type a new name and call it, bullet, this is all very inventive, isn't it? So call it, bullet, and then let's write a script to make it move, shall we? So, I'm going to take down materials, that's write a new script.
Play video starting at :2:3 and follow transcript2:03
I guess I'm going to call this bullet movement.
Play video starting at :2:8 and follow transcript2:08
Yeah, make sure I spell that right. Here's another little tip for you. If you, make a script and then you think, actually, no, I don't do that. I want to call it, bullet moving. Maybe, you think you can just go and rename it there and it would be okay. Unfortunately, you would be wrong, because when I double click that and bring it up in visual studio, okay? And reload it, then what I'm going to draw your attention to, is how it says bullet moving is what you wanted it to be called and then bullet movement down here now. So this is

just a bit of a warning to you. Make sure that those two things are exactly the same. And what you'll get, if you don't do that is you'll get, well, so let's just carry on with the mistake, and I'll show you the kind of area that you'll get, and then you know what the area is. Okay, so we don't know anything and start. I just need one, this is really simple. I'm just going to call it public float, speed to be more secure. Here you go, see your last field and float speed. Let's go to update, transformed dot translate.
Play video starting at :3:33 and follow transcript3:33
33 there 3, 4 times time. Doctors time, remember, frame rate, independent speed, okay.
Play video starting at :3:48 and follow transcript3:48
Nice, jump back to UTT, it's compiling, okay. Hasn't give me a problem yet, drag and drop. But put it, there you go. Can't add script component because the script class cannot be found. So, there are one or two other areas that will come up, if you change the name after you've created the script initially. But it's because of this mismatch between the name of the file and the name of the class within the file. Okay, so let's go back to our thing, recorded, bullet moving, make sure it is, exactly the same. Say, obviously case sensitivity, and now compiling wouldn't give me there before.
Play video starting at :4:35 and follow transcript4:35
Doesn't give me now and now, so it's just shrink everything down, apart from put it moving speed. Just give it 10, so that's all bullet, our bullet is there, happens to just so be in our hierarchy and it's moving, which is nice. Let's just make that a little bit clearer.
Play video starting at :5:4 and follow transcript5:04
Select my, make some my main cameras down here at the moment. If I just orientate its here, its traveling forward, and that's because the main camera, align my view. There we go, and then when we get our game, okay, but just a bit simply to see that going.
Play video starting at :5:26 and follow transcript5:26
Wonderful, so we've got a bullet with selected etc. So now it's deleted from the hierarchy. That's not delete it from the hierarchy, sorry. So now we want to prefab that bullet, okay, so I just press controls. And all the way to correct my mistake, a couple of steps ahead. So to prefab it, hold, but it drag and drop anywhere in your project folder. But a good way to do it is to create a folder called Prefabs. And then now you can see that that is in my project folded down here, and I put it underneath prefabs, and then up here it has gone blue. Okay, so that shows that has been prefabed, now I can delete it. I can delete from the hierarchy, is completely gone. There's nowhere in my scene, but it's not disappeared from my project because it is still there. So, let's go game object to create empty, that'll do in terms of the location press f2 and stood on its spawn object or spawn point. Lets call it, okay, so go spawn point.
Play video starting at :6:40 and follow transcript6:40
And then we're going to write another script for that right click, creates C sharp script. Just going to call, create bullet, okay, let's double click that. Yeah, wait for a second, so in our bullet, make it public game object. Bullet, and we go, don't need anything in start,

but we need a public game object because we're going to assign that within the inspector. And then update, if input dot get button down, remember, get buttoned down because you only want it true, for the first time that it files. You could use file one which can be left controlled by default or your left mouse button or your main mouse button. I'm left handed, it's actually my right mouse button. You get the idea, your primary mouse button. So if it's that then instantiate. Now I'm just going to open this up here, and you can see it's got, like, one of 10. So the point here is that there's actually a lot of different ways in which you can use the instantiate method. Okay, it's often worth going up and down those things, to see how those changed, and you can see some of the possibilities here. It's not always easy to see what possibilities over to you what kind of, overloads available to in the documentation. But this starts to give you an idea, and it will guide you in what to feed it. What's expected and how to get your data into correct format, to feed into these functions and so on. For now, we're going to leave extension. But, I'm going to say bullet, which don't forget, we've defined the public game object, and it's going to transform.position. So it's the position of the spawn point, that's doing it and transformed.rotation.

Play video starting at :8:39 and follow transcript8:39

Oop, so what this does is when I click file, it will create a bullet at the same position in the rotation of my spawn Point. Okay, I've controlled S, control shift S and I've saved all of that. Let's go back, unity is waiting. It's weighted dragon drop, create, bullet spawn point.

Play video starting at :9:3 and follow transcript9:03

Okay, so remember, if I press play at this point, there's no areas. If I click now, remember I said before, this kind of value here has not been assigned. You will do this a lot in unity, I promise you. Okay, so you've seen that error, you know what it is? They know how to fix it. So it's waiting for an object to be assigned, so let me just do that again. I clicked and held this and I dragged not into their by dragged and dropped it all the over here on my inspector penalty. Other way, you can do is you can click this little circle thing there and it'll bring up the select game object. You can select anything that you can see. If you've got a big project, you can search and you can make these objects bigger or smaller, etc. Okay, so by that bullet, now let's play.

Play video starting at :10:4 and follow transcript10:04

Now, I click, it creates a bullet, if you want to see the effect that the position and the rotation has on it. Now that hasn't changed its rotations. Because up here at the global select, if I select local, there you go. You can see that has moved, but you can also see. The rotation of that has moved, that's the spawn point. Just click play, there you go, okay. Very simple, that's how you make a prefab instantiate it. One last little thing before we go, you notice that when I create these bullets now, if I just click, click, click, click, click, click, click, click, click. The other, the other, the other, the other.

Play video starting at :10:57 and follow transcript10:57

Okay, so I got closer, but it's going on here. Let me just pause my project because I want us to have a look at something. You can see all those objects are still there. Just because they're offscreen, doesn't mean they deleted themselves, okay? And in fact, there's our game screen, let's go to our scene view. You look well, hey, they are still all there, so there might be outside, but they are most definitely not out of mind of the computer. So, what's a white to sort that well, one way, can be to just create a little destroy self script. Now, you can create a script that destroys itself calling to certain conditions. What can be just nice and simple is to create a script which I'm going to do now. I'm going to call it something interesting, like destroy itself. Should have been using capitals for my scripts, I'm a bad person. Okay, so and destroy self, what you do is just put in the beginning of, in the start, just a single line destroy. And then it takes two arguments, the kind of, that can be it could be two overrides, cannot be destroyed subject. If you want to have a delay, you can use that second override, so destroy game object. That just means the game object to which this is attached. And then, you can put time, don't do that. Seconds, just do it, seconds, I need to just make a civilized field. Oops, flight seconds, okay?
Play video starting at :12:45 and follow transcript12:45
Just go back to T and T, I'm going to drag and drop to destroy self onto the prefab. Not anything, my hierarchy, onto the prefab because I want it to. So, if I did not 0.5 that make it easy to see, I want it to apply to every single prefab. And now, when I press play, okay, you can see that those are destroying very quickly. And because it's live, I can click, 12 for awhile, okay?
Play video starting at :13:23 and follow transcript13:23
They are going off screen, but my hierarchy, they started to go down. Okay, Sigma Z, okay. It could just be a very quick and easy, quick and dirty way to keep things from cluttering up the memory of the computer. [COUGH] And make sure it's not too many objects flying around, and that can destroy self script. Could be attached to any object you wanted to, and you just go to the object and change the the top of the seconds, in the in the field there so it can be used on multiple objects. Okay, that's just a useful thing to keep your hierarchy clean and less cluttered.

Okay, so just very simple set up to show you here. So when you start to use you're starting to develop, your game, design your game, you want to make a decision. About whether you're going to use forces, whether you're going to use the transform translate type method that I showed you in the last video. If you are using transform, translate, rotate, and similar functions like that, you cannot use forces because what you're doing is moving the objects position outside of the physics simulation. So a lot of people actually elect to take advantage of the very powerful inbuilt physics engine within unity and use forces. And that's what we're going to have a look at here. So to get a couple of objects to detect and you need a couple of things, so let's just set something up here.

We're going to create a we're going to create a plane, resize that slightly and then I'm going to make an object. I'm going to create a sphere. Okay, just move that we can make that a little bit bigger, I suppose. Okay, that's great. So we know the free press play on this, okay nothing's going to happen. That's just, put my camera alone review that we go. So let's have a look at that.
Play video starting at :1:27 and follow transcript1:27
Okay say that ball is not going to do anything. But are playing here. The floor has got a collider you can see over the side. And are sphere has got a collider. Okay, so there are collisions happening between those two. But what the sphere needs, is a rigid body. So the component, and then you can type in the search thing here. You start typing RIG. We've got rigid body. Okay, so this is what a rigid body component looks like before we start looking anything like that, let's just click play And our sphere drops. Okay, so we have a sphere that actually acts under gravity.
Play video starting at :2:18 and follow transcript2:18
Say that again particularly interesting, but you can see a little boom there, so there's a bit of a simulation going on, so it lands flat there. If I press play.
Play video starting at :2:32 and follow transcript2:32
Just pause it in fact, let's just split a window up here. I'm just going to minimize my console window because we don't need to see that. I'd like to see the movement handles here on our sphere so you can see how it's facing.
Play video starting at :2:50 and follow transcript2:50
Okay, see that's not rotating. It's not moving at all. But if I start to just tilt that plane, let's just watch the sphere.
Play video starting at :3:6 and follow transcript3:06
You see, that's starting to act under physics. Let's starting to rotate. Okay, so very nice. So what would happen just to show you that this is not all exactly the same, full of it. So I'm just going to rotate that sphere back a little bit. It's not quite so steep so we've got our cube. They're just going to move it over there. Let's get our game object to make capsule.
Play video starting at :3:36 and follow transcript3:36
Make it a bit bigger. [INAUDIBLE] You'll be able to see that there really is physics working here. No, you won't. Hopefully you've noticed my mistake. Why do I need to add, any to do with your body to both of them? So there's my cube, and there is my rigid body. Okay, so rigid body and the capsule rigid body on the cube. Rigid body sphere, right?
Play video starting at :4:11 and follow transcript4:11
So things will act differently according to whether they're a cube or can actually see the capsule is going a little bit differently. Come to a phase now. It's not going to roll over and roll all the way down, and the cube isn't either. But of course, if I probably.
Play video starting at :4:28 and follow transcript4:28

Or take that little bit [COUGH] ,then we'll see some different behavior. So it's a nice, simple there. It falls down flat. So you're going to see a nice, physics simulation in unity that's already built in. There's a lot to take advantage of that, which is great. But what's needed for these two things to collide, so just remind you the plane doesn't have a rigid body on it. But all three of those objects have a rigid body, and all of those objects have a collider. Now what happens if I disable and all I do is I selected my plane was going to the mesh collider. What happens if I disable the mesh collide on the plane? Okay, we're just going to fall through, and then what if let's put that back on.
Play video starting at :5:16 and follow transcript5:16
And what if I disable my box collider on the keep.
Play video starting at :5:28 and follow transcript5:28
Okay, so you can see there's a rigid body in my cube. The cube has a collider. Some collider had a rigid body. But what needs to happen here is that for any two objects to collide in unity, both of them need to have colliders, and one of them at least needs to have a rigid body. Now the rigid bodies are to provide a body of mass for the physics engine to work upon. But the colliders are for detecting collisions, so they both need colliders. One of them needs a rigid body they don't both need. Rigid bodies may not be appropriate or useful, but they do both need colliders, and at least one needs a rigid body.
Play video starting at :6:11 and follow transcript6:11
So let's delete out some of our objects here. Let's just focus on our capsule now, and let's have a look at some of the options on the capsule so mass. The bigger your object is, then the more mass it will have like per square meter, as it were. But you can make an object heavy and retain the same size just by dialing that up and you'll see it acts very differently. Drag is to do with the amount of drag it has movement wise. Angular drag is to do with the dampening of a turning force on it. And that would be talk force on an object. Does it, should it use gravity or not? Yes I should probably just float in the air there.
Play video starting at :6:58 and follow transcript6:58
It's not using gravity will just float in the air, but the condition that would make it move here as if there's another object there, there was knocking it, and then we could actually how we solve this. Let me just see if I can. So I'm going to use this view. I just want to make sure that Q is actually above my, so in case you missed it, I corrected the Y here. So I'm looking directly down the Y axis and I toggled my perspective, my isometric, and it just allows me to exactly positioned that cube. So it's over the capsule. So my capture rigid bodies there, I'm going to take gravity off.
Play video starting at :7:45 and follow transcript7:45
The keep does use gravity so the capture will float. But watch this.
Play video starting at :7:58 and follow transcript7:58
That everybody is on.

Play video starting at :8: and follow transcript8:00
It's the box collider. [LAUGH] There you go two colliders. So there's an example of where, there's too rigid bodies. They both got rigid bodies, but they need collides to detect it, right? Yes, that's what I was looking for. So you see how the capsule floats in air? It's not acting on gravity, but it's still under physics simulation. If it gets knocked by something. So this can be used some interesting effect. The cube is that you can continue to see the capsule float around, but it's not under gravity, because it's about to take off again. So you can mix and match these things to some interesting effects. So, we go back to what we're looking at. So use gravity. Yeah, so explain is kinematic. So is Kinematic means, is it simulated by the physics simulation? Now, if I put this kinematics in now, the box will fall, but the capsule is fixed in space. Now, it has a rigid body, so it has physics. It has mass, but it's not been acted on by the physics simulation. So, physics is kinematic is good. If you're trying to put a platform in it, if using physics in making a platform again. You want your platforms to be kinematic, because you don't want those platforms platforms that are in mid air to be acted on by gravity. You don't want things to be knocking them out of place. You don't want gravity to be bringing it down. You can move those items independently, but they still get hit by other physics objects, and that can be very, very useful. That's what is kinematic is for. And then interpolation. And we're not going to go into this here, but has to do with how the physics is calculated. You can read more about that in the documents. Remember, click the question mark if you want to go read about that in the manual straight away. And in collision detection. So where this comes useful is where if you have a very fast moving object, like a bullet in one frame, it could be. If you've got an object, you're trying to hit your bullet, your bullet could be one frame, one side. And then if it's traveling really fast, the next frame it could be calculated to be the other side of your object, and they haven't actually collided. So you would change options here. So discrete means, it will just calculate collision frame by frame, so you could skip through your clip, through objects. In that way, continuous will do all the calculations for every single position in between. Now, that is more expensive to do. But that could be what your game needs, in order to react accurately to what's going on.
Play video starting at :10:41 and follow transcript10:41
So there's some options, and then the last thing there at the bottom constraints is really useful. If you want your object to rotate in some axes and not others say, for example, earlier on when we had up my player controller moving and rotating, if I wanted to do that with forces, I would freeze the rotation around the X and Z axis, because I only wanted that player to move around the Y axis. So that's the kind of thing that's useful for, let's do on the cube shall we, because the cube is falling. If you froze it just in some axes, okay, so it's not going to go very far. Let's move, or capture it the way.
Play video starting at :11:36 and follow transcript11:36

Okay, so it's not going to rotate into the three axes. Therefore, it's not going to really go very far, so that could be useful for constraining your objects. Okay, so let's look at some scripting using forces. So I've cleared my objects out, I've flattened my plane. We're back here again with my spawn point, so I created them to your object up here. Spawn point, I got to create but it's script on there. You can see here, I've made a reference to a bullet. If I drag and drop might, so my bullet prefab is back. I've already added a rigid body, so if we press play, you'll see that will sink.
Play video starting at :12:20 and follow transcript12:20
And I also added some particles, I'll show you in a second. If I click my main mouse button, then those bulls just stack up, okay? But when they move, they emit particles. So what I've done there, just very quick thing. I'm not going to go into particles right now. Particles great fun, though. Let's go and open up the prefab, so I've clicked on the bullet in my project, I've gone to inspector. Click to open prefab, or you can double click it in the project for you. So here's the object here. If I go over to the hierarchy for the prefab, you can see I've got two objects here. I've got the main bullet, I've got an empty game object. This is called trails, now attached to the empty game object called trails. I've attached a particle system, and all you need to do is a component part p-a-r-t, they go particle system. Now, let's just show you quite hell complicated. Why, I'm not going into now. This is the particle effect editor. By the way I got this to do what it's doing, is you can play around with all these things here, but to get the particles to emit as I have made them, I've changed the starting color. The emission here, the emission tab. You don't need all these tabs. There's a basic one up here, and then all these are added and removed as you need. It will come up rate over time by default. I put rate over distance that means as the object moves, it will emit particles. It's good for bullet trails, etc. The shape is the shape of the emitter, rather than the shape of the particle. The particle you change with the texture. And then I've made that. So if I move over here, you can see the shape is a cone.
Play video starting at :14:5 and follow transcript14:05
And if we you can see if I move that around, then it creates particles as it moves. Okay, so it's in the shape of a cone. I haven't done too much though. I just fiddle these values a little bit. And then on the render at the bottom, you would add in the material the default particle physics. Sorry, the default particles system material. That's all you need for that. Is a good idea when used particles to child them to the main object, so that you can position those particle systems directly. So if, for example, if you had a rocket and you wanted particles come out your rocket, you would make a child object to the rocket, and attach the particle system to just that empty game object. And then, you can exactly position those particles coming out the exhaust, and they're not coming out the middle of the object, which would be silly. So that's where you put particles in. It's not colliders, itt's not forces, but it does make it look pretty. So it's got a rigid body on it already, and

it's acting under gravity. And I'll tell you what I will do, is I'm going to double click on my bullet again, and I am going to take off gravity from the rigid body.
Play video starting at :15:19 and follow transcript15:19
And then when it plays, okay, so you can see they're bumping into each other. But it's not using gravity to sink down, but is continued to travel like that. Let's give, Our bullets some forces, shall we? So, I've just created empty script, tickle forces bullet movement. So we're going to need a couple of things to start off with here. I'm going to need to make a reference to a rigid body, so I'm going to say rigid body. Just call it our base quite a standard way of doing that. You don't need to serialize field, because we're going to get it in script. And the next one, no, we're going to do serialized field. And that's going to be float force. So it's similar to how we used the speed variable in previous videos. It's that force control how fast it moves. So in start, we need to make a reference for our rigid body. So the RB, what is it? It's going to be the component that's attached to the game object, that this script is attached to. That's going to be the rigid body, okay? So getComponent, rigid body, not a problem. So we've done that. And then in, we're going to start with, that is going to we're start in now I'm going to change this to, not update, but fixed update. So the difference between update and fixed update is this, fixed update happens a regular, 0.02 of a second I think, it happens at a regular interval every time. So we talked about delta time in the last video. The delta time between each frame with fixed update is exactly the same. The time in between the frames for the normal update loop is variable, depending on the performance of the computer. And you want anything to do with physics, anything to do rigid bodies, anything to do with forces to go in fixed update. Because you want those update loops on all your different objects that are acting in your scene to happen at exactly the same time. Be calculated at the same time before they've reentered the screen. You don't want those update loops being all at different times, being all out of sync, it will lead to some crazy physics with your objects. So that's why you use fixed update. So in fixed update, we're going to put rb.addForce.
Play video starting at :17:47 and follow transcript17:47
And for the time being, we're just going to do transform.forward times force, okay? There are a few extra things to put in there, but not doing it quite yet. So force put movement, that's great. Let's go back to Unity, wait for it to sort itself out, drag and drop that onto our prefab. Have a look, inspect on the prefab, make sure we assign it a force, I'm going to give it 20. I've just done that here. And let's play. So you can see there, it starts to move, and every fixed update it's applying the force to it. So it's a bit slow to get started, but once it gets going, it's in a straight line, I haven't put the destroy itself script in here because it's not important. And those little particle trails, which are nice, okay. So it's adding force in each frame, that's good. What can happen sometimes is if you use transform forward, so if we can go back here, if you transform forward, this is just something to be aware of, it will move forward in the direction of the local object itself. If

you want the object tp move forward in terms of world space, then you would use vector three.forward. So let's just flip back to our object here now. If I just move our spawn point, so it's facing a different direction.
Play video starting at :19:26 and follow transcript19:26
Okay, so now our bullets are going in a different direction, but local said it's facing that way. But if I click to global, this is why I turned the spawn points going that way. So now you can see global goes one way, local goes another. So at the moment I've got it set to transform.forward times force, and it's going in that direction. Let's lick back to the script, say I didn't want that, I wanted it to go in the globe, the world space forward, okay. Save my script. Unity is thinking about it. It says yes.
Play video starting at :20:4 and follow transcript20:04
Now, those bullets are not going in their forward, they're traveling in the direction of global forward. So these are little distinctions to be aware of. Often, You probably default to transform.forward because you want it to come out the front of a gun or something like that. So where the gun is facing, you want it to go in that forward direction, not in the uniform forward direction or up or down or right or whatever of the world, okay? So, we got our that's transform versus vector three. Yeah, so there are different kinds of force modes. So we're putting up to see a huge amount of difference here, let me just comment out that line. And in our start method, I'm going to put something a little bit different. I'm going to do, well I'll tell you what we're going to do. We're actually going to copy paste this line up here, but we're going to add another field. Remember there's several overrides to be had there, you can see, forceModes. So if I do forceMode.impulse, Let's see how that works, shall we? Okay, so that's
Play video starting at :21:31 and follow transcript21:31
So now if I click, So for Start, you can see it's coming out instantly.
Play video starting at :21:44 and follow transcript21:44
So it's not waiting for anything to happen, but what I will do to show it's a bit clearer, is I left gravity off. And my one there.
Play video starting at :21:57 and follow transcript21:57
So let's see how gravity works. So you can see it falling down. I'm going to reduce the force here. Maybe the force is just a little bit too much. Right, there we go, this is more I was looking for. So we force impulse. This means it's just given a short, sharp shock one time. So it happens in one frame, [SOUND] it's given an instant force like that. If you use the other mode, and let's go back and show you that I've activated those things a little bit more. Un-comment one, comment the other. And just to prove to you, so the default is forceMode.force. And this is where a type of force is applied every single frame. So you're using the fixed update loop. Unity is thinking about it.
Play video starting at :22:56 and follow transcript22:56
Okay.
Play video starting at :23: and follow transcript23:00

So that is actually applying forces continuously. Now, the reason why that is very strange
Play video starting at :23:12 and follow transcript23:12
Is because look at, it's acting on the simulation, right? But it's moving around everywhere. So I've already given it a little bit of force, but it's starting to writhe on the floor because it's trying to just keep going forward by a little bit every time. So this might be the reason why you get some strange movements sometimes. So there you go. There are other modes there, so you have four modes. And the other modes are acceleration, and I'll show the you code, it's easier. So Force mode, so you've got acceleration and you've got velocity change. So force takes into account mass of the object and velocity change takes into account the mass of the object, impulse and acceleration don't. They will just make it go faster at a certain rate, irrespective of the mass. So that is the difference there. Okay? Yeah. And so there's just a little bit of basic coding in the force differences, the force modes. Generally, you're probably going to want to use the impulse a lot for objects that are being lobbed, anything that's been launched. If you don't want it to go in a straight line, you want it to be lobbed like a cannonball, or an object has being thrown by a character then you use an impulse mode. And you can play around with the force, the force numbers, and the mass and the drag to get the feel of the game that you want. Okay, so just a little explanation of the different force modes. So if you look here, it's easier just to bring this up in code. So if I go to forceMode, go on there and if I print dot, it's going to bring up the four types of force mode that you have there. So you can see, you can actually toggle over it for little explanations. So acceleration, that is the continuous force. So it's ignoring its mass. But the force considers its mass. So the top two acceleration forces are continuous forces, one ignoring the mass and one taking into account the mass. So acceleration ignores, force takes into account the mass. The two below it, impulse and velocity change, they are the same, but for the one off instantaneous force. So you can see here, for impulse, it has an instant impulse force using the mass. But velocity change ignores the mass. So those are your four different kinds of forces. And again, it depends on the look and feel you want in your game.

Here we have a basic scene, just we have a plane and the warm that I've set up there, I've got spawn point and got a ball that's just a cubed that's stretched. I've got my crepe but it had been there, and if I click here, then we've got some bullets firing into the wall. This is where we've started. First thing to look at is there are two main types of collision going on here, you've got your colliders and you've got triggers. What do we mean by that? Let's highlight our cube here, which is acting as a wall, and let's zoom in on our box collider here, you see there's an option here, it says is trigger. If we click that and then we run this again and see how those bullets are just going through the wall. Is the collider disabled? No, it's not disabled. But what are the difference between these two is

that, if you use a collider, that's when you want something to be simulated by the physics engine. If you make something a trigger, it's not going to have any physical presence within the game world, but you can still use it to detect collisions. An example you might use in a game is if you're walking through, say, a first-person shooter or first-person anything kind of game and you'll pass a certain point and then an enemy spawns or a sound emit, a piece of narrative fires, and you get a cut-scene. That's the kind of thing you'd use a trigger for, and you want that to be a trigger rather than the collision because you don't want the player to bump up against collision, you just want to detect when they walked through an area and that's when you use a trigger.
Play video starting at :1:54 and follow transcript1:54
That's what triggers are. Can you still detect when it goes through a trigger? Yes you can. We're going to cover that in the next video. We'll do a bit scripting, it's easier to show way and how you might use that. Other things you can do, let's continue with our box here. If I wanted to edit that collider, well, it's simply, you can see there's this Edit Collider function here. If I click that, then you can see now that I've got these handles that appear and I can click and drag those as I was manipulating, because this is a Box Collider. Click and drag those as I was manipulating the faces on a box, which is my study. I'm just going to zoom in a bit there, I got a bit lost there. I can do that by hand using my hand was here, but I can also do it using the Inspector here, so I change the x and change the y like that, and then same with the size. If I wanted to scale it in both directions at once, then I can do that, be careful going into negative for the size because then that invest in your collider and that can get a little unpredictable. These are the kind of things that you can do and it's going to undo that, so I get back to zero. Places where you might use that is say you've got a player character that you don't want to actually put part of the body into the wall, you want the collision to be a bit further out from the wall, and then just bring your collider out here, that means your character can move up to this bit, but not actually because they're either walking into the visual mesh in the game. That's a really common technique in video games, and you'll notice this if you look for it, you're coming up against invisible walls all the time and in fact around complicated bits of scenery, so you've got a complicated fence, et cetera. What we will do in video games is we will draw a very simple collision mesh around there so that you don't get stuck on the visual mesh. It's a very common technique to use. You can edit your collider like that. We've got Box Colliders. You know that we've got other things like capsule collider. I'm looking at the 3D ones only here. Capsule collider, sphere collider, etc. But what's this mesh collider? Well, lets bring in an example. I've got my, I pressed the wrong button I've got my thing there. Here's an object that I made earlier. Of course, this apparently is a 11 foot 12 century Colombian, I believe, Jaguar Vase. Here's an object. I've just got that here. You can see up here it's a Jaguar Vase. You can see that all I've done is bring it in as a 3D model, I've just gotten model, I've got no collision on there at all, it's just a visual mesh. Nothing would collide with it. Fine. Let's go ahead,

and what would I do here? What your first port of call in many cases, if you haven't got a basic model is just to add a mesh collider, so I've just gone mesh there, I'm going to click "mesh collider", and it adds a mesh collider. But that's great. Now, mesh collider has an extra box here called convex. Now, what does convex mean? Convex means that's going to make a mesh collider to try and solve approximate your object, and it's going to ensure that there's no holes in it at all, so it's a completely solid mesh. Let's do this. Click "convex". That's done a fairly good job of approximating that. You can see it's done the best that it can, but there are a few caveats to be had here with mesh colliders. First of all, you can see is not actually mimicking the shape of our object completely and that's because mesh colliders can only be up to 255 faces. You can see actually in our area here at the bottom couldn't create a convex specialness source because within the maximum limit, the partial hole will be used consider simplifying your mesh. Really isn't that complicated? If you want some, let's just turn the mesh collider off to show this. If you want some illustration of how this is going, I'm just going to open up the pre-fab because the lighting's a bit better to show this. Here's our thing. If you go into the top left here, I'll go to my shaded wireframe view. Holy the cow. Yes, that is a complicated mesh. Each of these triangles is a face. There's a lot faces. The reason I illustrate this is because it leads into my other point. I'm just going to drop back into scene view now. This leads into my other point in that,. Mesh colliders can only be used for static objects that are never going to move. It is way too expensive to be using mesh colliders on anything that moves.

Play video starting at :7:19 and follow transcript7:19

You've got your mesh collider on this object. What if you've got a complicated visual mesh and you actually want it to move and do something, what would you do? Well, the answer is, you wouldn't have a mesh collider. I'm going to remove my mesh collider and you would add a primitive collider. Maybe for this example, I'm going to use a box collider. I'm just going to make this a little bit bigger. Let's go into my isometric view. That's pretty good there. I'm not actually gaining or losing a huge amount of identity just using a box collider but the box collider is only got six faces to it.

Play video starting at :8:10 and follow transcript8:10

It's worth using primitives wherever possible, really. In a way, try and avoid mesh colliders unless you really need them. You can combine colliders together. I could add another one here. It'll probably be best off doing this on a child object by Canada sphere collider if I wanted to. This would also be helpful. I'm just going to go to my sphere collider. Lets go back to perspective because it hurt my eyes, and I can actually move the sphere collider up and down if I wanted to get a slightly more accurate view on things. If I wanted to use the sphere for the middle body bit there, and I wanted to use a box for the more chubby bit, I can maybe do a box on the legs, that would still be much better using this primitive colliders, than using a mesh collider because the engine is also optimized for each of those individual primitive colliders. That's one option, the

other option, I'm just going to remove this sphere collider, the other option would be to create your own simplified mesh. What you can do is you can create a simple mesh that's not a sphere or box or capsule, it's a simple polygonal mesh, you can create that in a 3D program, import that and use that as your mesh. The way you do that so if I move my mesh collider via the, if I move the box collider over and add the mesh collide again, you can see that the mesh here is a default mesh. Now what you can do if you click that, you could use another mesh that's here. We've got our default mesh there, which you won't use for the mesh collider, if you made it convex, then you can bring in a different mesh and assign it here, it'll be within your project and it will just could be a flat model and it won't represent it as a visual mesh, it'll just have the wire frame model used for collision there. You can do that in an external 3D package or you can do it within Unity, as part of ProBuilder if you wanted to. Again, ProBuilder as I mentioned previously, is beyond the scope of this course, but it is there for those of you that have those skills, and want to use it or wish to learn it.

Play video starting at :10:28 and follow transcript10:28

Just to finish this off, an extra little thing that you can get to improve the efficiency of your game sometimes, is to get your layers selectively colliding with each other. The way you do this, let's go up to the top left then go Edit, and then go up to Project Settings, we're now on Project Settings window here and I'm clicking on, it's come up on Physics but you click on Physics. I'm interested in this bit down the bottom. This is the collision layer matrix and what you can do is you can change which layers collide with each other. At the moment, I've just got my defaults there, but here's a way that you might use it.

Play video starting at :11:9 and follow transcript11:09

I'm just going to dock that window for moment, because we'll come back to it. It's back to Project Settings in there, now on any object, doesn't matter, you go to your Inspector and we've previously talked about tags, but let's still look at the layers here, this is your collision layers. You can see the default ones that we had in the collision matrix before, I'm going to go to Add Layer and then in the same way that you add a tag, we can add tags here. Maybe we had bullets and we want to consider bullets as one layer and we want to consider enemies as another layer, so we've done that, and then if we go back and select our objects, then in the layers, we can say that, let's just say for example this is an enemy, we can tag this as an enemy, and if I go back to my Project Settings, you will see now that we've got bullets and enemies there. Now if I didn't want my bullets to collide with all the other bullets in my scene, say I didn't want the bullets that are shot in the scene to bounce off other bullets because whilst that might be vaguely realistic it's not actually what you expect to see in a game, I just turn to check that and in one fell swoop, you've just ignored a whole load of collisions, it's a whole load of collisions that the engine doesn't need to deal with, it doesn't need to calculate them, it doesn't need to detect them, it'll make your game run faster, and then also you'll change the behavior so

it's more the behavior that you would expect. Maybe, if you had enemy bullets, you wouldn't want the enemy bullets to collide with the enemies and you can uncheck that and so on and so forth. That's the type of place we can get fine control, over where things collide with each other and then assign them to each of your objects, as you desire. Again, I'll go to my bullet, I can select my bullet object now, and then I could open a prefab, vested in the prefab, just show we're in a prefab window in here, I can just select bullets. Do you want to do that for the children? No subject only actually, I'm doomed for this because it's asking about the trails objects underneath, but there's your bullets and now that'll be calculated in the collision matrix as a bullet. Good options to give you control.

Okay, so back to the same basic setup, our wall is there. Bullets are spawning, and they hit the wall. Okay, great. So, you can see that the wall has got a box collider, but it doesn't have a rigid body, but my bullet does, it has a collider and a rigid body, it's because the wall is not moving so it doesn't need a rigid body. It doesn't need to be simulated by the physics, but it does need to have collision in it, but the bullet is being simulated by it. So that's great. So the bullet hits the wall and then drops. Can we detect this in code? Yes we can. So I'm going to go down to my scripts. I'm going to create a C# script. Let's call it something interesting like bullet collision, okay. I'm going to bring that up in Visual Studio.
Play video starting at :1:4 and follow transcript1:04
There you go, all right. So, what we want to do, so Unity has a bunch of built in methods that we can leverage here. So we're not going to look at Update or Start for the time being. We're going to do void OnCollision, Enter. So you can see that it populates this here with void OnCollision Enter, Collision collision as an argument. So you can just leave that as is, and that's fine. So let's say something like this. Well, first of all, you can just debug.log, sorry, not speech marks, collision.gameObject.tag.
Play video starting at :1:52 and follow transcript1:52
And what this will do is whenever the bullet collides with something, it will look at the game object, that's the tag that the game object that collider is attached to and look at the tag of the game object and it will print it to the Windows. Let's just go back here, drag and drop the bullet collision onto the bullet.
Play video starting at :2:14 and follow transcript2:14
Go down there, just going to click that and then clicking on window, wall, they go floor, wall, floor, wall, floor. So you can see whenever it hits something, that's what happens. And I can do that because my wall has been tagged as wall, even though I didn't actually name it properly. And then this plane here, also haven't given that a name, there you go, but I tagged it as floor, all right? So, that's great. And we can make that selective if we wanted to. We can say if collision.gameObject.tag tests to floor, I'm going to debug.log I've hit the floor.

Play video starting at :3:17 and follow transcript3:17
So you can make it selective. And this is a use case scenario, you say, have I hit a thing? Then send a debug.log to help me work out what on Earth is going on in my game. There you go. Hits the wall, nothing, hits the floor, I've hit the floor. Wall, nothing, hits the floor, okay? So that's a function to see that we're hitting the collision, okay? Now, let's just change the code slightly here, because what I want to show you,
Play video starting at :3:49 and follow transcript3:49
So if we Change that so that it hits the wall, Okay, let's just leave it as it is. If collision.gameObject.tag tests to wall, then destroy collision.gameObject.tag, okay? It has to be a game object that's fed into the parameter for destroy. So if you hit the wall, we're going to destroy the wall, all right? Let's go back, you need to think about it.
Play video starting at :4:29 and follow transcript4:29
Let's play. Yeah, okay. So let's click. Now what I want to draw your attention to is when we play, watch how the bullet hits the wall. So it hits the wall, the wall destroys, but it's still acting as though it hit the wall. It's still acting as though it hit the wall and that affects the simulation of the bullet. Now this comes back to what I was talking about between colliders and triggers earlier. What if we wanted to not have it be affected by the presence of the wall? What if we want it to destroy the wall and then just go through it? If you wanted to do that, then the way that you would do that is you would use triggers. So, I said, let's use triggers, so here we go. We've got isTrigger. I've clicked the isTrigger for the wall. So let's run it again.
Play video starting at :5:25 and follow transcript5:25
Okay, it's gone through the wall, but nothing is firing in my console down here. Why is that? The minute you've made it a trigger, it's not a collider anymore. It's a trigger rather than a collider. So if you wanted to make it a trigger, ooh, I did the return to the wrong place. If you wanted to make it a trigger, you got to use different function and that would be void onTriggerEnter. Now, I'm being a bit lazy and just typing on with a small O, but please make sure, and this is a feature I see students trip up on a lot, make sure that your onTriggerEnter is a capital O at the beginning, okay? I mean, Visual Studio will help you, but just to pick that up. So if you do onTriggerEnter, then this will happen. So if we do debug.log, hold on, if we do if, now notice here That we're not using collision, it's collider, okay? And it's given it, to help you tell difference in the code, it's given it another name. Every time there's no real reason to change that steps to other. If other.game.Object.tag tests to wall, I can say, debug.log went through a wall.
Play video starting at :6:57 and follow transcript6:57
Okay? Let's go back.
Play video starting at :7:3 and follow transcript7:03
And now let's play. You can see this went through the wall. Now just to try and show you can't have both things going at the same time, I'm just going to angle my collider down a little bit.

Play video starting at :7:26 and follow transcript7:26
Okay, and I will just make my bullet force a little bit. So I want it to hit the wall.
Play video starting at :7:33 and follow transcript7:33
And the floor.
Play video starting at :7:35 and follow transcript7:35
Just to show that we'll get both functions firing. There you go, went through the wall, I've hit the floor. So we still got both things firing, and you can see it rolling off down the bottom, there you go. So both functions are firing, okay? They don't interfere with each other. If you've got lots of triggers, you need to check which trigger it's gone through or what the other object is. And this is the standard way. So if other .tag, you can use .tag, you can use name, that could probably get a little bit fiddly. You could test whether there's a component present on the other object, which you could use to typify your objects. So, if it has component or if get component equals not null then do this. So these are ways that you can use onCollisionEnter and onTriggerEnter to sort out which things are happening. In addition to the collision matrix that we talked about in the previous video, these are other ways to fine tune the interactions between your collision. So, yes, it's similar code, and just the other thing I wanted to highlight there is just that, we're not seeing it quite so much now that I've rotated it, but that bullet is not affected by the walls presence whatsoever. So if I, actually to make it equivalent, I'm just going to delete my onCollisionEnter, I'm going to delete my update for now as well actually because we don't need that. Through wall, destroy, then that could be other.gameObject.
Play video starting at :9:7 and follow transcript9:07
The thing that we want to show here, in contrast to how things were before, I'm just going to rotate my spawn point up a little bit to how it was before, that should be all right.
Play video starting at :9:21 and follow transcript9:21
See how that bullet isn't affected by the simulation. So when we had a collider, it stopped the bullet, the bullet fell, although it actually hits sidewall. But now that trigger has no physical presence, it's not affecting the simulation of that bullet. So these are the reasons why you have collisions and triggers because you don't always want things to be affected by the physical presence. And this becomes a common thing, you'll find that as you go through a box that you want to disappear, you think, well why is my player stuttering as it hits it? Because it's a collider because it's stopping your player for moving, and so you want to turn that into a trigger, actually, albeit a trigger of a visual mesh, but you want that to be a trigger.

Hello. In this topic, we're going to look at what I think is one of the most important, useful and really exciting aspects of the animation, which is physics. Now, everything in the world, in fact everything in the universe obeys the laws of physics, and one of the

great things about the laws of physics is that they can all be expressed in mathematics. Why is that great? Well, it's great, because computers are really, really good at maths. So we can just code up the equations of the laws of physics in a computer program and that will simulate all of those physical laws. So that our objects in our program move realistically as if they were in the real world with real physics. And that's exactly what game engines like unity do. They have a physics engine which implements not every single aspect of real world physics, but the most important ones. And that lets a lot of our objects move very realistically but importantly also interact realistically. So it's not just about stuff moving normally, it's the fact that we can pick an object up and throw it, and whatever direction we throw it in however fast, however slow it will respond in the correct way physically. So that gives us a lot of dynamic fine grained interactivity. It's not just pressing a button, it always does the same thing. The fine details of how we throw a ball or how we kick it really deeply affects how it moves. And you can see that with a lot of really exciting physics based games where so many of the puzzles are to do with solving how objects move around from realistic classics like Half Life and the more recent VR version Half Life Alex. Which have physics throughout their gameplay to more sort of casual Two D games like angry birds still fundamentally based on physics. So it's a really important animation technique, and that's what we're going to learn about in this topic.

Play video starting at :2:41 and follow transcript2:41

You've learned a little bit about physics and physics subjects, and hopefully you've made your first attempt at creating physics simulations in unity with using the physics engine, maybe getting some objects to bounce off each other. In this video, I want to go a little bit more into detail about the maths behind some of these physics and in particular to talk about forces. So, we've learnt that in physics you have physics objects. Here's the physics object. And let's have a look at what actually happened to that physics object. Now, physics opposites have a position, and X and Y just like any other object we know about that. That's transformations. They also have a velocity. Now that's the change in position, so the velocity is also a vector. It has an x, y ,z, and so the x is how much the exposition changes in the second, the wise, how much the Y position changes in the second and the Z is how much the Z position changes in the second. So every second you're adding on that other vector and with your velocity, it's the change in position. You can also change the velocity and a change in velocity is called an acceleration. So the acceleration is again how much the velocity changes. It's an X and Y and Z acceleration.x is how much the velocity changes X in the second civilly Y and Z, so you will have a velocity and acceleration will happen to it, and you have a new velocity. Now this change is a vector that's really important that we commonly think of acceleration is just being speeding up, but it's actually different across x, y and Z, so you can as well as speeding up. You can have a change in direction. You can see this new velocity isn't just longer. It is a bit longer, so it'll be a bit faster than the previous

one. But it's in a slightly different direction, and you can also have negative acceleration so that acceleration can slow you down. What in sort of common parlance you call deceleration, so let's say, slowing down. Now what causes acceleration? Acceleration is caused by a force or forces, so a force applies to an object and causes an acceleration. In fact, you could have several forces applying to an object and the acceleration is caused by what we call the resultant force. So if you add up all the forces again, all the forces are vectors. They have an x and y and z. Now, how does this work? It works with this formula, super important formula and physics. And it's actually Newton's second Law of Motion. So Newton, maybe the world's most famous physicist. Maybe but he is known for discovering or inventing gravity, but this is just as important. This is a very fundamental physical laws of how things move. So it says that f the force. His time is equal to a the acceleration times M. What's M, M is the mass of the object. So the mass is a property of the object. It's proportional to the weight of the object, but it's not the weight. The weight is actually the force according to gravity. So in space you're weightless, but you still have a mass. But it's pretty much what we think of on earth as weight. So it's measured in kilograms. You can think of a heavy object as having a high mass as long as you have constant gravity which we are have earth. Say the force is equal to the mass times the acceleration. This means that for a bigger force, sorry for an object with a bigger mass, you need a bigger force to create the same acceleration. Or to put it another way, for an object with a bigger mass the same force will create a smaller acceleration. So mass kind of resists force. Let's look at this in a bit more detail in terms of maths, so, f equals MA force equals mass times acceleration. Now, most commonly we know the force, and we want to work out the acceleration. So you just rearrange equation. Acceleration is force divided by mass. So the bigger the force, the more acceleration, the bigger the mass less acceleration. And then, what's the acceleration? The acceleration is the rate of change of velocity. So this is a differential. If you're not so some calculus. Don't worry. You just think of this as. How the acceleration changes over time, and we'll see a simpler version of this equation in a second. And the same applies to velocity in position, velocity is the rate of change of position. Now, you might not be very familiar with differential calculus. Doesn't matter because computers can't really handle it either, because differential calculus is about changes that are infinitely small. They sort of happen continuously, whereas computer's can really only handle things in discrete time steps. So we have a discreet version of this. So instead of having like an infinitely small sort of chunk of time called dt, we have a finite chunk of time, instead of delta t. Now this is really important, this is what we call the time step, in our physics simulations. And so every delta t, which might be every 60th of a second, it's similar to frame rate in a game of your experience. We would update the velocity, and we'd update the velocity by saying, the new velocity, the velocity at time t plus delta t is equal to the old velocity. The velocity at time t plus the acceleration times delta t, so you're multiplying the acceleration by the amount of time it

happened over. So the acceleration is happening over time, and it's time, times that acceleration. The acceleration you will have to calculate it by the force divided by the mass. And then you do the same thing for the position. So the new position is equal to the old position plus the velocity times delta t time over which is acting. So this is how we do a physics simulation is each time step, which could be each frame. But you generally actually do physics calculations a bit faster than you do graphics frame make calculations. You work out the acceleration, which is the force divided by the mass. You add up all the forces divided by the mass.

Play video starting at :10:11 and follow transcript10:11

Then you use that acceleration to update the velocity. So the new velocity is the old velocity plus delta t times acceleration. And then the new position is the old position, plus delta t times the velocity. So you're updating each of these in turn every time step.

Play video starting at :10:32 and follow transcript10:32

And that's the basics of a physics simulation. So actually, not so complex, you have to remember that all of these things are vectors, but otherwise that's how you would write your own physics simulation. I'll say in a minute why it's a very good learning exercise to do that and I encourage you to do that. But I wouldn't try to write your own production level physics engine because that's actually really quite complicated, we'll see why in a sec. Okay, so let's look how this works in practice.

Play video starting at :11:7 and follow transcript11:07

So here's our object, and we need to give it a bunch of initial properties. So let's start time = 0. Let's say that the force is gravity. Now, gravity is a force that's proportional to your mass. So the heavier you are or the more massive an object the more is pulled down, you know that? So the gravity on earth is actually minus 9.81, so it's minus because it's going downwards and times the mass.

Play video starting at :11:37 and follow transcript11:37

That makes it very easy to calculate the acceleration, because the acceleration is the force divided by the mass. So the 0.981 times the mass divided by the mass is just equal to 9.81. So that's very easy, because what that means is that gravity is actually independent of mass. It causes a constant acceleration as long as the gravity is constant as it is on earth. So you don't have to worry about the mass of the object, so that's quite nice. So the acceleration is always going to be -9.81 force of gravity. Let's give it an initial velocity of 10 in x 10 in y, and a position of 0.

Play video starting at :12:28 and follow transcript12:28

And we're going to do a time step of 0.1. So this is our initial conditions t 0, we have a velocity and position at 0.

Play video starting at :12:44 and follow transcript12:44

Let's move to our next position, which is where we're moving our position according to the old velocity. So our position 0 is our velocity is 10, 10, our timestep is 0.1, so we're dividing by 10. So our new position is 1, 1 and we're also updating our velocity. So we're

subtracting a tenth of the acceleration from our velocity, so I've got a new velocity as well. And we continue doing this, so second time is equals to 0.2. Then we've got subtract two times the acceleration from the velocity, we're subtracting the new velocity. We're adding the new velocity to the position, so it's not going up quite as much. So you can see it's still going across at two units across, but it's only going 1.9 up, and we carry on this simulation. And as you can see, the velocity in y the vertical velocities keeps on going down and we're not moving us so fast in that way. So we're not going up as quickly and as you can imagine, eventually you'll hit a point at which you start going down, because that's what gravity does. But you can see from these equations it does that by constantly subtracting off this acceleration due to gravity each frame. So your y velocity is gradually leveling off and then starts going downwards.

Play video starting at :14:40 and follow transcript14:40

Okay, now I want to do this again, but with a different time step. So now let's look at the time step of 0.3 a bigger time step exactly the same initial conditions. But what happens at time? The first time step you moving on to time t = 0.3 now, and you can see the velocity has gone down quite a lot more, but we're adding the initial velocity times 0.3. So it's 10, 10, it becomes we remove the position by 3, 3 rather than 1, 1 over that first time step to making a big movement. And in that time step, we haven't updated the velocity yet, so it's using the original velocity.

Play video starting at :15:28 and follow transcript15:28

So in a sense, it makes sense, you're spending more time, so you're moving more. There's nothing particularly wrong with that. But if we compare that now with our original simulation, what we see is that in those smaller time steps, each of those smaller time steps we're updating the velocity each time the velocity is changing. So we're moving in a different curve. We're moving in that ark, whereas, If the time step is 9.3, we've gone a long way without ever updating our velocity. And that gives you a less accurate results. So it should be doing something much closer to what the 0.1 time step is doing.

Play video starting at :16:12 and follow transcript16:12

So, what that tells you is that the bigger the time step, the lower the accuracy of the simulation. But it's a trade off because the bigger the time step, the slower the simulation. So sorry, the smaller the time step, the slowest stimulation. You're doing many more calculations. It's much more computational intensive. So you've really got this trade off between if you want to be fast and efficient you want a big time step. But if you want to be accurate, you need you use a low time step, a small time step, and you have to really trade those too often make a decision. And there's lots of much clever ways of doing this update. So this is one way of doing what we call integration. And there's more sophisticated integration methods, which can have better results with bigger time steps. I won't go into that in this lecture. But just be aware. I mentioned why is it difficult to write? At physics engine, one of the reasons is that you really have to trade off this time step issue. Okay, so we've looked a bit at how the math of physics

simulation works and in particular also how it's implemented in terms of time steps really important concept. Now I want to go into a little bit more detail into forces. We've already seen how gravity works. I want to look at some of the other forces you're seeing in a physics simulation. But first, I want to sort of distinguish between two types of forces.

Play video starting at :17:54 and follow transcript17:54

Standard force, which is what we've been dealing with here is a force that applies over time. So gravity is, you're always having gravity. And as time goes on the effect of gravity will increase because you're having more acceleration. And in particularly the equation is something like this. The velocity is updated by the force divided by the mass. The acceleration times delta t. So there is that important. You're always multiplying the result of the force by Delta T. But you've got another type of force which is one that happens instantaneously. We call it impulse. And that's something that happens so quickly that you don't have to take into account time. So this is something you just want to give an object to shove. So if two objects collide, they have a force on each other instantaneously very, very quickly. And this is very useful in physics simulations and games. If it's a game you want to catch to jump, you can just apply impulse to it and the council jump up and then that force will apply once, and then they're moving under gravity. Also, as we'll see collisions happen instantaneously. An impulse happened, so they're useful. In theory, all forces happen over time. But so, if you're really being theoretical there's no such thing as an impulse. But an impulse happened so quickly you can treat it as if it's happening instantaneously.

Play video starting at :19:33 and follow transcript19:33

So let's look at collisions. We've talked about them in the previous lecture. There probably the biggest most important thing that happens in the physics engine. So two objects move towards each other. They inter penetrate. That doesn't happen in the real world, but it happens in physics simulations. And once you've spotted that they've moved into each other, you need to apply force to move them apart. And that is not just any force. It's an impulse because it happens instantaneously. So there's two impulses pushing them apart.

Play video starting at :20:3 and follow transcript20:03

Now let's look at some of the math to understand what's going on. Now, an important thing about physics in general, is that momentum is maintained. Momentum is the velocity times the mass. And momentum is maintained in a collision. So what that means is the new velocity of object one that's V dash is the new velocity. So the new velocity of object one v dash one times and m dash one, plus the new velocity of object two v dash two times m dash two, is equal to the old velocity of the object one times the mass plus the old velocity of object two times its mass, the momentum. So we know that total velocity sum doesn't change. But that doesn't tell us much about what each object does. Because what can happen is that the objects can transfer momentum to

each other. So if you're thinking about snooker, billiards pool, whatever variation of the game you like. If one you hit the white ball and then the white ball hits a colored ball, then the color ball is moving but once the white ball hits it, the color ball starts moving. What's happened is, the white ball has transferred some momentum to the other ball and it bounces off. So you can transfer momentum in collisions. And if two objects are moving in the opposite direction, they could have zero momentum. Because one is moving this way, the other is moving this way. So their velocities are opposite, and they could hit against each other and stop moving entirely. So they're cancelling out each other's momentum. So there's lots that could happen in between that isn't explained by this formula, unfortunately. So we need to look at something else, which is the elasticity of a collision. Now and if a collision is elastic then all the energy in that collision is preserved. So, there's no loss of energy there. Object's still bouncing about. What that means is, there will be bouncing off each other maximum. So all they're moving fast and they'll carry on moving fast. They'll bounce away really well.

Play video starting at :22:41 and follow transcript22:41

But a lot of collisions in fact, all real world collisions are in elastic, which means you lose energy. So, that is lost in heat or you can hear sound when two objects collide. That's energy coming off it. So energy is lost in some way. And so there could be not as much bouncing or zero bouncing,. So as then they could just collide into each other. So all real world collisions are in elastic, but we can treat a lot of them as elastic because the energy lost is minimal. And this elasticity is controlled by something called the coefficient of restitution. Now that is just the some of the two velocities after the collision, divided by the sum of the two velocities before the collision. So if the coefficient is one, that means that sum of the velocities is the same going out as it is coming in doesn't have to be the same two velocities. It could be like the snooker balls where one of them is transferring some energy to the other one so that one starts moving. But the total velocity is the same. So lots of bouncing, elastic collision. If the coefficient of restitution is less than one, it means that the velocity going out of the coalition is less than the velocity coming in. You've lost some energy. It's an elastic and in particular to zero. Then there's no velocity going out. They stop moving, they just stick together.

Play video starting at :24:24 and follow transcript24:24

So coefficient of restitution is really important parameter. And it's one that's featured in physics because no one really knows what the coefficient of restitution is. It's given a slightly more intuitive name bounciness, because it controls how much about things bounce off each other.

Play video starting at :24:38 and follow transcript24:38

So let's look at another force. So it starts, something's moving in there.

Play video starting at :24:43 and follow transcript24:43

It doesn't just carry on moving, it's opposed by a drag. So this is air resistance, or it could be water resistance. The same thing happens in water or any other fluid or gas.
Play video starting at :24:55 and follow transcript24:55
So, let's have a look at the drag. This is the equation for drag. Now there's various factors in there. Let's look at it from left to right. Rho here is the density of the air or fluid, so the denser the air, the more resistance there is. A is the surface area. So the bigger the object, the more resistance there is. And then there's just a coefficient of drag, which is see, C which is just another parameter, which controls it. And in unity, you can control another to the physical material is the coefficient of drag C, which kind of also can include the density there.
Play video starting at :25:44 and follow transcript25:44
And the most important thing at the end is that this is all proportional to velocity squared. So the more velocity, the more drag. So it's really opposing velocity. So if there's zero velocity, if you're not moving, there's no drag. But once you start moving, there is drag. And because it's squared, it goes, it's not disproportionately. It really ramps up quickly as the velocity gets quicker. So low velocities drags very small, but then it gets really big. And what this means is that you get something like it called the terminal velocity. So if you're falling under gravity, your force is always going to be the same of gravity. But as the velocity gets bigger, your drag force opposing that is going to get bigger. So eventually your drag force is going to increase so much that it's equal to your force of gravity. And at that point there's no resultant force and your velocity is going to stay the same. That's your terminal velocity. That's why parachutes work. They increase air in this equation. They have a really big surface area, so the drag force gets bigger, quicker. And you hit terminal velocity much quicker than you would, if you didn't have a parachute. And so you hit a terminal velocity that's lower than you would. Normally, you're moving less quickly. You don't die when you hit the ground, which is good. Now velocity is resistance due to air. If you're sitting on top of a solid objects, you will also have friction against that object. So you have a fault, forceful movement and in the opposite direction you have a frictional force.
Play video starting at :27:28 and follow transcript27:28
Now that force is proportional to the contact force, the force that's happening between the objects. That's very commonly the force of gravity on the top object acting on the bottom objects. So, the more massive the object on top, the heavier the objects on top, the more friction. There's actually two types of friction. So when objects aren't moving, there's still friction that will oppose you that will resist trying to move. So if you're trying to start a heavy object moving on a rough surface, then what you're saying is actually is the static fiction stopping you moving. And it always opposes the force that you're applying to it. So it's all equal and opposite to the force. Well, at least the force in the plane that's parallel to the surface it's on. So the more force you apply is going to resist, but it only goes up to a certain amount. So the maximum force is equal to the contact

force N times the coefficient of friction of static friction called mu static. So that gives you the maximum force that's going to oppose. When you go over that force, you start moving and you get a different type of fiction called dynamic friction. And this is opposite to the velocity and is equal to the contact force times another coefficient of friction, the dynamic friction, which is mu dynamic. So you've got two different coefficients of friction. One for when it's moving one for when it's not moving and you'll see in unity that in the physic material have two versions of of friction, static and dynamic, that's the difference.
Play video starting at :29:19 and follow transcript29:19
So there we are. We've looked at some of the maths of physics. We've looked at how you implement that in a physics engine with time steps. And we looked at some of the most important physics forces that you're getting a physics engine. So you've got a lot more understanding of what's going on behind the scenes of a physics engine. And you could go away and implement single one simple one yourself, though I've warned you, creating a professional level one is very complex. But I also recommend you just go and play around with the unity physics engine. Tweak some of those parameters, those coefficients of restitution or fiction and see what effect they have. And then try to create some sort of fun, dynamic interactive simulation with that physics.
Play video starting at :30:10 and follow transcript30:10
Great job. You've taken your first steps in using physics engines, which were really keen, important part of 3D graphics and animation. Whether you're using them to create some really fun, dynamic interactive games or to make VR worlds that feel real and really respond to you like the real world. Physics engines are one of the most important tools you can have, for making objects and environments that really respond like the real world. So I hope that you're gonnahave a lot of fun creating physics based worlds. And that as you do so, you'll get a real sense of some of the interactions and possibilities they make forward. So, enjoy your physics based worlds and we'll see you for the next topic.

The stuff that we've looked at so far is moving forward, moving backwards, moving side to side, what if you wanted to turn? We did turning when we do transform translate, but haven't looked at it in terms of forces. It's just a quick thing here. Got a plane. I'm just going to create myself a cube, it's down there. I'm just going to make it a bit bigger. You've got something big enough to look at, focus on that, bring out, and then I'm just going to use my move and then my vertical snap to snap it to the surface, of the plane, okay, great.
Play video starting at ::44 and follow transcript0:44
Here's our cube here. Let's create a script. It's going to create C-sharp script, and let's call that rotate square. Since that is all I'm going to be doing for this very quick video,
Play video starting at :1:5 and follow transcript1:05
and it'll come up eventually.

Play video starting at :1:13 and follow transcript1:13
It's because it wants me to reload some modified topic, that's why. If you change things in between, if you change the scripts like deleting, modifying scripts notes, in Visual Studio or for fit, just be aware of that you haven't broken it. Well, you might break in something but haven't necessarily broken something. So don't overly panic. We're going to look at rigid bodies. I'd say what before I do that, I'm going to add a rigid body. This is Box Collider. There's our rigid body. I need to make a reference to the rigid body, so rigid body I'm going to call it RB. Then I'm also going to use float and we're going to do rotation force. Somebody can modify and inspect [inaudible]. We'll play with that. The Start method, runs once when you start up. Make the reference to RB to find what it is. Then just in, fixed update. Remember fixed updates we're using forces we'll do RB add. Now, so far we've mainly been using force. You can see there's lots of other things that you can go and explore here, explosion force, force at position, et cetera. I'm just going to show you a torque. A torque takes a variety of things. You've got a few overrides here. Let's just Zoom in and have a look at those. You've got a torque, just the torque as a vector three, the force mode and you can do an xyz et cetera, and you can also do the xyz and force mode. I'm actually going to go for the first one, which is the torque now and the torque vector. Now, the vector three, that's asking for that is the axis around which that object is going to rotate. What I want to put in there is I'm going to put vector three.up and If I want to do that local to the object, I'll do transform.up, but actually I wanted to just rotate around the global up at the moment, vector three.up times rotation force. Zoom in again, so that should all be fine. I've saved my scripts just to go back. A little I think, drag-and-drop my rotate thing onto my cube and then just go and click on cube, go to scripts and rotation force, and just give it 100, and let's see what that brings us. Let's go around that reasonable pace, which is nice and of course this is Unity so we can change our numbers now. You notice if I go for 200, that's going quite fast. But not really much different but if I put 1,000 in, it's not really changing that much. Remember there is physics at work here, it's also reached the maximum velocity that it can do there. I put in a 100 to begin with but in the next I'm putting 50, we're not there yet, 20. So 20 is also not enough force to move that object. But don't forget of course, that because you're using physics and you have to play around these values, you can change the mass of your rigid body. It's not just changing the rotation force here, but you can change the mass of the rigid body. If I make that 0.5, I see now it's running again. So I just also wanted to show you that you can change how these things work and then drag, it's going a bit slow. Look at how that straining's move just by keeping going. But if I take drag down to 0, it's going a little bit faster.
Play video starting at :5:13 and follow transcript5:13
That's interesting. Putting some fibers made it stop. That's also just gone just over the threshold. There you go 0.6, seems it was enough to make it stop. So 0.5, it's just about able to keep it going. But I think about 0.5 since it started slowing down. Remember that

you can play around these values because it's physics. You can have different mass and in fact, if I made the object with different size, whoops. Let's see if it will. We had 0.5 and 20, we have a gamble to use out there, let's snap it to the surface here, is getting to that, I mean Physics can be fun. So 0.5 and 20, it's not moving at all because the cube is smaller. Now if I make this smaller this time, there you go. You see and that's because, and it's got dark because it's fallen and then in land not quite flat. The reason that is working like that, is because I've made it smaller. That means the object is actually weightless. The mass is the mass as compared to the size of the object. You can play around these things to get the feeding that you want. Usually, you're not going to change the size of the object, you're going to change the mass and the forces that are applied and don't forget, of course, finally, that the same things apply here as it does to add forces and then you've got acceleration and force and impulsive velocity change and remember that things, there's different use cases full of these, you could use impulse or more commonly velocity change if you wanted your character to jump up or to rotate around just a single knock to be turned in one direction, and then use acceleration force for constant movement. That's constantly turning, constantly moving forwards, and backwards and forwards. Have a little play around those as befits the needs of your game.

This time what we're going to do in here, we're going to work for a practical example of how we can actually use the conversion between different coordinate systems in unity in a useful way in the game. What I want to do here is actually I'm going to have a top-down twin stick shooter type thing where we use the keyboard to move a player. We use the mouse for the player to look at it. This involves conversion between screen point and the world point. Then there'll be some shortcomings of this method, which we'll go over in the next video where we look at how we can use right costs to solve those problems. But for now we're just converting between coordinate systems. I wonder what's the thing I was going to tell you. Here if I drag and drop my player. We know already that's a prefab. We know it's a prefab because it is blue. Up here in the corner of the plate is blue. Now, I told you how to make prefabs, I haven't told you how to reverse that. If you wanted to drag and drop in incidence of a prefab. Then you thought, actually I don't want that to be connected to the prefab cookie cutter down here in my project anymore. I want that to be a player object, but I don't want to be connected to the prefab and you don't want to lose anything to do with that. All you need to do is right-click on the player and to unpack prefab. Now that player is exactly the same as it always was.
Play video starting at :1:39 and follow transcript1:39
You can see it's still got its nose. It's still blue, etc. But you can see up here in the hierarchy that it has now become an unprefabbed instance. It just stands by itself. That can be really useful sometimes.

Play video starting at :1:58 and follow transcript1:58
Let's look at movement code. I'm going to create a script by clicking this C-shaped script and cause something interesting and inventive like player movements. Provided I can actually type in properly.
Play video starting at :2:22 and follow transcript2:22
There's a few things I need to set up at the top here. First of all, I shall stop the references [inaudible]. We're going to need a reference to our main camera. We're going to do that and Koscher reference just to make things a bit quick in the code and make the code a bit more easy to read. We need a rigid body because I'm going to be using continuous use forces here. Then I also need a value for Levenspiel. I'm going to serialize field. Then I'm going to make that floats movement speed. This is the basic things we could do.
Play video starting at :3:1 and follow transcript3:01
Anomalies that demonstrate way, but we are going to make the reference, so cam equals camera.main. Camera main just means the main camera, the main camera that spawned, and so we look for the object that is called main camera [inaudible]. Main camera and that's what camera.main is a shortcut to. We're just going to shorten that to cam and that become useful later on. Then we do a usual thing, that will be just get components [inaudible]. Great. We're going to keep updates. We are going to be using update, but we'll go into fixed update because we're using forces. We're using forces for the movement code, but we're using normal update loop for the looking code because it doesn't involve forces that involves summing different. In affix update, it's going to be similar to how we did things with our transform, translate a while ago. I'm going to get access raw because I have a personal preference for vertical movement. The forward and backwards movement is on using a vertical, what's the vertical axis? It's up and down keys or W and S, and same for the sideways movement. Not rotation because I wanted just move left and right. We're going to use the mouse to make our character rotate. Sideways means, it's the horizontal axis, left and right, A and D. Those two things that I'm going to do, vector 3, new position equals new vector 3.
Play video starting at :4:39 and follow transcript4:39
We're going to make a new position, and then what we're going to use. We're going to use a function called rigid body dot move position to add this value to wherever the current value of the player already is. Let's just carry on with it and I'll show you. The side fatty is the x of the character. Decide what way your code is going to move in space. But it makes sense to keep the x is sideway, left and right and the z is forward and backwards because that's what they are by default. Vector 3, so we going to use side, zero not going to change the vertical nature of our player and then zero and then forward. Already decided that, define that. Then times that by a movement speed, and then times that by time. Now, there's two ways to go around that you can do Time.deltaTime. You could do FixedDeltaTime. If you use Time.deltaTime actually within

a fixed update loop, it will fire with the same frequency as the fixed update loop. If you use FixedDeltaTime, what it means is you can control the frame rate of unit with some project settings, if you use FixedDeltaTime, it allows you to control that a bit more independently. For the time being, I'm just going to stick with delta time because it doesn't actually make a difference in this context. It's a bit of my new show there for. An RB, Rigid Body made [inaudible] MovePosition, transform.position, which I said, plus newPos. Now, of course you can pull this in one line, but why would you want to make your code readable later on? It's not as easy for me to teach you, but it's easier to read it later on. That should be absolutely fine. Okay, so let's go back here. We're going to drag and drop this on the player. It's going to play now, I got my camera in a certain alignment. Okay, there's a few things. I don't need that spawn point there. The reason I'm not moving it's not because I had areas because I didn't put any value into my movement speed. Let's do that. Now, if I up, down, left, right, it seems to be moving [inaudible] 20 scenes, fine, of course it resets to zero when we go back because I changed it whilst in play mode, so it will revert to wherever it was before play mode. Let's put it in this 20. My camera's looking this way, but I'm just going to make a note here that the camera's at different rotation. Now, with moving characters, so you can see at the moment, I've got local values, light pressed. It's showing the local. But what I really want in this case is I want the global handles to be shown so that I know which way is forward. Because then when I press forward, press up and down, I'm going to move forward and backwards in this blue direction on the global axis here. Why is it going to be global? Because I used a vector like this rather than basing off the transform, I'm basing it off of the vector 3, the common forward, the common side, [inaudible] movement side. My camera is actually rotated like this. If you're in your games and you're doing something like this, and your character's not moving up and down, it's moving in different directions, check your camera rotation first of all. I spin out there so it's easier to see in seeing. There's our movement. Now, I need to explain something, I'm going to jump over to good old-fashioned pen and paper to explain how I look code is going to work. Sometimes it's just easier for me to use a picture to explain these things rather than show something in the engine. At the moment, what we have is we have our overhead view like this. We've got a little play here, and we've got the moving in the z and the x-direction. Now, if we're going to use a looking script and use our mouse cursor on the screen, in order to look somewhere in the 3D space, we need to work out how to translate the 2D position of the mouse cursor into the 3D position of the world, and we're trying to find if you like this y value. If I draw this from the side, so if this is the top, this is the side.

Play video starting at :9:20 and follow transcript9:20

There's our little player again. Here's our camera. Our player is moving in the z. It's moving in and out towards us as it were using the x. We're trying to work out what this value is, the y, because our mouse cursor is working on the plane away from the

camera. If we just draw another side for you here in a bigger zoom. We're trying to work out what this y is. But our mouse cursor is actually operating on the plane about here, where the camera is or maybe there. Just take away that. The mouse cursor only operates here. We've got the our z and our y on this plane here. Then we need to translate it into where that coordinate is in the world because the player needs to be looking at this level here. If we didn't have the y, then our player is going to be doing all sorts of weird behavior, looking up here into the air which doesn't make any sense at all. For the time being, this is why we need to have our camera perfectly over the world in terms of the y-axis. If we had an angle, and this is for the z and the x, if you had the camera at an angle like this. Often, in a twin stick shooter, some sort, you'd expect to see the camera slightly off at an angle. I don't know why I switched to pen. You've got your player there. Now, the y is still the same of course, that's not changing, we just fix in between there and there. It's not that that's changing, it's the setting the axis that's going to start become a problem because the way the camera is angled, it's reading the coordinates along this plane now. Which means this is not always going to translate because this distance there, is going to be further away than this distance, and this is the problem that we're going to solve with great costs in the next video. For now, this is why our camera needs to be perfectly central over our game areas so that we can get them looking to work. I hope that little explanation helps show why we have to keep our camera absolutely deadlocked over world at the moment and how we're going to solve it that introduces a problem that we can't see solving a different route with the next video, but let's stick what we're doing right now, shall we? I've done the movement code already here in the fixed update and then the update loop, as I said before, we're going to start writing our looking code. Let's break this down into sections. First of all, we're going to make a temporary vector 2, because it's two-dimensional. I'm going to call that mousePos, and that's going to be input dot mouse position.

Play video starting at :12:28 and follow transcript12:28

That's the first thing. Let's zoom out a little bit there, so we get the mouse position. That's going to get the mouse position on the screen. The second thing we're going to do is, we're going to create a vector 3. We're going to call that the worldpoint.

Play video starting at :12:51 and follow transcript12:51

It's using the mouse position, but also adding in the distance that the play is from the main camera. Worldpoint is going to be new vector 3.

Play video starting at :13:5 and follow transcript13:05

That's going to be mousePos x, which we just made. It's going to be mousePos dot y. Although y in 3D is up and down in terms of the screen point, it's going to be the x and y of a 2D plane, then the z is going to be cam dot transform dot position dot y minus transform dot position dot y.

Play video starting at :13:42 and follow transcript13:42

Now, if we are all looking at the z axis, which I've just typed in those calculations for, why am I taking the value from the y of the camera and the y of the player. Well, let me show you. Let me go to the documentation. The documentation that we're going to feed that world point into, is this. We are looking at the camera dot screen to worldpoint function. Now it's worth taking tension here. The z position is the world units from the camera.

Play video starting at :14:25 and follow transcript14:25

In that instance then, and let me just go back to unity, it means that, where I'm I supposed to write? Unity is just thinking about the scripts I've just written. If I zoom out and find out where my cameras, there's my camera. If it's the world unit is away from the camera, there's actually along this z axis. Now I'm on my global handles here, so I'm looking at the global axis so do not get confused with local axes. The distance from the camera is along the y axis, even though I have to put it into my function as the z value. Hope that makes sense. If it doesn't rewind the video, look at documentation, read along, have a think about it. It's a little slow off, but that is just how that one works. That's the 3D point that we're aiming for, but we need to actually convert that into something that makes sense. So we're going to make a third vector 3. I'm going to call that target. That's going to be new vector 3, so target is going to be cam dot screen to worldpoint, that function that we were just looking at. I'm going to feed in worldpoint which I just defined before. Then finally, so let's just recap what we've done so far. We have taken the mouse position that's on the 2D screen, we have created a 3D coordinate of vector 3 using the x and y of the mouse position and the distance of the player from the camera. Transform dot position of course, this is attached to the players, it's going to be the player transform. Distance between the camera and the player, and then in the third one, we're creating a new vector 3 that does a conversion of the vector 3 that we feed it, which is this worldpoint. Lastly, we're going to do transform dot look at and then target. Now you could write all of this in just one great, big long line of course. But a, that'll be pretty difficult for you to get your heads around if this is the first time you've ever encountered this. B, that makes your code not very readable later on down the line, so just split it up. There's no harm done.

Play video starting at :16:55 and follow transcript16:55

Let's do that. Go back to our thing. We'd already attached our script. Let's run our game. You can see that our player, and no matter where we look, no matter where we move, our player is moving around. Now, there's little little bump there. Do you see that? I'm not pressing anything, but sometimes there's a little drift from my player. Let's go and fix that, but our look script is working completely fine. I'm going to add a couple more things here before I finish this video. First of all, let's just see exactly what is going on. That's game view. Let's just zoom in on our play here. If I go in there, you can see that he's bumping around. He might be bumping up a little bit, but it's also like drifting. I've got the input in there. How do we fix that? Let's go to our fixed update. Let's fix the drifting first.

If you want to make it short and snappy and he stops instantly, that's fine. If you do, if forward tests to zero and the other side tests to zero, then rb.velocity is going to be a new vector3(0,
Play video starting at :18:27 and follow transcript18:27
rb.velocity.y, 0). The reason I've put it like that rather than just using vector 3.0 is, you could give the ability for your character to jump later on. Maintain its y velocity, jumping up and down, but if I'm not pressing forward or back, it's not going to move forward or back. It means that if it bounces up, it's going to come back down. If I'd made this vector 3.0, our character, every time it bumped up, it would just stay flying in the air. It may drift down a little bit as we're moving around, but it just makes it actually fall back to the ground if there's no input. That's going to stop the drifting. Then last thing which can be useful, you can use a little rays to draw from the camera. You can draw debug rays if you want to see where something in the world actually is. The player is looking around the level at a point that we have calculated using this code behind me that we've just written, but what if you wanted to see whether actually was in real time? We can draw some little rays in debug mode. Let's make that happen. To do that, it's going to Debug.DrawRay.
Play video starting at :19:46 and follow transcript19:46
Then this is going to be cam.transform.position. Then it needs a start, and it needs a target. That way, it needs a directions. You can't just specify a coordinate, you have to specify direction. That's going to be target minus cam.transform.position. You can add a little color. I'm going to do color.magenta.
Play video starting at :20:20 and follow transcript20:20
Didn't do with a capital as the answer. That line here is going to draw a line from the camera to wherever we are looking in the color pink. Nice. Let's see how that works, shall we? Two things to test. First of all, we want to see where that line is so you can see that that line, if I put it into isometric,
Play video starting at :20:52 and follow transcript20:52
let's just put that flat like that, if we click on our thing here, we can see them perfectly flat plane. You can see where exactly that line is and it's exactly at where the y value of the player is. Indeed if we just pause that,. Look at the player. It's perfectly in line with the y of it, so it's useful. Let's play again. I meant to click the on pause button first. Then moving around. Where we bumped him before, it's not letting me stop drifting anymore like it was before. I let go of the controls, it goes to zero. I got the controls, it goes to zero. If I bring it around here and can we find that guy? If it bumps up. You can see it bumping up in my scene view, even so slightly. Then because I've made allowances for it, it will actually come back down to ground. There's a way of converting between two coordinate systems in order to do something useful within the gaming context. In the next video, we're going to sort out this problem so that we can actually have our camera

at perspective, a slight off angle perspective. I'm going to use raycasts to solve that problem.

So here we are, we're back in our level. Just to show you so you can see now that I've moved my camera offset and so If I press play now, you can see that look script is not working very well anymore because I've set my camera off angle. So this is the problem. In the last, video we put the cam directly over the head, directly perpendicular to the plane so that look code that we were using worked there, here, it's not going to work. We're going to look at some examples of how to use ray casts. What are ray casts? Ray casts are very simple, what they are, they just a line that's almost like a ray of light that goes from one point to another, and when it hits something, it hits a collider, it gathers information about where it's hit it and what it has hit. So often this will be, you might do ray cast from a camera to an object to get some information about it, you might do a ray cast from your player character or an enemy to survey the environment around it. You might use this to stop a player for moving in one direction if there is an object in the way, because if the ray cast is positive, then he won't move that direction. Often, say, in the first-person shooter game, if you fire against the wall and you see those bullet holes come up against the wall, that's not because actual bullets are fired out of your gun, that's a ray cast that's gone from the end of your gun into the wall, and it's just created little bullet mark graphic, bullet mark decal on the wall where that ray cast has hit the wall to give the effect of a bullet hitting the wall. So Ray cast have got a lot of different uses, in this context, what we're going to use a ray cost for, we're going to use a ray cost to direct the player where to look without the player bending up and down and looking a bit weird, like it does in our example, right now. Because a ray cast can be any length of light you want so whatsoever. We're going to do some extra finessing of the ray cast algorithm, to make it works with our player. I'll show you how this is done. So first of all, just do a basic ray cast just to show you, how to put ray cast and then, we'll start doing the extra maths needed to make the look script work with a camera offset like we have here so that our player can do the whole twin stick moving things, so camera, mouse and keyboard movement, but using ray casts. So first of all, let's go back to my code. I'm going to strip out all of the looking code here in a moment. I'll start playing the movement script, I'm just going to go back and [inaudible] think about it. I'm just going to dock that again behind there and that's just double-check that we're moving normally, yeah. No looking but our player is moving normally because he terms and he hits. So you can see, it's fallen over, so there you go. I spoke about constraints before. [inaudible] , rigid body, and then freeze rotation in the X and the Z plane. If we go back to seen the Y is where we want it to rotate around, but I don't want it to fall over in the X, I don't want to fall over forward, I don't want to fall over to the side, so it's the X in the set of plains that you constrain in, rotation sorry not plains. There you go, so I can spin around lots. But it's not actually making me fall over, okay? So yeah, that's nice.

Let's just show a simple ray cast, I'm going to attach it to the main camera and it will just identify objects within our game okay? So I'm going to make a new script, [inaudible]. I'm going to call this LED ray because it's going to identify the objects that we hit. Let's open that up. Visual Studio, okay? So we only need one thing, the beginning here, I'm just going to, make a reference to the main camera make to make our script to be shorter. So same as what we did before okay? So you're camera dot main. So then after that, what I'm going to do, I'm going to create a ray. So ray is just one of these beams of light that goes from one point to another, and I'm going to call this ray, my ray inventive. So I'm going to do cam screen point to ray input mouse position okay. So what this will do, it'll draw a ray of light from the main camera through the position where my mouse cursor is on screen through, into the world. So what this would do in terms of the example that we're building here, in this example, it'll go from the main camera through the plane where the mouse is acting upon, through that plane and then hit the world. So in essence, what it gives us, it means when we click on something on the screen, it will draw a ray of light from our camera through to the object that we're clicking on the screen and we can get some information about that. So I've got my ray. If, and then this is to click on something, get button down. I'm just going to keep the fire one.
Play video starting at :5:56 and follow transcript5:56
If we have that. Then you use a physics Raycast, usually in an if statement because it will be turn true. If the statement turns true, then do thing, so If. In the way we access is Physics.Raycast.
Play video starting at :6:12 and follow transcript6:12
You can see here, if we just zoom in a little bit, you can see there's a lot of overloads going on there. Just a point here, the documentation doesn't always show you each individual overload, that's there, so it is worth consulting documentation. It'll tell you what all the fields are and what they all do, but it won't tell you necessarily all the combinations that are available. It is worth separately, looking at the documentation and just clicking through all your overloads to see what's going on there. In this instance, I going to do Raycast and I can do myRay, which is the ray that we've used. Then we are going to do out RaycastHit hit. Now, what that does is, it will use myRay. This be here, the out RaycastHit. The out means, where the Raycast hits. We're going to create a RaycastHit item, it's going to collect information about whatever that hits, and then that object is going to be called hit. Now, I can address that RaycastHit whatever's in that data structure. Here, I can do Debug,Log. This is the the space and concatenate them to hit the name of the RaycastHit object, transform.name. I've used tags before [inaudible] username for change plus their space object.
Play video starting at :7:49 and follow transcript7:49
Debug.Log, that's going to get the name of the object, it's going to say, object case, it just printing that to our console, so we'll save that. Go back to Unity. Unity is going to have a little think and drag-and-drop the ID ray on the main camera. Then if we press

"Play" in our game screen, we click on the wall. This is the wall object, this is the floor object. Then they click on that, this is the player object. Let's just prove to ourselves, shall we? It is taking the name of the tag, let's call this amazingFloor, and let's call this awesomeWall because the graphics are so high-def in this example.
Play video starting at :8:44 and follow transcript8:44
Let's call this greatestPlayer. It's not common case, I prefer to do that. GreatestPlayer. This one again,
Play video starting at :8:57 and follow transcript8:57
this is the awesomeWall object. Click on the floor, this is the amazingFloor object. Click on, they get the greatestPlayer object. I'm taking the name and not the tag. All very well and good. It's a simple promise, the Raycast one last time. You draw a ray or it's like a ray, a straight-line, a ray of light from one place to another and whatever it hits, you can get information about it. Let's look at how this could be useful. For example. I'm going to create a new script.
Play video starting at :9:35 and follow transcript9:35
See, I'm going to call this PlayerLook.
Play video starting at :9:41 and follow transcript9:41
We are here, I'm just going to go back to the main camera. We don't necessarily need that script, so you could remove it. I'm just going to disable it. Let's start working on our PlayerLook script.
Play video starting at :10: and follow transcript10:00
Let's just do a practical example. I've just disobeyed my own rules of start with capital now to PlayerLook, if I change it there, it's two or three loads. I have to bring it back. Remember, if you've changed the title of the script, change the title of the class as well. A few references we need to make here, so Rigidbody, rb, I'm going to need the camera, it's called a cam. Then we're going to define our ray, first of all. This is just to make the code look a bit neater so it's easy to follow along if this is the first time you've done it. Then we also need a Vector3 hitpoint.
Play video starting at :10:56 and follow transcript10:56
Vector three hit point. In the start, do the usual things.
Play video starting at :11:4 and follow transcript11:04
Get components, rigid body. That's that. Then cam is equal Camera.main. Usual references making the main camera, and the rigid body attached to this game object. In update, the first thing we're going to do, is we're going to create a ray, pointing from the camera, through the position where the mouse is on screen. Mouse ray equals Cam.dot screen point to ray. That's going to be input.mouse position. That's the first thing. Then the second thing we do, we're going to just define this first of all. I'm going to make a ray cast hit, to collect information about wherever that ray cast hits. We're going do a ray cast hit item. I'm going to call that hit info. These are just temporary variables. I didn't need to pre make them. The mouse ray are predefined because that works to become

useful. Then we're going to do ray cast with this ray. I will collect the information into our hit info.
Play video starting at :12:23 and follow transcript12:23
Then we also we are going to extract the vector information from that hit info object. Let's do that. If physics.ray cast and that's going to be using the mouse ray. Mouse ray out, hit info. Ray cast hit that we already defined. Then also we are going to put in the distance. Now, you don't have to put in the length of the ray cast if you don't want to. The main thing you don't want to do is, you don't want to make it too short. Because otherwise, if you're trying to hit an object like here, from here, and you make the ray cast only that long. Then you'll see, you are not going to hit that. If you make it too far, that's absolutely fine. If you don't put distance in, you'll make that ray cast infinite, which can be fine, until you've got lots of ray cast going on. If you've got lots of ray casts going on, like in first person perspective game. We're using ray casts all the time, from the main camera, the player camera that starts to affect your performance. It can be a good habit to get into just a distancing of some sort, because even the distance that's really long is better than infinity.
Play video starting at :13:40 and follow transcript13:40
If it hits a thing, and hit point is going to equal. Hit info. [inaudible].point. Hit info.point. Here you can see if I zoom in, make a bit bigger, the impact point of both space with a ray hits the collider. It's getting the 3D position. Not if the object is hit, but of exactly where that ray has hit. That's useful. That's our hit point. Now, the place for the player to look at, is going to comprise of the x and the z, of our hit point, wherever that ray cast is hit, but it's going to need to use the y of the player transform, otherwise, it's going to be looking down into the ground and we don't want our player actually looking at the floor. Or looking up in the air, if it hits a high point. We want it to be looking on the level, so you need to use the y of the play objects. Let's do this. I'm going to just create another temporary vector three, called look target. Again, splitting things out to make it easier to read. Using good naming, so it's easier to read back. Make things clear. The vector three is going to hit point,
Play video starting at :15:4 and follow transcript15:04
transform.position.y, and then hit point.z. Remembering
Play video starting at :15:17 and follow transcript15:17
that hit point is the vector information extracted from our ray cast hit info object. Then the player is going to look at that object. It's going to be transform.look at. It's going to be look target. There you go. Then same way as we did before, just to help us visualize in case there's an issue going on. Debug.draw ray, which can be useful. Cam.transform.position. It's going to start there. Then the direction is going to be looked target minus cam. transform.position. Then we can do it in green today, because really doesn't matter.
Play video starting at :16:12 and follow transcript16:12

That's all safe, let's go back. We need to make sure how we'll think about it. IK, is there, so let's drag and drop the player look onto our greatest player object and let's see how that runs. Look, now, our play is not looking anywhere, so it's a little stuttering around the wall. The way you can improve that, you could use a layer mask, which I'm not going to go into now. Or you can use a layer mask to make it skip the wall basically and put the ball in a different collision layer. But you can see that that play is not now bending over, and it's actually behaving as we might expect by moving around in that manner. It's not bending over like we had in the previous one, so let's carry on there. I don't see my debug gray area. Okay. Here we are. If I go isometric, that's just zoom in a little bit and here you can see from one perspective, again, if I pause that just to show that it's at the same y position as the player. Okay, so this is why it's not bending over. Essentially what we have done is we have cast a rate point from the camera to follow through our mouse until it hits an object in the wall, and then we've extracted the information exactly where that ray has hit an object in the wall, be it the floor or the wall, and then we have raised that up to the level of the player, and then we're looking at the x and the y level of the player, so that we get a nice look script and our camera could be placed wherever angle it doesn't need to be directly over because we're using those Ray costs,

Play video starting at :18:7 and follow transcript18:07

so yeah, that should be all that we need to do. I'm just going to check. Also, one other thing that you could do if you want to do something a little bit funny as well and this [inaudible] will say, what, if you wanted to use, just create objects. Well, let's do that. The other thing you can do is use some slight different code as well, so still an update. If input dot get mouse button down. Now this is a different way I just thought I'd put this in as a different way to access the mouse, so Fire 1 is configured at the moment as left control by default or mouse button zero. Zero is your primary mouse button, and you got 1 and 2 for the right and the middle respectively. It depends what's going to work better for your game. You're often best off using something like the axes that are defined in the input manager. Because then that gives the potential for the player to remap those controls to suit their needs. If you use something like get a mouse button down, it's the same as using a key code. It hard wires that into the engine. But it's just another way to access that's again my mouse button down zero instantiate. Alright, so we need to define the end, so I'm going to make it just public game object bullet, so create a variable for my game object. Come back to my instantiate, now can use bullet, at hit point, quaternion dot identity, extra bracket. The thing there is, I'm just going to create a bullet at the hip point. I'm not bothered about it being the same level as the player, and we'll see why and then quaternion identity is just a default rotation. Just a simple thing there. Wait for you to think about it. Go talk rates player, find our script. Doesn't have a script. I've got my forces bullet, this is the bullet with a rigid body on it that we used for forces movement. But now, anywhere, I click if I'm exact, I can just balance them along

the wall on the player's head. If I like, I can just put on the wall or go directly on the floor. Some they were dropping through because the plane is not convex and so some of them are finding that they're clipping through that layer. If you wanted to solve that and the way you do that is, let's go back to our, wall, we can just click the plane here. Actually amazing floor. If you click convex. The difference this makes is convex means it has to have some solid mass. You can see there's actually a thickness of collider there, which can make it look slightly funny, but it doesn't mean those bullets won't click through, so you can see the play jumped up a little bit there. But it does mean that none of those bullets are going to clip through there. Already start to bounce, and you could have a bit of fun just kicking him around. You could quite easily see how you could click just to make it football. Click him around.
Play video starting at :21:23 and follow transcript21:23
Haven't bounced off the play heads. You get the idea. There's a way in which re-casting,
Play video starting at :21:32 and follow transcript21:32
a nice simple method you can use to solve a variety of problems, so you can use it to detect when you're close to another object and then do this, you can use it to detect where an object should go, and you don't have to use all the data. You can use path of the Data as we've used here. I hope that helps you understand recasts through an example where it's actually really useful.

You've learned about physics as a way of adding movement and dynamism to your 3D graphics world. Physics is fantastic, but it can't do everything. While, in theory, everything in the world is simulatable by physics, there are some things that are just way too complicated. If you think about human motion, the physics of that has to go right down to the motion of muscles and even individual cells, and how our brains interact with their bodies. Trying to simulate how we move through physics is immensely complicated, and so it's going to be very difficult to create expressive, realistic human movements through physics. Even movements isn't just the only thing. If we really want to create expressive stuff, we need something a little bit different. In this topic we're going to look at a technique that goes back right to the beginning of animation, way before computers were used, to the early days of screen animation by the likes of Disney and other pioneers. It's a technique called key frame animation that's been readily adopted by computer animation techniques. It allows us to bring some of the artistic expressiveness of those early animations to both field based 3D animation and also interactive animation like in games. Let's start this topic by having a look at key frame animation.

Keyframe Animation
Transcript missing

One of the advantages of using computer animation is that we can use computers to generate the in-between frames for keyframe animation, making it less labor-intensive for animators. The process of generating the in-between frames is called interpolation. In this video, I will explain how this is done and the maths behind it.
Play video starting at ::33 and follow transcript0:33
Let's start with a simple example. Assume we have to animate a point which is set to be at position 0, 0 at frame 0, and then it moves to position 1, 1 at frame 10, and then finally 2, 0 at frame 20. Now we have those key frames. We want a computer to generate the in-between frames, frame 1-9 and then frame 11-19. So that as a result, we will see something like this. The most straightforward way is to do it linearly, as in the point will move with a constant speed between the key frames.
Play video starting at :1:18 and follow transcript1:18
But how do we calculate the positions for each in-between frame? If at frame 0 we said x value to be 10 and say frame 4 I want to x value to be 30, what will be the x value at frame 1, 2, and 3, assuming the point is traveling with a constant speed?
Play video starting at :1:42 and follow transcript1:42
If we're going to move from 10-30 within four frames, we need to move forward five each frame. At frame 1 we will move to 15 and frame 2 to 20, frame 3 to 25, and finally at frame 4, we arrive at 30.
Play video starting at :2:15 and follow transcript2:15
Now you have a basic understanding of how to generate the in-between values. Let's take a look at accretion of it and also how it is coded in most 3D graphics engines. Say that I have two keyframes, keyframes 0 and keyframe 1. The actual frames of these two keyframes could be say 3, and 8, like example which has had. But here we're representing them as t_k minus 1 and t_k. At t_k minus 1 the position is given as Pt_k minus 1, at t_k the position is Pt_k. Now we have to generate the in-between frames. This means that given any frame t, that is between these two key frames. It could be frame 4, 5, 6 or 7. We should be able to return the value of the position Pt.
Play video starting at :3:17 and follow transcript3:17
How do we calculate that? As it is linear interpolation we're doing, we know that the relationship between time and space is linear. Or the travel speed from t, t_k minus1 to P t_k is the same as from Pt_k minus 1 to Pt. We'll have the relationship where Pt_k minus Pt_k minus 1 over t_k minus t_k minus 1 is same as Pt minus Pt_k minus 1 over t minus t_k minus 1. We can rearrange this equation to get expression of P t, which is t minus t_k minus 1 over t_k minus t_k minus 1 times Pt_k plus 1 minus t minus t_k minus 1 over t_k minus t_k minus 1 times Pt_k minus 1. This is an equation behind a linear interpolation which allows you to calculate the position of any given frame between the two keyframes. But it doesn't look like something easy to remember, does it? Well, the good news is that there is a simpler version of this equation, but we'll have to make some adjustments. Basically, if we set t_k minus 1, the first keyframe to zero

and t_k or the target keyframe to one. We know that t, which represent the in-between frame, will be something between zero and one. This is called normalization, which is the process of mapping or scaling the parameters of a problem into the zero and one range, making it mathematically a lot easier to represent. If we replace t_k minus 1 with 0 and t_k with 1, Now we can rewrite the equation into a much simpler form. Pt equals t times P_1 plus 1 minus t times P_0. Note that t is between zero and one.

In computer graphics we used the lerp function for linear interpolation. This is a pseudo code for a lerp function it takes two vectors P0 and P1 which gives the start and end positions and a float number t which should be a value between 0 and 1. This function returns the interrelation between the two inputs P0 and P1 for the given parameter t.
Play video starting at ::38 and follow transcript0:38
Let's look at an example, so I have two key frames at frame 0, the position is 0,0 and at frame 10 position is 1,1. How do we use lerp function to calculate the position vector for frame 5? And what value does the function return? First of all, before calling the lerp function, we need to normalize our parameters. You can use the built-in normalization function to do it. If we map frame 0-0 then from 10 to 1 then frame five will be 0.5. And now we can use the three variables to call lerp function P0, P1 and the normalized t, which is 0.5. This function then returns the vector 0.5, 0.5 and that's where the position will be at frame 5.
Play video starting at :1:37 and follow transcript1:37
So this is linear interpolation which generates a constant speed between two key frames. But as you can see here, when you have three key frames, the result of the linear interpolation could look a bit jerky. This is because there is a sudden change of direction of travel.
Play video starting at :1:59 and follow transcript1:59
So very often instead of linear interpolation, we use spline interpolation which produces a smoother curve with no sudden changes of direction of travel. Or in other words, the spline interpolation produces a continuous velocity. We're not going to look at the math behind spline interpolations, but there is one final concept I need to introduce in this video, velocity and tangent. If you remember, velocity is different from speed in that it is a vector which describes both the speed and direction of travel. The direction of the velocity is a specific point on a curve can be visualized using the tangent at that point. The tangent here is a straight line which touches the curve at only one point. So ideally in animation interpolation to make the movements look more natural, we like to have some continuity also on velocity and spline interpolation takes care of that. And the tangents or velocity of the animation could be automatically generated, but you can also edit them to have more control over your animation as well.

Here is our little virtual reality scene that we've been creating in throughout the specialization, and it's time to add a little bit of movement to it. And in particular, we've got this kind of creepy close circuit TV camera that's kind of watching our every move. Let's have a look at that in the actual via there it is.
Play video starting at ::36 and follow transcript0:36
Now, what I want to do is make it move, give it a bit of life. So the first thing we need to do is go to assets and create and animation, and we're going to call it camera scan maybe. So this is our animation. We can open it's already actually open. But if we go to the window and open the animation window, we can see this view which will allow us in a minute to start animating I'm not going to do anything with it yet. Because before I want to do anything with it, we need to add it to the CCTV camera itself, the close circuit camera itself. And for that, we need another asset, which is called, an animator controller.
Play video starting at :1:42 and follow transcript1:42
So if the animation provides one clip of animation the animator controller allows you to combine different animations onto a single object. And what I'm going to do is to select the CCTV and add animator to that object. And this needs a controller, which was the controller that we just created camera animator. And I'm going to open the windows animator and this is what it looks like. So these are basically there are three what are called states. And the states mostly correspond to an animation. But these are actually all special states entry and exit. So entries what happens when the program begins, exit means, if you reach that exit state, you end. So basically what we want to do is go to entry, and then we want to move straight away into our animation because we want to do that all the time. So let's just grab our animation camera scan and put it in here, and it will automatically make it orange, which means the default states, which means that there's a little our from entry to camera scan, which means when we start, it will start off camera scan. So that's great. You got something but we don't actually have any animation you have to. It's not actually doing anything. And this is where the animator window comes in. We're now going to use the animation window to start animating this object. Now, I could do add property, which will define what bits of the object we're actually going to animate? What aspects? And in this case, we're mostly going to be animating transforms. But I'm actually what I'm going to do is hit record. And then anything I do to the object will just be recorded as part of the animation. So I'm going to animate the transform of the CCTV, the main object and I'm basically just going to kind of my interpretation. So I'm going to grab it. And if you see if I move it, it will set a key frame at frame zero. It's kind of what I want. I wanted to start their frame zero, but I don't actually kind of want to start in the same position. So you've seen I've kind of moved in a tiny bit, but not actually really moved it. I've just done a little movement to set a key frame. Now what I'm going to do is I'm going to slide that key frame and copy it. I'm just going to do control C and move to one second and copy that in. Why have I done that?

Well, I actually want the same frame to exist at both zero seconds in one second. So it starts and ends in the same place so we can loop it so we can be sure that the end of the animation is identical to the start of animation. So whenever I want to loop and animation, I take the first frame the frame at zero seconds and copy that straight to the last frame. That's the first thing I do and then I create the stuff in the middle. So in the middle, at about 30 seconds, I wanted to turn this way. And there we have a basic animation. Oops, that's not what I want. Let's go back here, delete that key frame again. Let's do that shows the pitfalls of animation. Let's try again. Slightly small one.
Play video starting at :5:18 and follow transcript5:18
And there we go. It's moving quite sensitive, actually. You know what? That's a little bit quick for me. I would like this to be a little bit slower. So what I'm going to do is I'm just going to move this final frame key for two seconds and from 30 to 1 second and we go, that's a bit nicer. And what we can see here, the great thing is we just have three key frames, two of them which are identical. And we're already creating quite a nice movement. So if we wanted, sort of a more complex movement office. We need more keep frames than that. But actually, it's quite nice as it is. So I'm going to turn, record off and hit play. And we can see in VR that we've got this animating object. Let's look at that again quickly. Now it's not quite what I want because it's just done one movement. It's done the movement once. What's the problem with that? Well, it's not a big problem if I go to the animator, go to camera scan. I can just double click on this movement, and all I need to do is loop time and what loop time means the animation loops. It goes back to the beginning and starts again once once it's ended. And that's really all you need to do to create animation obviously, will create a lot more key frames. If you'd like something more complex and you can also, I've just animated one particular property. The rotation you could add animate lots of other stuff. For example, I could have animated some of the sub objects independently of the main object, but I think it's a good starting point. Start with just a simple animation, like a rotation or maybe a position, and that can add a lot of interest and movement to your scene.

Before we start talking about events and listeners and how to implement one, it's really a good idea to understand why it's even necessary and what's the point of this pattern. Let's take a common example that you might find in a game, such as the death of a player. We've got our player here and let's say in the code, there is a on Player Death function and that fires when the player dies. Now, what things might you want to happen? Well, in this play a death function, we may want a sound to play. Sound to play so that's going to come from one script. It might be on the player object, it might be in an audio object elsewhere. We'd probably like the enemies to stop moving. That means to stop, reset. Maybe they'll have a little dance over your grave because you're dead. You may lose a life. If you've got a certain number of lives. What else could we have? The game state will change. The game state change. The game state might change

because you've died, therefore, the game needs to pause, maybe a new animation needs to play, or sound needs to play and then you need to reset the game or end game or something along those lines, but you've changed the state of the game, you're not in game-play, you're in between death and resetting. These could all be on different objects.
Play video starting at :2:11 and follow transcript2:11
This game state change is probably in an in a state controller of some sort, maybe. There's one central object that's controlling the state and then this might be in a audio object. Maybe you've got one object that's managing the war music and our sound together. These are over individual or they could be a collection. Could be a collection of enemies. Lose a life. This again might be in our state controller.
Play video starting at :2:57 and follow transcript2:57
If we were going to do this, the way that we've been talking about things so far, the way this would happen is we'd have to make a reference to the state controller. We'd have to make a reference to the audio object. We'd have to make a reference to the enemy or the collection of enemies. Another ref there and maybe we need to do another ref to state controller but we certainly need to make a reference to maybe not the object, but the component that controls about the lives. You might have player stats as one script which is different from the game state stats. We'd have to create references to all of these in our Start object and then in our on Player Death, that we'd go through and execute all of those. The functions that are relevant within it. The problem of this is, is what if we don't want to lose a life anymore? What if we want to change that? Well, we're going to have to rub out, not only this, but we're going to have to rub about the reference as well. We have to scrub out the reference because it's not needed anymore. What if we've got a bunch of enemies here but we want to add extra enemies in there? We're going to have to make extra references over here to add in the on Player Death, etc. If these enemies change, then that reference breaks and you start getting quite a lot of spaghetti code because these two things are very tightly coupled. The player has to know about all these objects in order for them to execute. This is the standard model that you do when you first start with Unity. Let's talk about a event and listener program. Let's get short of that.
Play video starting at :4:44 and follow transcript4:44
An event and listener model is what's known in programming as an observer or subscription subscription publisher model. Let's look at it in this example it went like this. In the middle we have an event, let's call it on Player Death
Play video starting at :5:4 and follow transcript5:04
and that is published by the player.
Play video starting at :5:12 and follow transcript5:12
This is our publisher, the thing that's making the event and this is our event.
Play video starting at :5:21 and follow transcript5:21

Now, all of our other objects on the site are over here. These are all of our subscribers. Our subscribers are going to be our state controller and that can be with the lives lost function and that could be with the game state change.
Play video starting at :5:58 and follow transcript5:58
I got one there, we had our audio object and then we had our enemy, maybe collection of enemies. You've got lots of enemies on screen. What happens is that each of these objects would subscribe to this event like this. The lives, actually, not the objects source of the lives function would subscribe and the game state change would subscribe and then the audio object with its things would subscribe and the enemy collection would subscribe. When this event fires, when the player makes this event happen, then these will all get notice of it. You can imagine like these are subscribing to an email newsletter. The player puts out email newsletter on Player Death and then these people receive it and then they will do something in response to that. Now, the thing here is that this player never needs to know about any of these things here at all. It doesn't care about who's listening for the on Player Event. It needs to have something subscribed to the on Player Event. This needs to not be empty, so it needs to be a listener somewhere, but it doesn't really care about who's over here on this side. These things are completely separate and these things don't necessarily need to know about the player per se. They just really need to know about this event here. This means that our code is a lot more decoupled. It means that things become a lot more modular. If say we don't want to play an audio sound anymore on the Player Death, we want to remove that, we want to do something different. We just remove that on subscription. Nothing's affected. No code gets broken, no reference needs to be removed. No reference gets broken. If this enemy moves to another place and we rename it or record it something else. As long as it's still subscribed to this event, it doesn't really matter. This won't cause a problem. This broken link won't cause a problem, unlike the references that we we're talking about earlier. I hope that makes it a little bit clearer how this works and how this can be really useful in structuring your code and making it a lot more flexible, a lot more modular, a lot less tightly coupled, and therefore a lot less prone to bugs.

So here we are back in unity. I've skipped a few steps. I just built the things. I've got floor. We've got a player as a capsule here. I've got enemy as a cube over here. And then I've also, if we go and look at the hierarchy. I've got one of the floor, but I've also got a sound manager and I've got a game manager as well. So this is just to give the variety of options here. I'm going to do example, based off what I explained in the previous video. So let's look at our game manager. So on our game manager, we're going to have an event manager script. So I've let's do so I got scripts folder here already. Let's create, I'll see shot script. And it's called that event manager because it kind of makes sense to have all your events in one place. Okay, so reload that. Okay, so I've got a script here. So at the beginning, we don't need anything in the start method

and in fact I am going to delete that at the moment because we don't need it. So at the top we're going to do, so I'll show you two different ways of doing this. We'll start with one, it's going to be public delegate void. And I'm going to make a event called player death groups like that, and then after I'm going to do groups public. I'll lead the static for time. Event player death. So like that on player death, okay. So an event in unity. Event in unity is actually a kind of delegate. It's like a multicast delegate in C sharp. So that's essentially it's a rapper for it. The reason you would use events rather than delegates is that you would use, is that events have a little more protection. Delegates are more prone to being overwritten and have an execution order messed up. Whereas events have got an extra layer of abstraction and they're less prone to being fiddled with, essentially. So events are a good choice for this kind of model that we're doing here. So let's, that's that one set up. Yeah, okay, and I'm just looking right. So what we want to do here is in the update method, we're going to do. We're just going to do a simple if input, get button down, fire 1. It works for explanation purposes. So here's an old school way of doing it. Which you may see in lots of places on, and it actually explicates what's happening with the next bit code that I'm going to do. So if player death is not null then on player death, okay. So what's happening here is that it's checking to see that play death exists or sorry, that play death has something subscribing to it because it is a delegate. And if it has something subscribing to it, then it will fire. If you try and fire an event that has no subscribers, then it will produce an error and you'll crash your, you might crash your program, okay? But will produce an error. So you need to check that exists. It has subscribers because it definitely exists. It has subscribers before it fires Now. Another way, to do this, if I do control E C. Which is a corded keystroke visual studio. Then the other way you can write that, that's simpler. But just a little more obvious caters I could do on player death. I could put a question mark there and then do invoke, okay. And that achieves exactly the same as what this code around here does. Okay, so is on play deaf there in that case invoke it. All right, so there's a simpler way to do it. So, we set up our played deaf event. It's published. Let's go back to unity and before we forget. So, if you need to think about it, drag and drop event manager onto the game manager. Okay, Just double check it's there. Yes, it's their cool. So that's our first scripts. Let's start making some stuff to fire when the player dies. So what kind of things might happen? Well, kind of things might be you could do the player loses a life. Okay, well, let's make a play life script. So it's going to just play a life, okay? Sorry, shuffling bits, papers to my life. Yeah, to do. Okay, so at the top here, doing a few things I'm going to do public in life. Rather I'll tell you what, serialized field tent life, okay. And then here, we need to make so no I notice that today I'm going to do Life equals 3 at the beginning. And then I'm going to write a I'm going to delete updates, I'm not using it here, I'm going to write just a little function here, void, lose life. Okay, so that's going to be life, my equals 1 and then debug dot log just to have something going on. Got yeah, say we've got X number of lives left. Okay so we've got play life. It's tracking the feet, it's

got function to lose a life when the time comes. Now, in order to subscribe this to the on play deaf event. We need to find that event manager and we need to make it easy to get in touch with now. There are a couple of ways to go about this and just looking at which we're going to do first, yeah. Okay, so let's let's do it this way and I'll show you what might you can make a choice after this. So what you want to do you want to do on. So if you type on the navel will fill out the rest of the signature for you. What you want to do, you want to find object of type event manager because that's what our script is, okay. And then on player death. And you're going to notice that are not actually putting sorry, wrong thing. I'm doing loose life, okay. So let's just look at what's going on here. So I'm finding objects of type event manager that's fairly obvious. [COUGH] So you're looking for that component. I want to find wherever that is in the scene. We're presuming that there's only one of them. This makes the assumption that is only going to be one of them. So you've got to be strict with your code structure here. Don't player death. Notice that I haven't put the functions there after on player deaf because it's a it's a delegate. I'm adding lose life to delegate because it's a function of variables. It's like function of functions, okay. Container for functions, so we're treating on player death and adding to it as though it's a variable. Okay so you don't need any things there. So I've just subscribed in this line here. I've subscribed my event lose life too on play death. Now the other thing, there's two things I need to show you here. The next thing is if you subscribe it. You going to make sure that you unsubscribe and if you're going to. Unsubscribe you need to put it on disabled and then what we can do is copy paste, just take that away. So just be aware I'm adding in the enable and I'm taking away in the on disable, okay? So if you subscribe an event, you've got to find a way to unsubscribe it. You subscribe and unsubscribe in the unable on disk on disabled events. If your interest in the execution sequence of that actually on the on enable happens in between awake and start, okay. Hopefully don't come into those kind of execution order issues. But it's a thing yeah, that's valuable to see on the United website if you ever need to refer to it. Simpler way of doing this. Let's go back to my event manager class here. Of course, if you had something like an event manager. And you had something like an event manager and it's going to be one place we're going to handle all your events or of a certain kind of event maybe then you only want to have one there. So if you only got one there, it's reasonable that you could say that this event is a static event. So you're going to make that a static event just add static here. And it's just my car code a little bit neater. They go, that should come up as never. But if we just tidy this up now, we can just go event manager dot com player death because the event manager static. There's only one of them, okay. And then the safe, unsubscribe. Now I realize here that if you're using it like a singleton. Then you should put a load of wrapper code around that to ensure that event manages the only instance that exists in your game. But I'm not going into that there. But just understand that this is a singleton way of thinking. And you would need some extra wrapping going on there in your event manager script to ensure

it's the only instance going on there. But I just want to focus on events and distance for now. [COUGH] So, there you go. So we've got play life. It's going to lose life that has subscribed. Let's go back to Unity. And this add play live. Where did you keep the player lives? You probably keep the player lives in the game manager, actually. So it's a separate script. It's same object. It's a different thing that's going to set 23 at the beginning anyway. So you don't need that. Okay, so I got play life. And then the next thing we could do is maybe we could have the game state. I'll tell you what the player life play life on the player. I'm just going to take it off the game manager just to spread things around a bit because my next script is going to definitely belong on the event on the game controller. So let's look at the game state, may be that the game state thing go on. Let's open that up. Yeah, thank you, great. So here we don't need to start and we don't need update. So I'm going to delete break for them. I'm going to write my function 1st and then I'm going to subscribe and unsubscribe it. So avoid restart game. So let's imagine, when the player dies, you need to do something to restart the game. Okay, I'm not going to actually do anything, I'm just going to say debug dot log. Yeah, restarting. Okay, it's just a proof of function obviously put like stuff in here. Okay so let's go for on enable. Get just fit out for us. The event manager dot on player death plus. So subscribe Reese. I thought it would take the suggestion there. Thank you. Alright, restart game, copy that on interesting but you no doubt had that, okay. Right, so I subscribed and unsubscribed it, okay. Encode and yeah I think that will do for I tell you we'll do one last thing. Just do something other than a little debug of code. Let's do a player sound script. So I'm going to gloss over. Let's do yeah so player sounds. Before I forget I was going to drag and drop this onto the sales manager. Maybe you want one place for your sound to go through. You'll do some sort of centralized processing on it or you just want to know where to go to process your sounds. So actually let me just add on to ask if I've got a sales manager. Are they? I did it already. So all my sound manager just to let you know, I've already remove that [LAUGH] Because I did it twice already added it before having that extra copy. I've got an audio source, so an audio source just plays audio. And I'm not going to that a lot at the moment. I just want to have something different to do. I've got an audio source there and then I have dragged this sample over here onto the clip there, okay? So having done that, we can now go into players sounds. And again up here I'm going to have to make a reference to the audio source. So we've got audio source. I'm just going to call that sounds, then start, we can do sounds equals, get component audio source. Okay, so that's that done. I don't need update. I am going to do a void player death sound because that sounds like a logical name. Doesn't it play this sound? And I'll just do sounds dot play now sounds play. We'll just play the sample that I've assigned already. It does that. That's all fine. And then obviously I need to enable and then I'm going to do event manager dot play death. Sorry, dot on player death. Plus equals and then play death sound. Okay, let's copy that on disabled. Unsubscribe, subscribed, cool. Okay, and then last thing, because I did put

an enemy in their. An enemy might just want to dance on your grave, really? If it's, if you've been giving them issues, et cetera. And again, this is just to show you can spread this around. So, enemy actions maybe.
Play video starting at :15:34 and follow transcript15:34
Okay, and then let's double click that. Don't need to start an update for this. So I'm going to write my function first. That's quite avoid enemy triumph. Debug dot log to indicate Hooray. The hero is dead.
Play video starting at :16:5 and follow transcript16:05
Yeah, it's not just players, they get to have victory. It's enemies that have victory to all too often. So on enable and an event manager. You get the idea now on play death plus equals enemy triumph. Hoops not in the actions, that doesn't make sense and it will give you an error, hoops. Okay, copy hoops.
Play video starting at :16:38 and follow transcript16:38
Somehow the P got lost there.
Play video starting at :16:41 and follow transcript16:41
Can't replace that line unsubscribed, okay. So hopefully let's just check a few things here before we click play. What we do need to do is add the enemy actions onto the enemy. So my player, I've actually, I've got my player life script on there. And that should be printing to the debug every time. We've got the enemy, which has got does little how you're dead. Sales manager plays the sound. I'm just going to double check by default when you have an audio source. If I just zoom in here. It'll have play on the wake that will be ticked by default when you first had an audio source. And sometimes you press play and gets cacophony of sound as every audio source and your game plays that you just make sure turn it off.
Play video starting at :17:29 and follow transcript17:29
And then the game manager has got the event manager script in it. So that's all triggered to come off with my fire button. And if I press fire >> [SOUND] >> There you go. Parade here is dead. I've got two lives left, okay. And so where's our game start? There's one thing missing.
Play video starting at :17:58 and follow transcript17:58
I restart game. Dura, restarting game. Did that come up? I don't remember. I bet you. Yeah, there you go. So something's not coming up and you expect it to. Live bug fixing. Because I hadn't touched the script, so it doesn't exist, basically. Okay, silence player dies >> [SOUND] >> There you go. I've got two lives left. Got the other two debug messages, which I was expecting. Everything is good now >> [SOUND] >> Got one life left >> [SOUND] >> And then >> [SOUND] >> We can go into minus lives if we so desire. Okay, so that is how you can quite easily subscribe and unsubscribe events using code. And in the next video, I'm going to show you how you can do a similar thing, but using unity events, which is slightly different.

We're back here and what I've just done in between these two videos, is I've stripped out all the subscribe and unsubscribe code. Where I had on enable and disable, then the actions and the PlayerSounds, the GameState, and the PlayerLife. I've actually stripped all of that code out to show you what UnityEvents do and how they work. Another thing I'm just going to do before I show UnityEvents is going to EventManager. You remember down here, I said that you can put a question mark after onPlayerDeath here in order to check whether it exists so you don't fire an event that nothing is subscribed to because I've asked you get an error. I take that out. I'll just show you this. I had it already there. I just checked that it worked. I've taken out the unsubscribe and subscribe codes and nothing subscribed to this event. Then I've taken out the null check for the event, and look here, because the event is fired but nothing subscribe to it, then you will get an error. That's just to prove my case there. But at the moment, I'm going to set that back anyway, but for the time being, I am going to comment that out because what I want to show you is a UnityEvent. UnityEvents, special things from Unity. First, we could do Serializedfield, and then I'm going to call this UnityEvent and I'm going to queue it on UnitEvent. There's a reason for the error onPlayerDied.
Play video starting at :1:48 and follow transcript1:48
It's like onPlayerDeath. Same, but different. So there's an error there and you can see that my syntax is not color-coding because it doesn't recognize it. When things like this happen, in Unity, often it's because you need next to using statement at the top. In this case, you're missing a library basically. It's using UnityEngine.Events.
Play video starting at :2:15 and follow transcript2:15
Now, everyone is happy. We've done that, we saved everything.
Play video starting at :2:24 and follow transcript2:24
I'm going to clear that at the bottom because it bugs me. It says, that's never used, hold on then. That's a fair point. We created the UnityEvent that no one ever used.
Play video starting at :2:46 and follow transcript2:46
Played on. What's happening there?
Play video starting at :2:59 and follow transcript2:59
UnityEvent is especially common event unique to Unity. It has a method within the delegate called invokes. If you want to use this thing, use the invoke method. Let's go back and I put this script of my gameManager, just go to game manager. Let's look at the inspector now. The new thing that we've got here is we've got onPlayerDied here in the inspector and you can see there's some debris, that's the clicker debris. We've got an option here, runtime it. It's a runtime, runtime only, just leave it as its default. It's waiting for an object and a function.
Play video starting at :3:38 and follow transcript3:38
I did it with code in the last example because I was using C-sharp events, plane C-sharp events rather than UnityEvents, where you can do UnityEvents as you can subscribe and unsubscribe in the inspector. There were different use cases for this. If

you had lots of it, things that were being spawned and unspawned within a level at runtime, it's probably a good idea to do more programmatically with C-Sharp events. They also tend to be a bit more performing actually as it goes. But if you've got a smaller collection of events, like a play death and you don't need quite so many things to be happening and you're just trying to tweak one or two things per event. Then maybe the UnityEvents adding and subscribe and unsubscribe and inspector is the way to go. It's also good if you're working with people who aren't so code-friendly. Basically, if you're working with designers or artists and other people, they can add and subtract from the inspector. You're essentially exposing the inner workings of an event to people who aren't so [inaudible] with the code. It is also less lines of code, although arguably C-sharp events are kind of easier to manage and they're easier to debug. Hopefully, you can see the different use cases possible. How do we use this? We've got an object there. I wanted to say to the player sounds. If I drag and drop PlayerSounds over here, then when I click, I've got PlayerSounds there. Sorry, I've dragged and dropped the wrong thing. I want to drag and drop the Sound Manager, not the script, sorry. I put a GameObject is in there. No function, but if we go to PlayerSounds then everything that's in that function is available, but it's not functioning. Why is that? Because it's not public. PlayerDeathSound, let's go back to that. Again. Often in Unity, can't see a thing you expect to see. Is it public?

Play video starting at :5:50 and follow transcript5:50

PlayerDeathSound. Now you've got that going on. Let's run it. There you go. We've got a player death sound and so you could carry on doing this, so you add this, and we drag and drop our enemy. Again, if you had lots of enemies, you would have an object with a collection of enemies there. EnemyActions, and again, I've got the same issue. So you do want these things. This is where you want to start to make these public not to be seen.

Play video starting at :6:36 and follow transcript6:36

Let me trope. Okay, and I click. There you go hurray. The hero is dead and lots of enemy rejoicing, lots of death sounds. I don't need to go through so obviously so you get the idea. You add and subtract and then it's really just relatively easy to subtract things as well. Now, actually, if I go back, I'm going to subtract the last one. We'll just drag and drop our enemy and do the EnemyActions, and we [inaudible] They see that's a lot quicker than typing in that code. It's a lot less fiddly arguably. It's good for prototyping and throwing things together, but as your projects get bigger, you're going to start making a decision about what's most appropriate and who's going to be using this. You can drag and resort these. I thought you could, maybe you can't do that. You can only unsubscribe the last thing that you added there. Just be careful of what you are or are not Adding. There you go. There's UnityEvents have got their own use and C-sharp events got their own use. A lot of people do tend to move towards C-sharp events, but if you're working with people who are less technical, UnityEvents are great.

All I have here is, I have a floor, I have a player cube.
Play video starting at ::18 and follow transcript0:18
All it does is I move around. Let's just maximize my game view here. The movement, it's slippery,on ice type of movement because just very basics forces movement. Move forward, backwards turns around and a button for jump. Let's just stop there and go and look at the code. The code here you can see, so I call this OldPlayerMovement, so I'm going to show you one. It's just the same as what we've done before, we've got moveSpeed, turnSpeed, jumpForce. We made our assignment, taking a button for the jump. I suppose I should have put that in a different function, but I didn't. I should actually put that and there you go, that's better. I don't need to update for time [inaudible] and fix update. I did same things as before, took the input access, had some constants move and turn, then times the by the transform. The transform force is relative to the direction of the object. Remember vector 3.forward is going to be forward globally, transform.forward is forward locally, so just be wary of that. That's all I have there. Now, that's the old input manager and if we go back to Unity, and if I go into the top right-hand corner and I go to Project Settings from the Edit menu, then we have the Input Manager. This is the thing that we have looked at before we got different axis and I know where to get horizontal axis, I know what keys will trigger the horizontal axis and I can go to jump, and I know what keys will trigger the jump action.
Play video starting at :2:13 and follow transcript2:13
The problem with this is this Manager designed a couple of control systems for the PC really kicking about. There was mouse, there was keyboard and maybe an X-box 360 controller. Unity is quite an old engine and some of these features have been around for a very long time and there weren't really that many different devices to play on at that time that you could develop on the PC and then import to these things. Unity has grown to become this cross-platform engine where you can write once and publish all over the shop to many different places. With that, you have to take on board the variety of different input methods that can arise. You've got different game-pads, PS4 pads, 360 pads, you've got touch, you have XR, VR and AR controllers. The trouble with this system here in Unity, is that if you wanted to add any of those controllers, you've got to account for all of them. There's one or two of them set up for you by default but you notice there was no game-pad setup here. It's just mouse and keyboard. If you wanted to have the game-pad as a mouse button, etc., the game-pad is a fire buttons or jump button, etc., you'd have to program that in yourself. Unity have created a new system that helps you do a lot of this. We're going to delve into that now. I have to do this in two parts because you have to install something and reboot the editor. What you're going to do to do this after this is go to Window, and you're going to go to Package Manager. That's taken a moment to load, which you can see down here, it says, loading packages. There you go, it's loaded now. The Package Manager is a bunch of features that are included with Unity, but they're not included by default when you open up Unity.

That's to prevent infinite bloat basically of the editor because, for example, I personally have had no need to use Oculus, Android and Oculus desktop because I don't do development for VR. There's lots of things in the here that I'm not really interested in, so I don't need them in my editor when I run it. But what we are going to look at is the Input System. With the input system, you're going to go to here, you're just going to click "Install" and then once it does this, it's going to ask, can I install? It's going to say, can I reboot the editor using a different system? Then we're going to say, yes, and that's what we're going to come back to it. Here we are, the editor has rebooted and we're ready to go, so it should switch over to new Unity system. The first thing you're going to notice is you try and run it as you were. Then A, your controls won't work and B, [inaudible] throwing a bunch of rows down here because it's saying you switched over to the input system. Why you trying to access the input manager? Let's just set ourselves up for doing things differently. I'm going to remove the old player script so you don't have it there at all. I'm going to go over to my project. I'm going to duplicate that control D, I did there. I'm going to rename this new, not new, it's not a cat, new player.
Play video starting at :5:38 and follow transcript5:38
That can be useful for you to keep your old code you compare. Open that up. Make sure, of course, when you rename a file that you rename class name as well. That code is exactly the same. Okay, that's great. Then before I forget and think that I'm going crazy, I'm just going to drag and drop the new player thing there. It's pretty not old phase it 40, 30, 10, I think we had. The first thing you need to do if you're going to use the input system, you need to use one of the components that input system affords you. Over on your inspector, you can do at component like it did there, you're going to put an input. There's a few things there. What you're looking for is this player input components. The player input component, it's got a bunch of things that you can attach to it, and do with it. But the thing that we're interested in for this video is this top one, actions. It's looking for an input action asset. That is a new kind of file that you [inaudible] wants us to make. Let's go to Project. We can right click, we can create, and you can go down to the bottom here and it's got input actions. Let's do that. Input actions are going to call that player controls, [inaudible] to shift there. Let's double-click on it to open it up. Adopt that my window for ease of use. You can see there are three columns on the screen behind me: action maps, there's actions, and there's properties. Action maps are groups of controls that fit a situation in the game. If you had to play it in a third person viewpoint game with cameras up and above your head here, you might have different controls. What you might be the first person game as well actually, but you might have different controls for when you're running around and picking things up and putting things down or shooting or throwing things. You might have different controls when you get in a car and drive, when you find a boat and use boat, when you jump into an aircraft and takeoff or use a helicopter, you get the idea. Different situations have different controls relevant to them. Action maps are groups of actions. What's an

action? An action is a thing that you do in the game like move, jump, shoot, pickup, put down, drop, etc. Then the third thing, you got properties there since the properties of the different actions and this will become clear soon. But the third thing in the input system that you can't see in these three columns is the bindings and the bindings of the buttons that make the actions happen. That's where you link a button to do a thing. Let's do an example to illustrate this. Actions maps are going to go through game play. There. We've got new action there. I'm going to call it move. At a moment, we can see the properties for this. Up here's just zoom in a little bit bigger. The mound action type is button. No, I don't want this to just be a button. I want the move to be for the forward and backwards and the turning motion of my player as I had before. I actually want it to be a value of some sort that's going to be different. The control type for this instance, because I want it to be forward and backwards and turn left and right. That's equivalent to a y-value for me go forwards and backwards, and a X value to determine me turning site. I want to go for vector two. We've done that. Then what I can do on the little plus menu on the move is go add binding, it has one by default, I forget that. At the bindings, so in the properties, binding got this thing called the path. Let's go to the path. This is where you start to really realize just how much there is in the input system. There's all these different kinds of controls including all your exile controllers. You got generics and you've got like specific ones down here. It was [inaudible] they actually support down there and then other controllers, they go just generic HID keys as well, if they're not part of your well established brand. For the time being, I'm going to map for a gamepad. A GamePad and the Left Stick. That's nice and easy, and actually because it's often the case. I want to do a GamePad and a D-Pad as well. Left Stick of a GamePad, left stick of a D-Pad. This would also be, notice, I can go into more detail and say, I want to use the D-Pad, on a PS4 controller run D-Pad on anything. But there's no point for me doing that. You may be in situation where you want an Xbox controller to work different to PS4 controller to work differently to Switch Pro Controller. In which case, you can have that fine-grained control. But for a lot of things, and certainly for us here in this example, I can just choose the D-Pad on the generic GamePad. That's great, but I also want to keep keyboard control as an option. Let's go back to here add a binding, I'm not going to add a binding, I'm going to add a 2D Vector Composite. Now on the keyboard, there's no analog controls to be had here, and I don't have any D-Pads obviously or Sticks to move. It needs to take an input from a bunch of things to create a vector 2D value and this is going to be up, down, left and right on the keyboard. Add the vector composite, give that a name of keyboard, and so we're going to assign these keys here. Now you saw me click through and they'll drop you where you were last time. You see rather than just a D-Pad, it's actually looking for certain value from the D-Pad. I could go to keyboard and I could go by Character Mapped to Key and I can scroll through here and it's not actually there. Let's go to this one. Here we're using US lay-out. There we go, there's my Up Arrow. I could do that and that is fine. You can see that works. Let's see a

few different ways you can go to path, and I could just put down in the top here, it'll bring up every single down thing. But I go down keyboard, great. The other way you can do this is I can click the "Listen" button and then wherever GamePad, wherever control device you got connected to your computer, it will listen, I could press "Left", there you go Left Arrow. Now, I can do the same for the Right Arrow. You can see fairly quickly, we can get that together. One other thing I'll show is if you want to have a bit more finer control over if someone's doing the GamePad that want them to use these values or if someone is using the keyboard and wants to use these values, and so fine tune. Also if this gets very cluttered here, then you may want to have setup different control schemes. You could do a keyboard control scheme, add a keyboard for that. The keyboard control scheme is going to use the keyboard device, and then let's add another one, I can say GamePad. I could do the same for the generic GamePad. I'm going to keep it as. What I can do is over here you now see there's this extra box here. I can say users in control scheme, D-Pad, GamePad, and then this one is the key, I think I'll just miss click that one. I know why. It's because I've gotten the GamePad. If we click on the "keypad", that's going to bring up keyboard because I click keyboard for time being, there's not that much so I'm going to say all control schemes, but there is another option for you to play around with. Were done move, and then I want to jump. Another action, going to call that jump, binding, T bindings, of course, one is going to be listen, space, which is from my keyboard. Then you have one add a binding, and that binding is going to be my GamePad. This is a nice little thing I like they've done here. You know how most GamePads are broadly the same since the PlayStation error, GamePad design has broadly converged. You have two sticks, you've got Left Stick, you've got Right Stick, you've got D-Pad in the left, you've got 4-phase buttons on the right, you've always got for trigger buttons on the back, two on each side, and you've always got a button underneath each stick. Even Nintendo's foldering convention these days with the switch controller. When you look at those buttons, the button at the bottom of an Xbox controller is A, but on the switch it's B, and on a PlayStation control its cross. How do you know which one of these buttons that you want to do? To made things easier, Unity, have chosen to use cardinal directions, which is great and that definitely makes things a lot easier for us. We're going to do buttons south because I want that to be the bottom most button, which is commonly used for jump button in the game. I'm going to assign it to GamePad. Did I do that from my keyboard? Yes, I did. Okay, I'm done set up my controls. One last thing, same as was saving your scripts, you get little star here. Make sure you save the asset otherwise, it's not going to do anything. Right. Go up here, head up here input over here of May, my player asset now, drag and drop that over there. One-player asset, you can choose if you've got more than one default control scheme, you can choose it AIR sleeve with any, and you can't set it to auto switch if you want. If a player starts a in the keyboard and picks up a game pad, you can't have the move to gamepad live in the game if you so desire, which is great. We've got our player

input component. I've added my player controls, etc. What happens if I play my game now? Because I've got my new player movement and script, still the same thing. Still my controls don't work and still, I'm getting these areas down the bottom, which is fair enough because it's old code. Let's make that newer code, shall we? Okay. New player movement. I'm going to add one more thing. I'm going to add the couple of things at the top here to use anything to do with the Unity system. We need to add the input system using UnityEngine.Inputsystem, okay? Make sure you're doing that. I would say where that cause in a moment if you don't do that. Another thing that I'd add to top here is [inaudible] and I'm going to declare a vector3, I'm going to call it movement vector, movementVec. That will become relevant in a moment. In the start method we still need to make a reference that's fine. We're not changing that into a folder up to get out of the way. Now, fixed update, this is using the input manager. We looked at the input manager,, we polled what was going on there and stored this value of the axis in the float move. We did the same for ten. We're using the input system now. Let's get rid of those. At first we're going to still need those. I'm going to leave these strictly as they are because you don't want to put in there yet, but we will do. Then I'm going to delete this code here because this is also using the old input manager code because it's using, get button down or get key down. But is going to comment out that, the jump, and we'll come back to the jump stuff in a second. Now, before I move on and do anything else with fixed update, I'm just going to leave it there. It's broken, that's fine. We're going to create two new functions. The first thing we'll do is public void [inaudible]. Make sure that the Autocomplete web work for you here, just press Escape to get out of that. Then input value, this is where you need to using UnityEngine.Inputsystem at the top. Its input value, let's just call that input. It's going to have two functions, we can do another function. Same thing here OnJump won't auto complete for you. Resist the temptation to do that. InputValue input. Actually we don't need to have an input for that as it goes.
Play video starting at :18:42 and follow transcript18:42
If it's not autocompleted, it's not actually inbuilt into Unity, how do I need the code to write a function if we don't move nor jump? The name is important. Why is it important? Is because if I go back to Unity and I go back to my Impact Assets, I've called my actions move and jump. If I create is an action that we shoot, then I would write a function that is OnShoot. If I created an action that said did a funky dance then the function will be do a funky dots, just put on and then name the action. On is capitalized and so is the action, okay? Let's go away from move. We're taking input, remember this was a vector2, so we need to grab that somehow. I'm going to make a temporary Vector2. I'm going to call it x, y input, just for naming sake, and it's input, which is the parameter we fed into the function GetVector2. The input class of the input system has a method called get and I'll just grab that variable for us. It's doing similar to the input.get access of the input manager, really in terms of its function. Then we're going to

say movementVec, it's used to clear first of all, new movementVec. I haven't written that properly
Play video starting at :20:13 and follow transcript20:13
That should be hold on [inaudible]. I'm going wrong there. There you go. I did wonder why it's arguing with the correct spelling, but not the incorrect spelling. Strange. But movement vec is Vector 3, x, y, input, dot, x, 0, then x, y, inputs, dot, y. Now, we know that in a vector. Okay, it's just seemed [inaudible] a bit. We know that in the vector so similar to what we've had before, we know that in the vector you need x, y, and z. If I used a y-value for the z-value of this Vector 3. Remember that a Vector 2 is in x and y, and on the book you go side to side and up and down. It's just an x and y but in 3D. Yeah, y is our vertical position but our left and right hand side to forth and backs and side-to-side, it's z and x respectively. So that's why I'm feeding the y-value of the Vector 2 into the z of Vector 3. Okay, you're going to set movement vec there. Then after we've done that, before I move on to the jump, because I've declared my Vector 3 up here, it means that in our fixed update, we can do movement vec dot z for movement speed and just replace this with movement vec dot x for turn speed. Now that's all, it would be nice, wonderful. Then this which I commented out earlier, we're going to cut and paste into on jump, because down here, this function is going to happen on the button. So isn't going to happen in one frame, so it's all right to keep it as it is. I don't need to put in fixed update because it's not happening every frame, is only happening in one frame. So on jump, it's a button one frame. Do the impulse force for the jumping action. Okay. Then so we've done that. There's one little thing actually that we can go back and do. If you are actually using physics throughout your whole game and you're using input system, probably advisable to go to "Edit" and then "Project Settings" and you'll see now there is a input system package thing here now that wasn't there before. Now, most of these things are default and we're just going to change this update mode. So we're going to create a settings asset, and that creates a new thing in here. Okay, we can just leave that as is, it's not a problem, but I'm going to change the update mode here. So we've got dynamic update, fixed update or manually so you can actually have full control over when you read the updates and when you process them. But I would love tapping in fixed update just to make sure that our physics are okay. It's probably not necessary, but just as a precaution. Okay. We can just close that, we don't need to save it if things safe there. Let's just double check a few now, shall we? So we've got our player, we got our controls there. We have got our play controls over here. We did that already. Yeah. Okay. So got values set up there. Let's use the keyboard, and here we are so, this is left and right and turning. This is my space function. Okay. Now, you remember that we also, ID'd a game pad. Okay. So here we have a game pad, so I'm just going to plug that in.
Play video starting at :24:10 and follow transcript24:10

Game plugged. Little bit wires. Okay. Wait for my play one thing to go and look. Sorry, get myself out of the way. And now my, let's just do this. My D-Pad is working if I go to the side, so there's a thing we forces going on here. So my value needs to be big if it's going to turn on the spot bytes of a game. It's because the momentum we had before the back is moving. My A button works, the other ones don't, but the A button does. I didn't have any code saying don't jumbo stem id because there's that. Another thing I can show you that is my left-stick it won't go there yet, because the force is so forward and back, etc and it's turning with my left-stick. Cool. So even best on that is I can use different pads so I can just unplug my X-box controller whilst the game is still running. Pickup my PS4 controller though I'm using wired for the sake of simplicity. Otherwise, you got to be here waiting for me to put together. Here's my PlayStation controller. Here I am using my D-Pad, and that's turning, etc. Then, here I am with my left stick, my right stick, and I'm pressing cross to jump. Wonderful. That's all working.
Play video starting at :25:45 and follow transcript25:45
Don't forget that I haven't actually paused the game. I haven't had to pause the game and restart it, having connected the right controller, I can plug in and unplug, hot plug as I go in the game. The input system really is a very powerful way to get lots of different controls in there and you didn't have to write every single code line by yourself. Very powerful, gives you access to lots of different controllers will be particularly useful if you're interested in doing VR headsets and hand controls and things like that. It makes the process of multi-player a lot simpler if that's something that you're interested in because you can have as many input module. These play input modules that you have over here, you have one of those on each player and you have a player input manager which was a different component that you might have seen in the list earlier and that will manage all of those. It makes multiplayer a lot easier to do and a lot easier to get into. Then one last thing that I will show is let me zoom out. There's a debugger that you can have here, which padded I have there had my PlayStation controller. I may see this, here you go on the dual shock pattern. If I double-click on that, I've got a window of to the side. I have to show this in my play mode, I'm afraid. It works this way, makes sense, but input debugger will register everything that I'm pressing. You can see that it's registering the presence of this every fraction of a second but as I move my left stick, and if I scroll down the right stick, is there. If I use my button, here you go. Here's the buttons itself, and I'm pressing "X" so it crossed there. This gives you a lot of scope for getting in there and debugging, very powerful debugger there. Don't have to write your own debug code to display to the screen or display to the debug console. Useful piece of kit. I hope that gets you set with input system. They hope the advantages to using the input system really clear especially even if you're just using standard controllers and you just using the keyboard but especially when you start using lots different controllers to give you different control options within your software or your game. One last thing that I nearly forgot to show is we were writing on move function, on jump function, and then

put it on structures within there. There is another way of doing this within the input system. Let's just look at the input component. You can see that behavior and this the default behavior by the way is broadcast messages. There is a drop-down box behind here and what you can do is you can invoke. Broadcast messages, a lot of you into programs don't like them because they tend to be inefficient. They're really good for getting up and running but as your project gets bigger, and you want to have a bit more finer tune control and efficiencies become an issue later on, then you may actually want to, God help you not you send messages but invoke Unity events or use C-sharp events. If you look at Unity events, well, look, this is starting to look rather similar to what we had in the previous couple of videos. Here you have this is event that happens for every player input, which is when the device is lost, when it's regained, and when the controls are changed, so you're switching from keyboard to game pad and vice versa. Then up here, we can drop down and there's our action map. I only had one action map here, remember? But if I had more, I'd have more toggled lists. There's game-playing, there's my move function, and there's my jump function and then I can add functions to this forum on different objects. In the same way that I was using Unity events and C-Sharp events previously in the notes, you can go the same thing for C sharp, invoke C-sharp events and then go at it that way. Remember the Unity events from last time was when we could C-Sharp events you'd be using your publisher subscriber model encode. With the Unity events you got that publisher subscriber model but it configured through the Inspector which can be really useful especially if you just want to make a few quick fixes or you're working with people who don't have quite so much access or ability with the code. You've got move and jump there and any action that you defined would be there as well. It's just another thing to make you aware of what's there in the input system.

Key frame animation is a great way of creating compelling, interesting movement. And we might want to get our objects to respond to our interactions through key frame animation. For example, we might have a whole bunch of animations. We might have carrots standing still, running away or eating. And what we need to do, is know when to trigger which animation in response to different events. And we do this with something called a state machine. A state machine consists of a number of states. Each state is basically an animation, but the important thing is, it also consists of transitions between states. So that is when you go from one state to another, and each transition has a condition attached to it. So that's the thing, that has to happen for that transition to happen. So, for example, if I'm standing still and I see a predator, I might run away.
Play video starting at :1:6 and follow transcript1:06
On the other hand, if the predator is now out of range, I might go back to standing still.
Play video starting at :1:12 and follow transcript1:12

Same thing, if I see some food or I'm hungry, I might go and eat, and when I finished, I might go back to standing still. So a state machine is a bunch of animations, with conditions for going from one thing to another. And those conditions are basically variables, that we can set up in unity, and we'll see how to do that in practice in a minute.
Play video starting at :1:34 and follow transcript1:34
How does this work in terms of actual interaction with objects? So, for example, we had a door, the door might start off closed, and when the player enters its collision area enters gets near the box. It might transition to an animation which opens the door, and similarly, when the pert layer leaves the condition box, it will transition back to an animation that closes. So what we have is animations that are actual actions opening and closing the door, and conditions which happen when we which trigger those particular animations. And as we'll see, it's possible to trigger those animations, directly from player interactions.

Okay, so today we're going to be looking at the Unity animation system, also known as Mecanim. Now, in order to do that we first need to set are seen, as you can see I have a nice kind of field thing going on. I have my, agent who were going to be animating at the moment, the first thing that we need to do, is that we need to open up the animated view. So I have my open already, but if you wanted to find it, it would be in the animation term.
Play video starting at ::48 and follow transcript0:48
There we are. Then the second thing that we want to do, is that we want to have an enemy's a component, attached to the actual game object. This allows us to actually, create animations that are going to be for this game object specifically. What was good about that is that once we do have a controller, we can also give the same controller two different, game objects or say players or agents and they'll be able to do the same thing. So right now let's set up for this agent over here, we don't have a control at the moment. These are mostly related to animating a humanoid avatars that we're going to ignore that today, for this tutorial and focused on this controller here. So we're going to go into our project folder and we're going to create a kind of mental control them, we're going to name it Kennedy controller, and a really original. But let's keep the naming clear and consistent. So we have our enemy here, and once we create that we can either drop it in there like so or we can find it in the list here. So things like that and now we have any controller. So immediately you can see we have a bit of change in our animator window. Let's take a look at, this space for a moment, here we have the kind of visual representation of our transitions between states and animations. We have a topic ward layers, which allows us to kind of lay out our animations together and put different weights on saying animations. For example, if you wanted someone to, shoot and walk at the same time or, run and wave put different weights on those different animations to

kind of bring them together. But today we're going to be focusing on mostly the perimeter section here, this allows us to set the conditions, in order to change from our different animation states. So speaking of animation states, here we have some default ones that are given to us already in our base layer here. If we zoom in, we can see that we have, an entry state, any state entry state and then we have an exit state so all animations are going to take place between this entry and exit state here.
Play video starting at :3:32 and follow transcript3:32
So if we play well, see that nothing is happening, as we don't have any animations in place yet. So the first thing that we want to do is at an animation to agent. Now if we did have, animation from the asset store or one that we create externally and another program we can just drop it in there. But if we are, going to be making it within Unity has something called the animation view which allows us to create key frame animations. So in order to do that it's quite simple, it allows us to create an emission clip, so let's start with doing an idol by your animation here. So we're going to save that and immediately were shown a timeline. Now this timeline allows us to basically, manipulate different properties of this game object frame by frame. So as you can imagine this can get a bit complicated, if you're doing really in detailed stuff but for simple movements, we can definitely get away with it. So you can see that it's already placed the idol animation in our base layer of our enemy time view here, but nothing's happening, so let's make something happen. So the first thing we're going to do is that we're going to and select a property to manipulate. So for our idol animation, I'm going to select rotation, and you can see that it's now position that in the, hierarchy window here and we have value set to zero so it's not doing anything. So there's two ways you can do it, you can actually physically, manually just just select the different values here to shift or what I like to do is that I like to record, the changes that I'm going to do in real time to my game object. And it will kind of record those different changes in value to the xyz, axis of rotation for me in this state here. So that's what exactly what we're going to do. So I want to kind of wobble this left to right to represent basically an idol state. So what I'm going to do is that we have, okay so it's one here, so in between in these kind of quarterly segments here,we're going to have it shifting left to right. So things like that, I'm going to start recording, I'm going to zoom in a bit, and I'm going to basically shift this to the next thought to the left first, right. So once I do that, I'm going to switch to this steak here and I'm going to once again, news that to the right of it.
Play video starting at :6:38 and follow transcript6:38
Right, so, that's maybe a bit too much, I'm going to just zoom it back a bit and I can now stop reporting and I can play my animation to see what I've done.
Play video starting at :6:52 and follow transcript6:52
So I'm just going to move this in a bit so it can return to the original position faster. So it really gave us, kind of beginning and end, basically representing 000 and zero those are in the state as well. So it returns back to its original state, creating a nice clean loop.

Play video starting at :7:17 and follow transcript7:17
Okay, so I think that's quite nice, and once you happy with that, if you go to your project folder, you can see that now that's in there as an animation and we can now manipulate this animation in the Inspector. One of the things that we can do is that we can loop the time, so this means that once I press play do that now the animation will keep going. And if we switch to our animator view, we can see that it's now gone from, t's now gone from entry to this state here to this animation reference here, which is idle and it's playing over and over again.
Play video starting at :39:55 and follow transcript39:55
Put it in brackets because it's a variable. And then also of course we want to actually make the animation play.
Play video starting at :40:7 and follow transcript40:07
So let's spell that right. Who wants to say anime? animController, and then we want to set float to, you can guess.
Play video starting at :40:22 and follow transcript40:22
You want to set it so that isJump is now going to be manipulated by this counter. So that value is represented by this variable here. The value of isJump is represented by this variable here which we're implementing. Okay, so with that let's double check that everything here is okay. So we're going to save that and we're going to check that Unity isn't angry. Ooh, that's a big angry one. Okay, so you're seeing Animator doesn't contain a for, yes because that word does not exist. Okay, so save.
Play video starting at :41:12 and follow transcript41:12
And, yeah, animator does not do-do-do-do.
Play video starting at :41:22 and follow transcript41:22
Again I made a mistake. So you have to be really careful about functions that are caps sensitive. Okay, so yes, now it's all done.
Play video starting at :41:43 and follow transcript41:43
And let's double check again. So, yeah isSpinning, we're not looking at isSpinning, we're looking at isJump. Right, here we are. So, if it's greater than 10, then we're going to start jumping, but then as soon as we start going over 20 then we're going to stop jumping. So let's see, I feel like I'm about to go into battle, right? Like you have to wait for the right time for your enemies to attack. So you can see this is going up. I'm going to be moving away so it starts jumping, and I'm just like, snap, I can't attack, so I'm going to wait for it to hit 20, and we're going to see that this stops and I can hit it. Well, I can hit it, yep. Well, I win, I win. Okay, so that is just a quick example of using these parameters here to manipulate small little state machine here with a few animations. Of course this is just one way of doing this. As you can see that as soon as we start our agent actually begin to being animated straight away. But if we didn't want it to start, just to quickly show you we can actually create a empty state. And instead of it going straight into our first animation we can actually let this be the default state.

Play video starting at :43:24 and follow transcript43:24
No we don't want to make transition, we want to make this the default state. Right, so once we do that [COUGH] it doesn't move, right? So you can actually have it start moving upon another transition here. So it starts moving after a delay, or we can actually put another parameter here that sets a condition for it to switch to this animation. So we're going to leave it here for today, and I hope you have fun playing around with Unity's animation system.

Okay, so today we're going to be looking at the Unity animation system, also known as Mecanim. Now, in order to do that we first need to set are seen, as you can see I have a nice kind of field thing going on. I have my, agent who were going to be animating at the moment, the first thing that we need to do, is that we need to open up the animated view. So I have my open already, but if you wanted to find it, it would be in the animation term.
Play video starting at ::48 and follow transcript0:48
There we are. Then the second thing that we want to do, is that we want to have an enemy's a component, attached to the actual game object. This allows us to actually, create animations that are going to be for this game object specifically. What was good about that is that once we do have a controller, we can also give the same controller two different, game objects or say players or agents and they'll be able to do the same thing. So right now let's set up for this agent over here, we don't have a control at the moment. These are mostly related to animating a humanoid avatars that we're going to ignore that today, for this tutorial and focused on this controller here. So we're going to go into our project folder and we're going to create a kind of mental control them, we're going to name it Kennedy controller, and a really original. But let's keep the naming clear and consistent. So we have our enemy here, and once we create that we can either drop it in there like so or we can find it in the list here. So things like that and now we have any controller. So immediately you can see we have a bit of change in our animator window. Let's take a look at, this space for a moment, here we have the kind of visual representation of our transitions between states and animations. We have a topic ward layers, which allows us to kind of lay out our animations together and put different weights on saying animations. For example, if you wanted someone to, shoot and walk at the same time or, run and wave put different weights on those different animations to kind of bring them together. But today we're going to be focusing on mostly the perimeter section here, this allows us to set the conditions, in order to change from our different animation states. So speaking of animation states, here we have some default ones that are given to us already in our base layer here. If we zoom in, we can see that we have, an entry state, any state entry state and then we have an exit state so all animations are going to take place between this entry and exit state here.
Play video starting at :3:32 and follow transcript3:32

So if we play well, see that nothing is happening, as we don't have any animations in place yet. So the first thing that we want to do is at an animation to agent. Now if we did have, animation from the asset store or one that we create externally and another program we can just drop it in there. But if we are, going to be making it within Unity has something called the animation view which allows us to create key frame animations. So in order to do that it's quite simple, it allows us to create an emission clip, so let's start with doing an idol by your animation here. So we're going to save that and immediately were shown a timeline. Now this timeline allows us to basically, manipulate different properties of this game object frame by frame. So as you can imagine this can get a bit complicated, if you're doing really in detailed stuff but for simple movements, we can definitely get away with it. So you can see that it's already placed the idol animation in our base layer of our enemy time view here, but nothing's happening, so let's make something happen. So the first thing we're going to do is that we're going to and select a property to manipulate. So for our idol animation, I'm going to select rotation, and you can see that it's now position that in the, hierarchy window here and we have value set to zero so it's not doing anything. So there's two ways you can do it, you can actually physically, manually just just select the different values here to shift or what I like to do is that I like to record, the changes that I'm going to do in real time to my game object. And it will kind of record those different changes in value to the xyz, axis of rotation for me in this state here. So that's what exactly what we're going to do. So I want to kind of wobble this left to right to represent basically an idol state. So what I'm going to do is that we have, okay so it's one here, so in between in these kind of quarterly segments here,we're going to have it shifting left to right. So things like that, I'm going to start recording, I'm going to zoom in a bit, and I'm going to basically shift this to the next thought to the left first, right. So once I do that, I'm going to switch to this steak here and I'm going to once again, news that to the right of it.
Play video starting at :6:38 and follow transcript6:38
Right, so, that's maybe a bit too much, I'm going to just zoom it back a bit and I can now stop reporting and I can play my animation to see what I've done.
Play video starting at :6:52 and follow transcript6:52
So I'm just going to move this in a bit so it can return to the original position faster. So it really gave us, kind of beginning and end, basically representing 000 and zero those are in the state as well. So it returns back to its original state, creating a nice clean loop.
Play video starting at :7:17 and follow transcript7:17
Okay, so I think that's quite nice, and once you happy with that, if you go to your project folder, you can see that now that's in there as an animation and we can now manipulate this animation in the Inspector. One of the things that we can do is that we can loop the time, so this means that once I press play do that now the animation will keep going. And if we switch to our animator view, we can see that it's now gone from, t's now gone

from entry to this state here to this animation reference here, which is idle and it's playing over and over again.
Play video starting at :39:55 and follow transcript39:55
Put it in brackets because it's a variable. And then also of course we want to actually make the animation play.
Play video starting at :40:7 and follow transcript40:07
So let's spell that right. Who wants to say anime? animController, and then we want to set float to, you can guess.
Play video starting at :40:22 and follow transcript40:22
You want to set it so that isJump is now going to be manipulated by this counter. So that value is represented by this variable here. The value of isJump is represented by this variable here which we're implementing. Okay, so with that let's double check that everything here is okay. So we're going to save that and we're going to check that Unity isn't angry. Ooh, that's a big angry one. Okay, so you're seeing Animator doesn't contain a for, yes because that word does not exist. Okay, so save.
Play video starting at :41:12 and follow transcript41:12
And, yeah, animator does not do-do-do-do.
Play video starting at :41:22 and follow transcript41:22
Again I made a mistake. So you have to be really careful about functions that are caps sensitive. Okay, so yes, now it's all done.
Play video starting at :41:43 and follow transcript41:43
And let's double check again. So, yeah isSpinning, we're not looking at isSpinning, we're looking at isJump. Right, here we are. So, if it's greater than 10, then we're going to start jumping, but then as soon as we start going over 20 then we're going to stop jumping. So let's see, I feel like I'm about to go into battle, right? Like you have to wait for the right time for your enemies to attack. So you can see this is going up. I'm going to be moving away so it starts jumping, and I'm just like, snap, I can't attack, so I'm going to wait for it to hit 20, and we're going to see that this stops and I can hit it. Well, I can hit it, yep. Well, I win, I win. Okay, so that is just a quick example of using these parameters here to manipulate small little state machine here with a few animations. Of course this is just one way of doing this. As you can see that as soon as we start our agent actually begin to being animated straight away. But if we didn't want it to start, just to quickly show you we can actually create a empty state. And instead of it going straight into our first animation we can actually let this be the default state.
Play video starting at :43:24 and follow transcript43:24
No we don't want to make transition, we want to make this the default state. Right, so once we do that [COUGH] it doesn't move, right? So you can actually have it start moving upon another transition here. So it starts moving after a delay, or we can actually put another parameter here that sets a condition for it to switch to this

animation. So we're going to leave it here for today, and I hope you have fun playing around with Unity's animation system.

A conversation is the archetypical human interaction, two people just talking to each other. But they aren't just talking. If you watch a conversation carefully, you will see that there is an amazing amount of complex body language going on. We communicate a lot using our bodies. When we talk, we typically use gestures maybe to point something out or just to emphasize what we're saying. Our posture can show if we are tired or alert. We express emotions through our facial expression. Eye is a really important in interaction. Making eye contact is one of the most important social signals. Even where we stand is a social signal. Entering someone's personal space can be uncomfortable. We're very careful about how we maintain an appropriate social distance. All of these factors make for an implicit undercurrent in a conversation that expresses how we feel, our relationship with the other person, and many other things. In a conversation, we also talk. Let's not forget about that. Our speech is the explicit meaning of the conversation, and all of the other body language is structured around the rhythms of speech. But even talking isn't just about words. Our tone of voice says a lot as well. Simulating all the complexity of a conversation can make for a really powerful social experience in VR. In fact, it is one of the most intriguing experiences we can have inside virtual reality. In the next few lectures, we'll look at all of these social signals, and how we can simulate them.

The human body is a single complex objects, but one whose individual parts move in different ways. My hands and head are all part of my body and tightly linked, but I can move them all pretty independently. Like most 3D graphics objects, human bodies are represented as a polygon mesh. But unlike other objects, it isn't enough to simply move the whole thing around with a single transform, we need to move our limbs. We could create a compound object with separate objects for our hand, lower arm, and upper arm and this is what a early computer games and computer graphics did. But it isn't very realistic, my lower arm and upper arm are not separate objects, even though they move relative to each other, they form a single smooth surface. So nowadays we do something a bit more clever. While the body is complex, and its outward appearance has many features, the majority of the movement actually comes down to the movement of the skeleton. The skeleton consists of a number of rigid bones linked together by rotational joints. When we move, our joints rotate, and so rotate our bones relative to each other. In 3D character animation, we have a single probably very complex mesh, to represent the body. But we also separately represent the skeleton, the skeleton is a series of rigid links with bones, with rotational transforms, the joints. We animate both the character by rotating those joints. These rotations are then mapped onto the mesh using a process called smooth skinning, which is done automatically in the GPU. This

results in a realistically moving mesh, but we don't have to think of the individual movements of all the vertices. We only have to animate the joints. This makes our animation data relatively simple. It's just a number of key frames to the rotations of the joint. Setting up this skeleton animation can be quite complex, as we need to create a realistic mesh, and also a skeleton which consists of a number of transforms. These transforms are hierarchical. That means that each transform has transforms attached as children, and these children themselves have children. So my shoulder has my upper arm as a child, which has my lower arm, which has my hand. My hand is very complex with a child for each finger. This skeleton needs to be attached to the mesh, which means that we need to say which bone each vertex is attached to. A process called rigging. To achieve smooth movement, vertices can be attached to several bones. This whole process is pretty hard work. Most animation packages will partially automate it, but it rarely works perfectly. Luckily, as usual, the Asset Store comes to our rescue and contains a number of excellent pre-rigged characters. Skeletal animation works through a process called Forward Kinematics or FK. I rotate a joint for example, my shoulder that moves my whole arm. I then rotate my elbow, which moves my lower arm, and my wrist, which moves my hand. The position of my hand is worked out by starting at the top joint, my shoulder, and applying its rotation and then going down each joint in turn, applying that rotation until we reach the hand itself. FK is just handled automatically by the way transforms work, and you really don't have to think about it at all. The only reason I mentioned it is because we need to know about its opposite, Inverse Kinematics or IK. Forward Kinematics can be quite complex to animate with. Often we just want the hand to be in a particular position, maybe to pick something up. Doing that without FK is very fiddly. I rotate my shoulder to where I think it should be, then my elbow, then my wrist. It's there but not quite or it doesn't look like, so I have to go back to tweak my shoulder and it all continues. That's why Inverse Kinematics was invented. IK is an algorithm which takes the position where you want the hands and does a load of calculations to work out what the rotations of all the joint should be to get your hand in the right position. It's very useful for animation. It's included as standard in most 3D animation packages like Maya, 3D Studio Max, or Blender. But it's also useful for doing real-time interaction. For example, getting characters to pick up objects, or even doing a fist bump with you. Isn't just for arms either. We'll see in a future video how we can use it to get characters to look at things. Skeletal animation is a very powerful and flexible technique for human body animation. We'll see in future videos, that it underlies all of the more complex things we'll want to do with body language.

In this video I'm going to take you through the process of adding some animations to a character. So as you can see in the scene and game window, I've used the male MCS Character which is the the character that Tara set up for you. At least the male version of that. And sort of wearing a standard clothes. So if we go here in the hierarchy we can

see that it's the male M.C.S. We can do we can change things we can and leave his vest for example. But I'm not going to talk about customizing the character. I'm going to talk about animation and we'll see here that I've got this folder animation with a bunch of animations and these are FBX files. For example this one is a character talking, now I'll note a couple of things that the animation type needs to be humanoid. And often if you loaded FBX file it will be set to genetics. So I'm going to change it to humanoid and apply that. Okay, let's grab some animations. I'm actually going to start with Wilmer Cap, that's a pretty common unity provided one which is on the asset store. And I'm going to look at one of the idle animations, so we're going to start the character off in an idle state, which means that not really doing very much like this. Okay, so this is a good animation, but I don't want all of it. So out of these animations that are loaded from experts from FBX files, I can create a new animation. Let's call it idle one.
Play video starting at :1:56 and follow transcript1:56
And what I do is I use these sliders to select a little bit of the animation.
Play video starting at :2:7 and follow transcript2:07
And in particular, I'm looking for the ability to loop here so that the end pose is similar to the beginning pose. And if it's green, that means it's good enough to loop.
Play video starting at :2:24 and follow transcript2:24
So let's take that one.And I apply that, and now you see Idle one appears in, under the prefab of idol one neutral okay good. So now we need to get the character to play it. So let's go back to the character and then feed email and we see the characters already got an animator but it doesn't have an animator control S.So I'll go back here into animations and create an animated controller which is male character animator. And here it is this is the animated controller. It is a state machine which means that, it consists of a number of states which are essentially animations and you can move from one state to another.I.e you move from one animation to another. So let's grab that animation that we just created Idle one and put it in there and it by default automatically becomes the the default animation. That's why it's orange and there's an arrow from entry which is the beginning of the program to idle. So I'm going to turn it on, we turned on leap. So if we now play, nothing happens because spot the obvious mistake. I haven't actually set the animate the controller in the animator. So if you go to animator, click male character animation there it will work I hope. Yeah there we go. So there's the character playing the animation.
Play video starting at :4:23 and follow transcript4:23
Okay so that's fine if we want just one but if you want interactivity we want to have several animations. So I'm going to talk through how we do that.
Play video starting at :4:34 and follow transcript4:34
And essentially it means creating a bunch of other animations. So I'm going to do a listening animation and I'm going to do exactly the same thing. Call it listening one and I'm going to leap that one as well.

Play video starting at :4:56 and follow transcript4:56
Let's just have a nice bit of listening animation like that, apply that. And then conversation.Big idea, same thing.
Play video starting at :5:28 and follow transcript5:28
Talking one, sleep like that.
Play video starting at :5:41 and follow transcript5:41
Let's choose a couple where the, and down there and apply that.
Play video starting at :6:7 and follow transcript6:07
Yeah, we've got talking one. So we've got now three states, three animations in here and what we need is transition. So I'm going to make a transition to here and make transition to here and now. If you watch this, let's now maximize it to see what happens. We can see that it plays through the idle animation. I've made this quite a long animation because it's, I want the character to be able to idle for quite a long time in case the player doesn't interact with it, with him and it's sort of fairly neutral. He's looking around and sort of shifting posture he wants to move. And they've seen it's going to end and then transition into listening. So there we go listening, nodding. Now you'll notice that the character's jaw is suddenly gaping open in not the most intelligent looking expression in the world. And this is something we don't want. I'll show you how to fix that in a minute. But we've seen that we've transitioned from idle to listening to talking. Okay so last thing to do is to sort out this problem with the jaw.Now that is a problem with the character's avatar. I don't want that.
Play video starting at :7:44 and follow transcript7:44
So let me go to MCD male avatar and configure avatar. Now why the open jaw happens is that, the character has a bone in its jaw so it's moving the jaw is moving. But the animation itself and we can have a look at the animation, actually need to close that a minute to look at the animation.
Play video starting at :8:24 and follow transcript8:24
The animation itself. If we go to listening one, it has a key frames from jaw close and that key frame is set to zero which seems like a reasonable value. And that's probably why it is. But in fact it's jaw close zero is an open jaw because the key famous for closed jaw.So it's the jaw is closed at one so that's slightly annoying. And it's something that comes up a lot.Basic way to do that is to go into the avatar which is the definition of which actual joints in the actual model. So which transforms correspond to which kind of notional joint, the hip, spine, upper arm, hand that McCann assumes.And I'm going to go to the head and going to the jaw and select none, so that it's no longer animating the jaw. I'm going to animate the face entirely. With move targets, blend shapes, which we'll talk about next week. So we don't need that. So let's go back select the character. No, we need to close that. Okay, that select the character, go to listening and hopefully the jaw is now closed. So we no longer animating the jaw with bones and that fixes the problem. Okay, so just to recap, we've got a a character with some animations and if we

press play it plays those animations. Not only one, but it will transition from one to the other. And so that's the start for making interactive character. But as you'll notice, not really interactive yet, there's no interaction. So that's the next thing we need to look at.

We use our body in many ways during social interaction and it can communicate many different things. In this video, we'd like to talk about two of the most important ways we use our body to communicate; through gestures and through postures. Gestures are movements we make while talking, mostly with our hands, but sometimes with our heads or other body parts. Gestures can be meaningful. They can add extra meaning beyond the words we say. For example, some gestures have very definite meanings, like the okay sign of thumps up. These are called iconic gestures. Iconic gestures can vary a lot between cultures. Nodding and shaking our heads can mean yes and no in Western cultures. But in India, head shaking gestures mean agreement. My mother is from Southern Italy where there's a very rich vocabulary of gestures. Whether it's complementing the cook on excellent meal or asking for clarification, maybe not in the most polite way. When I first came to the UK when I was 22, I have discovered the use of shoulders in expressing certain emotions, something we don't really do in China. Also, I was very used to counting from 1-10 with one hand, so this is how it count 6, 7, 8, and 9. I was surprised to discover that this is not the case outside of China. Iconic gestures act a lot like words. They have definite meanings that people understand when you make them. But gestures can convey information in other ways that might be hard to put into words. For example, I might say, "I'd like the pink cupcake on the top shelf between the chocolate cake, and the lemon tart." Or I could just point to it. Pointing is a very important communicative gesture that we use all the time. Some gestures on the other hand, don't mean anything at all. Most of the gestures I am making while I talk are called beat gestures. They make the rhythm of my speech and they can provide emphasis but they don't really mean anything. All of the gestures we've been talking about are very closely linked to speech. Sometimes they can substitute speech, but they mostly accompany and enhance it. The timing at rhythm of gestures is closely synced to that of speech. That means if you want characters to do gestures, then we need to create them together with speech. If we're generating speech for the chatbot, then we need an algorithm to dynamically generate gestures to go with it, which can be pretty hard. When I talked about speech, I recommended you use pre-recorded audio clips. For gesture, you can use pre-recorded animations. Ideally, you would record audio and motion capture the gestures simultaneously to make sure they're fully synchronized. But that needs equipment that is beyond reach of most people. Instead, you can get hold of clips of gestures and do your best to make sure they match the speech. Gestures are movements to particular parts of the body, normally hands. Posture on the other hand, is the pose of the whole body when it doesn't move. Posture is less of a directly communicative channel. But it can express a lot of things. For

example, emotion. We can easily recognize a sad, dejected posture and distinguish it from a pro posture. Like gestures, the easiest way to generate posture is to use pre-recorded animations. Maybe selecting different animations to convey different emotions. A useful way of combining posture and gestures is use layers. That means splitting the body into different parts and applying different animations to each part. For example, gestures applied to the arms and posture animation applied to the torso. If you've got to grips with character animation, then posture and gestures are a good place to start using body language because they mostly use the basic techniques. But with them, you can create expressive body language for your social VR character.

Hello, today I'm going to show you how to get an animated character into Unity. Now, your standard way of getting stuff into Unity would be the Asset store, and I can go to assets 3D and focus on characters. Probably humanoids, maybe humans, actually we'll do maybe fantasy characters this time because the game engine is a lot more fantasy characters. You can see there's quite a lot, as usual you click "Free assets" you reduce your choice quite a bit. But there are some characters there, and the very important thing is to look and check that the character is rigged.
Play video starting at :1:5 and follow transcript1:05
It's not very clear here, but it says biped head, which basically means it's rigged. You might want to look for another character that is, just making sure it's rigged. Don't fancy skeleton. I've made it to be 2D, let's see.
Play video starting at :1:34 and follow transcript1:34
Well, there's a bunch of characters, I would spend some time searching out for a nicely rigged character. But today I'm not actually going to use anything of the Asset store, I'm just going to show you another source of characters, which is Mixamo. This is by Adobe, and it's a free tool for creating characters. You have a range of characters, you can select them.
Play video starting at :2:4 and follow transcript2:04
Some of them are quite realistic.
Play video starting at :2:13 and follow transcript2:13
This doesn't seem to be working right. I'm going to refresh a minute and hope that we get it working better. There we go, there's a character. You've got sci fi characters, you got fancy characters, you got normal looking characters, you got cartoon characters. What was I looking for? I chosen this cartoony dancer character that they use in their advertising, and I thought I'd try and import this now. I can just download this and import it into Unity, but I'm actually going to also add an animation to it because I think they'll be quite good. Let's try and find an animation. Now, they make all these animations by gender. There's pink and is female, blue is male as well as some which are quite specific to characters. There's a lot of freedom in there, this female character includes a female dancing animation called hip hop. But it doesn't look particularly hip hop to me

but that's good enough. We've got a cartoony animation. We can download this into Unity. I'm just going to click "Download" These are all fine. SBX is the correct format to important into Unity. Yes, we do definitely want to skin. You could export animations without the skin if you just want the animations, that's another thing you can do. I'll show you how to add and separate animations later, and for now I'm going to download.
Play video starting at :4:8 and follow transcript4:08
No, I don't want to open that, I want to save it. Let's go back to Unity Main scene.
Play video starting at :4:26 and follow transcript4:26
Just grab my download window, I got hip hop dancing one, done this before when I was testing. Let me open up my scene and drag in hip hop dancing one. There we go. Let's have a look at this character. Well, let's drag her in. Now you'll see.
Play video starting at :5: and follow transcript5:00
Set it to zero. You'll see a few obvious things here. First off, while she's rotated, that's fine. First off, she's got no materials. Why is that? Well, we have to adjust the import settings. I've clicked on her hair in the assets. There's a couple of things. Actually, the first thing we need to do is we can click on the model. The model is generally fine. That's a mesh. We can adjust the rig. The rig should be humanoid.
Play video starting at :5:38 and follow transcript5:38
It means it works with the unity animation system except animations for like non-humanoid characters, which we've seen in the last topic, and humanoid characters. This is basically more powerful animation system designed for humanoid characters. There something called an avatar definition, it sets right from this model that we need that but we'll let that happen. Now she will work like a proper animated character. We'll come back to animations in the next video. But I first want to look at the materials. By default, we haven't got the materials set up. I want to extract them. I've just realized, I haven't created a folder for this, so she really needs to be in her folder.
Play video starting at :6:31 and follow transcript6:31
Let's put her in a separate folder for characters because it's a bit messy otherwise. What I'm just going to do is extract textures, extract into same folder.
Play video starting at :6:48 and follow transcript6:48
Now, she looks textured, which is great, and I'll extract materials as well, so we got the correct materials. There we go. She looks as she should be. That's fantastic. Now we've got a character model. Now, let's have a look here at the character model in the hierarchy window. We've got actually two things attached to the character model. Ch03 is at least a mesh renderer. This is the actual appearance of the mesh. This is where the mesh and all the blending, the skinning happens. It's a skinned mesh renderer, not just a mesh renderer, which means its skin, but the skeleton. You've got a bunch of stuff here. Now, you don't normally need to touch anything much in the inspector except if the character has facial animations to blend shapes. If you look here for blend shapes, there are no blend shapes. That means there's no blend shapes attached to this

character. In a future video, we'll look at a character with blend shapes. But here is the other important thing, this is the skeleton. Now, the skeleton is just a bunch of transforms of empty game objects and each of these game objects represents a joint of the character. If I grab left leg and start animating, I can get her to move her left leg.
Play video starting at :8:29 and follow transcript8:29
It's called left leg stands for a lower left leg or knee up left leg, is upper left leg. We can also look at the spine and we get into shoulders, left arm. Again, I can make her left arm move and control her head as well. This is the basic way a character works in unity. You have a mesh, which is a skinned mesh renderer. That's different from other mesh renderers. Then you have this hierarchical representation of game objects, each of which represents a joint. What I mean by hierarchical is that, each thing lower down the skeleton is a child of the previous one. The upper left leg is a child of the hips. The lower left leg is a child of the upper left leg. The left foot is a child of the lower left leg, and so on. This structure is really important. It's what makes it work. It means that by moving the shoulder, you move the rest of the arm, as it should work. The animation of the cat is simply manipulating these game objects, these joints, these transforms. These transforms have nothing attached to them but they are used in the skin mesh renderer to transform the mesh itself using smooth skinning. That's what a character looks like in unity.

So in the last video, we imported this character and showed you how characters work with skinned meshed renderers and with skeletons, which are hierarchies of empty game objects. This time around, I want to have a look at the animations. So I'm going to click on the character in the Project View. So this is the asset itself. Last time we looked at the model, which we don't have to worry about too much, the rig, which has to be humanoid, the materials which need to be extracted. We got that all working right. But today, I want to look at the animations. So here this includes all of the animations that are on the character, there's one animation. If you remember the last time we included an animation on the character. Now that's important. We have it. If you click to open up this prefab, we can see that we've got this animation here and we don't really have to do anything about it. But I just want to show you something that you can actually create some animations here. So I can create out of this animation, I can create a new animation, I can rename that to hip hop there. That didn't work.
Play video starting at :1:40 and follow transcript1:40
Now, why would I want to do this? Well, I might not want to use all of an animation. This actually has a bunch of different things, maybe I just want to cut it down to the first half of the animation so I can select that first half. Now, this is particularly useful for this example because this is actually quite a nice animation in itself. It works well. It's been sort of well-edited. But often if you're importing mocap data, you might actually just

import it as a very long stream of data and you might want to actually break it up so that you can use it in your game or your experience more effectively. So I just wanted to show you you can do that.
Play video starting at :2:27 and follow transcript2:27
I also generally put, well, I shall come back to these later. I won't set these values yet and then we can see what goes wrong if I don't set them. Then if I do apply, I've got a new animation, brilliant. So if we go back to the character, we can see she's got an animator attached to her, but no animated controller for now. So I'm going to actually create an asset here in the same folder.
Play video starting at :2:58 and follow transcript2:58
Just to leave it let me name it, but you should rename yours, so we've got the animated controller. Now we can see here that we've got an animated controller with just the standard entry, entry state and exit states. I'm going to drag this new animation I've created, hip hop, to there and I'm going to hit play.
Play video starting at :3:25 and follow transcript3:25
we can see here she's dancing and I'm actually going to move the camera slightly closer.
Play video starting at :3:44 and follow transcript3:44
Now we can see a little bit better.
Play video starting at :3:55 and follow transcript3:55
Okay, she's dancing but she only does it once. What if I want her to do it later times? Well, this is where I came back to some of the settings I mentioned earlier for the animation. So if I select hip hop, I want to do Loop Time and Loop Pose and no don't edit,
Play video starting at :4:16 and follow transcript4:16
don't edit and I have done that. Okay, so now that means basically this animation can loop. I should not edit this one, I don't want to edit this one. But yeah, there we go. She's looping very nice. Now, there's another issue, she's gradually turning around, why is that happening? Well, there's a slight. Change in her animation that's caused due to her movements. Now, the animation itself does actually have a root animation that stops it turning around, but it's not included.
Play video starting at :5:5 and follow transcript5:05
Let's have a look here. We've got this plan to bake into pose by root transform animation bake into pose. I'm going to click that, and that basically means that the root animation is done properly.
Play video starting at :5:26 and follow transcript5:26
There we go, we've got a nice animation. I was just turning round, that's because I was editing the wrong one.
Play video starting at :5:50 and follow transcript5:50

We've got a character and we've got our animating. This character came with some animations, we created them for her, but that's not always the case. Sometimes you want to add new animations to a character, so I just want to quickly show you that you're not restricted to those. I'm actually going to go to "My assets" because I bought something earlier. I didn't buy it, I didn't pay for it. There are a bunch of free animations. I've got some dance animations here,
Play video starting at :6:26 and follow transcript6:26
and I've imported them, have I? Common import. It's the import. Window is on my other screen, import that.
Play video starting at :6:53 and follow transcript6:53
I actually realized that there's something else I forgot to mention, because the pop-up popped up on my other screen, I will explain that in a sec. This was when I was importing the textures, extracting the textures from a character, a pop-up saying a material is using the texture as a normal match. The texture must be marked as a normal match in the import settings. This will normally come up. We'll explain normal maps later in the course, but for now, you just say, "fix now" and it does what it should do.
Play video starting at :7:35 and follow transcript7:35
I now have some dance animations. Now, most packages from the asset store come with a bunch of examples and example scenes so I'm going to open that up. I'm going to save my scene first.
Play video starting at :8:4 and follow transcript8:04
We can see here we've got a bunch of dancers,
Play video starting at :8:18 and follow transcript8:18
we got a bunch of different models, and models are not very important. The important thing are all the animations, and we can see in this folder we've got a bunch of different animations.
Play video starting at :8:34 and follow transcript8:34
I can click on them and I can preview them here, you can see that actually some of these have already been cut up, as I showed you with the last animation, to have small parts of the in and out animations for them. That's quite useful. They're basically transitions to a standard pose, so it's quite useful to have those transitions in an animator, and you could do that more. I'm not going to bother with that. What I'm going to do, actually, is I'm going to go back to my previous scene and I'm going to import these into the animator.
Play video starting at :9:15 and follow transcript9:15
What we can do is just grab some dance animations. I think it was this one that I liked.
Play video starting at :9:28 and follow transcript9:28
I think it's this long, this is the length of the full one, yeah.
Play video starting at :9:35 and follow transcript9:35

Make a transition between my hip hop dance and this animation.
Play video starting at :9:43 and follow transcript9:43
It just has exit time, so it's just going to play at the end. Let's have a look now, and what happens?
Play video starting at :9:55 and follow transcript9:55
First, in the animator, we play hip hop and then we transition to the dance pack, great. I guess, we actually want to see what happens.
Play video starting at :10:7 and follow transcript10:07
She does her hip hop animation and then transitions into the dance pack animation. This is the great thing about the humanoid animation, the animation system in Unity. It creates a single animation system for a humanoid body which allows us to take animations from one set of characters and apply them to another set as characters. We've taken some animations of an asset store and applied them to a character that is completely unrelated to that animation, they have different names to the rigs, but being humanoid, they have all of the same features. So that becomes possible. That's really powerful and it's really convenient. You can get characters from one place, animations from another place and it all works. I'll show you one last thing, which is, how this all works. It's something called an Avatar. Actually, when I first imported the character, I showed you that we have an Avatar Definition, but I didn't really talk about what it was. It's something that's in here, is part of the prefab and it's also included in the animator. So if I do configure Avatar, it'll show you what it is. What this basically is, it's a mapping from Unity's standard set of bones and joints, hips, spine, chest, upper chest, shoulders, Unity as a standard set into the actual transforms on the character. You've got those for body, head, et cetera. If something's gone a bit wrong, I can change these. Sometimes there's a jaw animation. A jaw animation sometimes causes a bit of problems in Unity, I think it's a little bit of a bug really, but I sometimes remove that if I don't want to animate the jaw and the jaw's doing some weird stuff. But otherwise, if you imported it normally works fine. So I'm just going to click done, I'm not going to do anything with that. But just wanted to explain. What you have now is you've learnt how to grab a character and import some animations and add them to the character. From there you can set up a more complex animator that will do a whole bunch of stuff. Now, it is also possible to create your own characters, you can't really do that in Unity, you have to use an external tool like Maya or Blender, and you can create your own animations. We've shown you how to create animations in Unity, once you get character animation, it gets more complex and Unity's more fiddly. So I wouldn't recommend using Unity as a tool for character animation. I would recommend using Maya or Blender. But, this isn't an animation course, you can learn to do great simple scenes, but creating a realistic character animations, that's a whole skill set of it's own. You're on a computer science degree, you're most decoders, so we're going to work with existing assets here and just focus on how to use them in your scenes and control them from code. Unless you're

really keen on putting on some time in learning those skills, I would recommend working with existing assets because there's some really great ones out there available, particularly on the asset store.

Maintaining the emotional facial expressions can be hard and time-consuming. Motion capture data from one person might not apply very well onto a different phase. If you animate a face by hand, it normally requires years of practice and some artistic intuitions. Some people are more expressive with their face and are more sensitive to subtle changes on the face than others. If you're not so experienced or does not have very good intuition, you might create facial expressions that look a bit strange or fake, which could lead to the uncanny valley effect, which puts people off. Sometimes less is more. It might be a good idea to use very basic facial expressions. We'll just leave it out. However, there are features on the face that are better animated rather than being left out. Blinking and lip-sync are the two very important features, is quite creepy to talk to an human-looking avatar who does not blink and it's really weird and could be confusing to interact with an avatar who talks without opening and closing their mouth. Of course, like everything else on the face, implementing realistic blinking and lip-sync is hard work and is advanced algorithms or someone very skillful. The good news is that there are some cheap implementations you can do to stop your avatar from looking creepy. If your avatar is not super photorealistic with very high fidelity graphics, these cheap solutions work quite well in making believable and expressive avatars. Let's start with blinking. To implement a simple blink, you can just get your avatar to close and open their eyes very quickly. If you're doing it with morph target, you will have to set the avatar to have her eyes open at the start of blinking and her eyes closed in the middle of it, and then open again at the end. There are at least three keyframes needed for eye blink. You will also have to pay attention to how quickly we blink. As it happens more quickly than most people thought. The average duration of a human blink is between 0.1-0.4 seconds. If you are rendering your animation at 60 frames per second, you will have to have the whole blinking finished in between 6-24 frames. That is, for a really slow blink, you should set frame 1 for eyes open, frame 12 for eyes closed and frame 24 for eyes open again. It's also useful to know that we blink about 15-20 times per minute. That's every 3-4 seconds. But it will seem pretty strange if someone blinks exactly every three seconds. You might want to use a random interval between the blinks to make it less predictable. The basic implementation with lip-sync works in a similar way. When avatar is talking, we just need to set her mouth to be continuously opening and closing until she stops talking. We can make it more realistic, but trying to match her mouth shape to her speech. Each sound of phoneme we produce has a distinctive mouth shape, which is called a viseme. Then we can analyze a speech by decomposing it to individual phoneme, and then match the phonemes to the visemes to drive the animation. This sounds like a lot of work but the technology is fairly

straightforward. There are plugins you can get to do it automatically. Finally, although blinking and lip-sync can both be implemented using fairly simple methods, they're not perfect solutions. There are so much more you can do to get them more realistic and expressive. For instance, most people blink more when they're thinking or feeling stressed. Increased blinking could indicate someone is lying because they have to think more. But if they are good at lying, they could make an effort to keep their eyes open and this is just the tip of an eyeborg. There's so much more going on about blinking and our psychological state. To really implement realistic and expressive character facial expression, it's always a good idea to spend some time to study social psychology.

Hello, so now I'm going to show you a little bit about facial animation, in particular, blend shapes. Here I've got a current model of the asset store. Unfortunately, it's a little bit harder to find characters that have built-in blend shapes. Most animation and games, mostly body animation, you don't have that much close up facial animation and that's slightly more advanced topic. When I said it's hard to find, it's hard to find free characters. There's plenty of really good paint characters but for the purposes of scores, I don't want you to be spending lots of money on characters. But there are a few out there that you can try if you want to have a go on facial animation.
Play video starting at :1:1 and follow transcript1:01
This is a standard character, she's got bones and skeletons for character animation, you can see her in the hierarchy. But I'm going to look at the base body. This is the mesh that we're using for the character, the main character animation. She's actually got a bunch of other measures like her hair, but the base body is the main one and that includes, of course, the face and that's where the animations are. So if you select that, you can see there's a skinned mesh renderer component. You can see an inspector and under skin mesh renderer, we have blend shapes. The blend shapes tab, that's where all of the current animations happens and you can see we have a whole load of sliders, lots and lots of sliders here.
Play video starting at :1:53 and follow transcript1:53
Each one controls a minor aspect of the face. So this is open, it doesn't actually look open. I don't know why it's called open. But it is affecting the various aspects of the lips. As we said in the videos, facial blend shapes often represent roughly action units. These are simple, minimal movements of the face, like here's a brow raise or brow drop, one sided and you can use these together to give a real raised eyebrow look. But rather than having both eyebrows raised, you have them together to give you separate so that you got that detailed control. You've got some of these slightly odd like Affricate. What what does that mean? Well, that's actually a term from phonology, so the study of language. It's a very detailed aspect of how you might move your lips while you're speaking. This is useful for lip synching. We have a lot of the lips, but we obviously have something for the eye. So eyes blink, closed eyes, and we actually have one for each

eye, but also one combine because it's such a common thing we want to be able to easily animate both eyes together and we'll use that in a sec. You can really do quite a lot with these blend shapes and create detailed facial expressions.
Play video starting at :3:56 and follow transcript3:56
We can make her make a smirk; that's quite an extreme smile. You can create these facial expressions. But this isn't the inspector. We really want to eliminate stuff in real time. Now, it's possible to create an animation in unity and control these in real time. But I always find once you get to character animation, using the unity animation system is quite fiddly. If you want to create one in unity, there's also small issues, it does not work. I would generally create counterman relations in another tool, whether it's a free tool like blender or a commercial tool like Maya. But we know this isn't an animation degree. This is a computer science degree. So I thought rather than trying to learn the whole process of animation, which is not really your specialism, nor is it mine. I thought it would be a lot cooler to look at how we can control facial animations in code. What I'm going to do is set up a simple link cycle, which is something you really want to do quite commonly, which is basically, just blinking every so often to give a character a little bit of life. I'm going to add component and I'm going to add one I created earlier.
Play video starting at :5:28 and follow transcript5:28
I called it Blinkx, let's open this up.
Play video starting at :5:36 and follow transcript5:36
Here we have my code, I want to highlight a few things. First off, I need the Skinned Mesh Renderer, as Tom will have showed you, I've used a serialized field for this and I just need to drag in the Skinned Mesh Renderer that we were looking at before, that has the blend shapes attached, so this is the main body. Now, I could have attached this to that mesh itself and just do get components Skinned Mesh Renderer that would have worked. But with characters, the mood of the character is often not the same as the mesh. As you can see here, we've got a character which contains multiple matches, plus all of the joint hierarchy. I think it's nicer to have all of the main scripts, the controller character on the main character game object. I've separated that out and allowed you to choose which of the Skinned Mesh Renderer you use. Let's have a look, now, these are the three other important parameters. The first is the name of the blend shape. This is the blend shape that we want to control.
Play video starting at :6:56 and follow transcript6:56
Then I just want to control how often and for how long we do a blink, so blink length is how long a blink takes. Blinkx interval is how long you spent between blinks. If we look here, I have an end to bend shape name, I'm going to just check what the blend shape is. It is called Is underscore blink, so I'm just going to enter that.
Play video starting at :7:35 and follow transcript7:35
There we go, eyes on the school blink now, blink length should be pretty small, I'm going to set that to not point to. Blinks pretty quick, but you don't want to be blinking too

often, so that should be a bit longer. For now, I'm going to send it to one. We'll see that's unrealistically short, but will do for now.
Play video starting at :8:3 and follow transcript8:03
Those are my variables. Let's see what it does.
Play video starting at :8:8 and follow transcript8:08
She's blinking slightly, weirdly often, as I said, blink interval to be more realistic, we might set it to every three seconds maybe.
Play video starting at :8:32 and follow transcript8:32
You could see you could add this to a character and it would give that character a little bit of life, especially if you combined it with a bit of links, lip-sync animation, then that's all you would really need to create a bit of life for the character, you don't necessarily need very complex animations. Let's have a look at the code and then I'll talk about a couple of ways we could improve this.
Play video starting at :9:1 and follow transcript9:01
I've got two private variables, the first one is timed to next blink, so you need to know when you're going to do your next blink. That's going to be roughly every blink interval seconds and the blink blend shape index. We know the name of the blind shape, but actually the blend shapes is a big array, as you can imagine, and we need to know the index of that. We need to convert that name into a number. In the start methods, we start by doing that. We get the Skinned Mesh Renderer, then we take from that the shade mesh. This is basically the mesh itself, the mesh object. We call on that the function get blend shape index and that takes a string, which is the name. Basically that will convert from the name into the index, into that away, and that's what we lead later on down the line to show that. Then in start, we just set the initial time to next blink to the blink interval. Now, what I'm going to do is I'm actually going to make these two methods public and the reason I want to do that is it means we can actually see them. Sometimes I use these public methods.
Play video starting at :10:46 and follow transcript10:46
I may apply methods, it should be private in the final code, but for debug purposes, I'll make them public for a little bit just so I can see what's going on. Actually, let's start. Here we have time to next blink is zero and blink blend shape index is zero. When I start it up, I've said that the time to next blink is three, which is what you'd expect. Then the blink blend shape index is 40. That's obviously what it was. I actually started off by manually counting down how many was and I got it wrong. That's a bad idea. But you can see it's quite a far way down. So 40 seems realistic and it's right because she's blinking. Let's now look at the update methods where the real stuff is happening. The important thing here is I'm checking whether we've reached the time to do the next blink. What I'm saying is time.time now, you probably encountered the time module and that has various ways of measuring time. Very often we use Delta time, which is the time since the last frame. But actually, here I'm interested in the absolute time. I'm just

going to see if that's bigger than the time to next blink. That means that past that we've reached the point of the time to next blink. Now the next bit is we check the time again, if it's less than the time to the next blink, plus the length of the blink. If it's in between the next blink time and the end of the blink, we need to set the blend shape to a 100, 100 means completely blinked. It's in percentages.
Play video starting at :12:50 and follow transcript12:50
This is acting out the blink and otherwise, it means that we've reached the end of the blink and here we have to set it to zero again. We unblink then the last thing we have to do is update the time to next blink. Basically, once we've finished the blink, we set the time to next blink to the current time, plus the blink intervals. It's another three seconds on. Then we can see all that working. We wait three seconds, she blinks, and then you know that's the time to blink increases by slightly more than 3.2 seconds each time. It's 3.2 because it's the interval plus the blink length. It's a little bit more because we're basically doing that at exactly the next frame. We could set it in different ways if we want it to be exactly 3.2. But that's not a big deal. In fact, it's a little bit too exact at the moment. What improvements can we make to this case? Now I'll leave that to you a little bit of an exercise. But there's two main things that are a bit problematic, it's too regular, we don't blink at exactly three second intervals. What I would say is, have a look at this code and if you feel like it, you could rewrite it so that time to next blink is set not always to the same blink interval, but at a random variation to it. The other thing I would say is that she's blinking instantly, is on and off. Actually, that's not too bad because blinking is a pretty quick movement. But again, we could actually use some kind of lapping to gradually fade in the blink, in and out of time. A lot of blank shapes, you would do that normally, you'd use a lerp method, linear interpolation to change the blend shape weight over time. That's another exercise for you to look at. Then there are probably slightly clever ways of doing the timing. You could possibly use something like Coroutine, if that's something you've heard of, that's a feature of unity. Also where for the entire blink, every frame of the blink, where we're setting the blend shape to zero, which is a bit unnecessary, there's other ways you could use a variable to stop yourself doing that. I wanted to make this quite simple code, so I think I left it at this. That's one example of how you might use blend shapes to animate a character.

Hello, so now I'm going to show you a little bit about facial animation, in particular, blend shapes. Here I've got a current model of the asset store. Unfortunately, it's a little bit harder to find characters that have built-in blend shapes. Most animation and games, mostly body animation, you don't have that much close up facial animation and that's slightly more advanced topic. When I said it's hard to find, it's hard to find free characters. There's plenty of really good paint characters but for the purposes of scores, I don't want you to be spending lots of money on characters. But there are a few out there that you can try if you want to have a go on facial animation.

Play video starting at :1:1 and follow transcript1:01
This is a standard character, she's got bones and skeletons for character animation, you can see her in the hierarchy. But I'm going to look at the base body. This is the mesh that we're using for the character, the main character animation. She's actually got a bunch of other measures like her hair, but the base body is the main one and that includes, of course, the face and that's where the animations are. So if you select that, you can see there's a skinned mesh renderer component. You can see an inspector and under skin mesh renderer, we have blend shapes. The blend shapes tab, that's where all of the current animations happens and you can see we have a whole load of sliders, lots and lots of sliders here.
Play video starting at :1:53 and follow transcript1:53
Each one controls a minor aspect of the face. So this is open, it doesn't actually look open. I don't know why it's called open. But it is affecting the various aspects of the lips. As we said in the videos, facial blend shapes often represent roughly action units. These are simple, minimal movements of the face, like here's a brow raise or brow drop, one sided and you can use these together to give a real raised eyebrow look. But rather than having both eyebrows raised, you have them together to give you separate so that you got that detailed control. You've got some of these slightly odd like Affricate. What what does that mean? Well, that's actually a term from phonology, so the study of language. It's a very detailed aspect of how you might move your lips while you're speaking. This is useful for lip synching. We have a lot of the lips, but we obviously have something for the eye. So eyes blink, closed eyes, and we actually have one for each eye, but also one combine because it's such a common thing we want to be able to easily animate both eyes together and we'll use that in a sec. You can really do quite a lot with these blend shapes and create detailed facial expressions.
Play video starting at :3:56 and follow transcript3:56
We can make her make a smirk; that's quite an extreme smile. You can create these facial expressions. But this isn't the inspector. We really want to eliminate stuff in real time. Now, it's possible to create an animation in unity and control these in real time. But I always find once you get to character animation, using the unity animation system is quite fiddly. If you want to create one in unity, there's also small issues, it does not work. I would generally create counterman relations in another tool, whether it's a free tool like blender or a commercial tool like Maya. But we know this isn't an animation degree. This is a computer science degree. So I thought rather than trying to learn the whole process of animation, which is not really your specialism, nor is it mine. I thought it would be a lot cooler to look at how we can control facial animations in code. What I'm going to do is set up a simple link cycle, which is something you really want to do quite commonly, which is basically, just blinking every so often to give a character a little bit of life. I'm going to add component and I'm going to add one I created earlier.
Play video starting at :5:28 and follow transcript5:28

I called it Blinkx, let's open this up.
Play video starting at :5:36 and follow transcript5:36
Here we have my code, I want to highlight a few things. First off, I need the Skinned Mesh Renderer, as Tom will have showed you, I've used a serialized field for this and I just need to drag in the Skinned Mesh Renderer that we were looking at before, that has the blend shapes attached, so this is the main body. Now, I could have attached this to that mesh itself and just do get components Skinned Mesh Renderer that would have worked. But with characters, the mood of the character is often not the same as the mesh. As you can see here, we've got a character which contains multiple matches, plus all of the joint hierarchy. I think it's nicer to have all of the main scripts, the controller character on the main character game object. I've separated that out and allowed you to choose which of the Skinned Mesh Renderer you use. Let's have a look, now, these are the three other important parameters. The first is the name of the blend shape. This is the blend shape that we want to control.
Play video starting at :6:56 and follow transcript6:56
Then I just want to control how often and for how long we do a blink, so blink length is how long a blink takes. Blinkx interval is how long you spent between blinks. If we look here, I have an end to bend shape name, I'm going to just check what the blend shape is. It is called Is underscore blink, so I'm just going to enter that.
Play video starting at :7:35 and follow transcript7:35
There we go, eyes on the school blink now, blink length should be pretty small, I'm going to set that to not point to. Blinks pretty quick, but you don't want to be blinking too often, so that should be a bit longer. For now, I'm going to send it to one. We'll see that's unrealistically short, but will do for now.
Play video starting at :8:3 and follow transcript8:03
Those are my variables. Let's see what it does.
Play video starting at :8:8 and follow transcript8:08
She's blinking slightly, weirdly often, as I said, blink interval to be more realistic, we might set it to every three seconds maybe.
Play video starting at :8:32 and follow transcript8:32
You could see you could add this to a character and it would give that character a little bit of life, especially if you combined it with a bit of links, lip-sync animation, then that's all you would really need to create a bit of life for the character, you don't necessarily need very complex animations. Let's have a look at the code and then I'll talk about a couple of ways we could improve this.
Play video starting at :9:1 and follow transcript9:01
I've got two private variables, the first one is timed to next blink, so you need to know when you're going to do your next blink. That's going to be roughly every blink interval seconds and the blink blend shape index. We know the name of the blind shape, but actually the blend shapes is a big array, as you can imagine, and we need to know the

index of that. We need to convert that name into a number. In the start methods, we start by doing that. We get the Skinned Mesh Renderer, then we take from that the shade mesh. This is basically the mesh itself, the mesh object. We call on that the function get blend shape index and that takes a string, which is the name. Basically that will convert from the name into the index, into that away, and that's what we lead later on down the line to show that. Then in start, we just set the initial time to next blink to the blink interval. Now, what I'm going to do is I'm actually going to make these two methods public and the reason I want to do that is it means we can actually see them. Sometimes I use these public methods.

Play video starting at :10:46 and follow transcript10:46

I may apply methods, it should be private in the final code, but for debug purposes, I'll make them public for a little bit just so I can see what's going on. Actually, let's start. Here we have time to next blink is zero and blink blend shape index is zero. When I start it up, I've said that the time to next blink is three, which is what you'd expect. Then the blink blend shape index is 40. That's obviously what it was. I actually started off by manually counting down how many was and I got it wrong. That's a bad idea. But you can see it's quite a far way down. So 40 seems realistic and it's right because she's blinking. Let's now look at the update methods where the real stuff is happening. The important thing here is I'm checking whether we've reached the time to do the next blink. What I'm saying is time.time now, you probably encountered the time module and that has various ways of measuring time. Very often we use Delta time, which is the time since the last frame. But actually, here I'm interested in the absolute time. I'm just going to see if that's bigger than the time to next blink. That means that past that we've reached the point of the time to next blink. Now the next bit is we check the time again, if it's less than the time to the next blink, plus the length of the blink. If it's in between the next blink time and the end of the blink, we need to set the blend shape to a 100, 100 means completely blinked. It's in percentages.

Play video starting at :12:50 and follow transcript12:50

This is acting out the blink and otherwise, it means that we've reached the end of the blink and here we have to set it to zero again. We unblink then the last thing we have to do is update the time to next blink. Basically, once we've finished the blink, we set the time to next blink to the current time, plus the blink intervals. It's another three seconds on. Then we can see all that working. We wait three seconds, she blinks, and then you know that's the time to blink increases by slightly more than 3.2 seconds each time. It's 3.2 because it's the interval plus the blink length. It's a little bit more because we're basically doing that at exactly the next frame. We could set it in different ways if we want it to be exactly 3.2. But that's not a big deal. In fact, it's a little bit too exact at the moment. What improvements can we make to this case? Now I'll leave that to you a little bit of an exercise. But there's two main things that are a bit problematic, it's too regular, we don't blink at exactly three second intervals. What I would say is, have a

look at this code and if you feel like it, you could rewrite it so that time to next blink is set not always to the same blink interval, but at a random variation to it. The other thing I would say is that she's blinking instantly, is on and off. Actually, that's not too bad because blinking is a pretty quick movement. But again, we could actually use some kind of lapping to gradually fade in the blink, in and out of time. A lot of blank shapes, you would do that normally, you'd use a lerp method, linear interpolation to change the blend shape weight over time. That's another exercise for you to look at. Then there are probably slightly clever ways of doing the timing. You could possibly use something like Coroutine, if that's something you've heard of, that's a feature of unity. Also where for the entire blink, every frame of the blink, where we're setting the blend shape to zero, which is a bit unnecessary, there's other ways you could use a variable to stop yourself doing that. I wanted to make this quite simple code, so I think I left it at this. That's one example of how you might use blend shapes to animate a character.

The eyes are perhaps the most important social signal of all. Making eye contact is a form of social connection. It shows that we're paying attention to someone and can indicate a range of social meanings from intimacy to aggression. When we say emulate gaze behavior, there are only way to two states, the character is either looking at the other player or not. If it's looking, that means its eyes are turned towards the position of the camera. If it isn't looking it can be more complex because there are so many other places it could look. We might simulate the complex patterns of gaze we use when we're thinking, or it might be distracted by objects in the environment. But people are basically pretty egocentric. We'll definitely notice if someone is looking at us, but we're a lot less likely to pay attention to the other places they're looking. Looking around a bit at random is probably fine. The main exception is if the player and the character are both focusing on an object. If for example the character is explaining a map to the player. The character should look at the map to encourage the player to also focus on it. The most important thing about gaze and social interaction is controlling whether the character is looking at the player or not. The basic rule is that you should look at the other person during conversation, but staring constantly can be off-putting; we normally look away occasionally. The key thing is how often we look and for how long. This depends on a lot of things but primarily on conversational terms, we'd look more when we're listening and a bit less when we're talking. We don't make much eye contact when neither person is talking unless we're having a romantic moment. Conversation state can act a bit like a state machine with states for talking, listening, and neither. The parameters of eye gaze behavior change depending on the state. Of course, gaze behavior is much more complex than this, and there was a lot we can do to emulate it. But just alternating between eye contact and not eye contact and changing the proportions of time based on whether we're talking and listening, can actually create a very realistic feeling character and give a strong sense of being in a real social

interaction. Animating the eyes is relatively simple, you can treat each eye as a joint and rotate them towards where you want to look. We can generally do this in a procedural way, it's simple enough not to need an animation and we want to make sure that the eyes are pointing in exactly the right place, which would be hard to do with a pre-recorded animation. We normally use some basic inverse kinematics to point the eyes. There are few things we want to think about, for looking at things straight ahead of us, we can just turn the eyes. But if you want to look at something at a larger angle, it's easier also to turn the head and maybe even the torso. You also need to limit the range of where you can look. If you try to get a cats look at something behind them, their eyes might swivel around in their sockets or their head might turn backwards. That can freak people out. If you want to get really advanced, you can look at things like how the eyelids move or how the pupils change sides. This isn't something I'm going to go into in this course, but definitely something you want to try if you're aiming for highly realistic animation. Eye gaze can be very complex, but a few simple techniques are enough to give the impression of a very realistic engaging social interaction, and you can include them in your characters very simply and that will make them much more expressive and create a much stronger social connection.

We now have quite a fully-functional character with body language, so gestures and postures coming from animations with facial expressions. That's responding to whether we're listening or talking, doing lots of interesting stuff. The last thing I'd like to add to the character is gay, so use of the eyes in an automated way. To do that, I'm going to add another component called Look At. Again, this is script I've written, and again, this is not the best possible script for this. I'll put links to more advanced ones you can use.
Play video starting at ::53 and follow transcript0:53
This script actually has quite a lot of parameters. Two very important ones that you can't do anything without the Look At Target, which is what the cartoons look at, that's going to be the camera, and the eye. We need to know where the eye is, and we're going to drag in, this is from the characters hierarchy of the character skeleton level going all the way up to the head, and then to the eye.
Play video starting at :1:22 and follow transcript1:22
Then there are a few other things. Look At Time is how long the cart spends looking at you, Look Away Time is how long it spends looking away from you, so that alternates between looking at you and looking away from you. That proportion will vary depending on whether your talking or listening. We want to change that proportion to how much we're looking. At the moment, the way I've implemented this is that when it looks at you, it's looking at you. Otherwise it's just playing back the original animation. It blends between the original animation and looking at the player. We could also have other targets explicitly to look into other places, but this is the way this particular script is implemented. There are other scripts which work in other ways. Then there are two

probabilities, the Head Look probability and the Eye Look probability. The Eye Look probabilities, is the probability that you look with your eyes, that should always really be the maximum value, which is one, but the Head Look probability is, the probability that the character looks with their head as opposed to just looking at their eyes. That's quite an expressive effect. If you have to turn more than a certain amount the cart will always look it with the head. But you can vary how often the character actually looks with their head if there is a small distance and these probabilities go from 0-1, so 0.5 is about half the time. These are the maximum rotations. The max x rotation and the max y rotations, the maximum total amount you can rotate and the max x and y eye rotations are the amount that you can retake before you have to move your head and you can rotate just with the eyes. You don't necessarily need to change these. Look at time parameters, I'll talk about these later. Transition speeds or how long it takes to switch between looking in one direction or another. That's how fast the character moves when he's looking at you. Let's have a look at this. Let's play it, and you can see my camera. He's looking at me. There he is and he's looking away. The characters alternating [inaudible]. This is good, we've got a character that can look at you. I mean, it's a very powerful signal. It's very important. But how much you look is very dependent on contextual factors. The character shouldn't really look very much when in the idle state, but should definitely look a lot when listening, so we should change these. Now, how I'm going to do this, is I'm going to attach an extra variable to some of these animations. If we go to the original animation here, which is idle one, I'm going to attach a curve. This is a variable and I'm going to call this curve actually let me go back a minute. I'm going to call it Look At Time, for reasons that will be clear later.

Play video starting at :5:6 and follow transcript5:06

I'm going to set the value and I'm also going to create one called, Look Away Time.

Play video starting at :5:19 and follow transcript5:19

Now, this is for the idle state, so we want the character to mostly be looking away. I'll set this to 10 seconds and this to one seconds, maybe two seconds. This creates a curve that sets the key frame at the beginning. You can edit the curve and let's just give that a constant value. It's possible to animate these curves over time, but actually we just want to constant value for these events. We can apply that.

Play video starting at :5:49 and follow transcript5:49

Then we go into the parameters. What happens to this? We've set up these new parameters that are attached to the animations. What they use is they set values of parameters in the animator parameters here. If I create a parameter called, Look At Time, and Look Away Time.

Play video starting at :6:28 and follow transcript6:28

These will come from the values that are attached to the animation. This is actually getting values out of an animator. Let's have a quick look what happens here. When I play the idle state and nothing happens, so I got it right.

Play video starting at :6:51 and follow transcript6:51
I have done it for the wrong animation, sorry.
Play video starting at :7:21 and follow transcript7:21
I'll apply that. Apologies for that. We can see it set these values to the values that were in the curves.
Play video starting at :7:39 and follow transcript7:39
If all works well, we can see that it's set the values here in the, Look At Script, as well. The values of Look At Script, have been set. Unfortunately, after that they will set back to zero, which is not what we want. But that means we have to go through and do this for all of the animations that we have in the project. Good to see the remaining ones are part of the talking and listening.
Play video starting at :8:20 and follow transcript8:20
When we're talking, we basically want pretty even, Looking and Not Looking.
Play video starting at :8:37 and follow transcript8:37
Maybe five and five.
Play video starting at :8:43 and follow transcript8:43
You'll need to play with these values to make sure they are appropriate, that they work.
Play video starting at :8:53 and follow transcript8:53
I'll need to do the same for all of these.
Play video starting at :9:10 and follow transcript9:10
I won't do it for every single piece of animation because it'll be too slow. But you get the idea. I'll just do it, to be very clear. I'll do it for the listening as well.
Play video starting at :9:38 and follow transcript9:38
I can set that here. We want to bias it very much towards listening, so I'm going to do 10 for, Looking At, at the beginning.
Play video starting at :9:53 and follow transcript9:53
That's 10 and 10 isn't it. Yeah. Look Away Time, I think should be two.
Play video starting at :10:10 and follow transcript10:10
We've set all of these values for listening, the talking animation, the listening animation, the idle animation. There are a couple of other clips that I haven't set, but it's exactly the same thing. If I start off the program, you can see the values are actually at two and 10. If I start talking, we can see it goes to five, should I restart? I didn't talk for loud enough.
Play video starting at :10:46 and follow transcript10:46
If I start talking quite loudly, he should recognize that I'm talking, and the values sets, didn't seem to be set right there. Let me just check if I've done the right thing. I have probably done the wrong thing. I set listening to
Play video starting at :11:33 and follow transcript11:33
I'll apply that, and let's try that again.
Play video starting at :11:38 and follow transcript11:38

As you can see, it's very easy to make errors even if you've been doing this for years. Now I'm setting it to listening, and it's looking at me much more. That's exactly what I want to do, now it's gone down to five, five when I'm talking.
Play video starting at :11:59 and follow transcript11:59
I'm able to change the details of the gaze behavior based on what am doing.
Play video starting at :12:12 and follow transcript12:12
That's almost it, I think I want to show you one last trick, which is that when this character starts listening to you, I want him to look at you. For that, that's something else that you can attach to an animation, that's an event. I'm going to attach an event at the beginning of an animation, and the event is going to have exactly the same name as a function in my look at script. It's a function called look at player, you don't worry if you're not a coder, all you need to know that this exists, and look away from player also exists. I can have a look at player, and I've applied that, and we've now got an event attached to the beginning of the Listen 1 animation. When the Listen 1 animation plays, the character should instantly look at me. If I start talking loudly, he should listen and he's instantly looking at me.
Play video starting at :13:19 and follow transcript13:19
We can control both the parameters of how the interaction works, and also set very specific events that we want the characters to look at. That's in the events under the particular animation you're creating. We've covered a lot of different animation techniques in this video, and we've shown how we can use these very sophisticated features that unity has, together with a couple of fairly simple scripts to create quite a realistic character that is able to interact with us, and have a mini conversation. Let's just have one more and just to see everything going on together. If I look at the character, let's actually maximize, can I maximize?
Play video starting at :14:19 and follow transcript14:19
He's ignoring me now. Hello there, hello, nice to meet you character, what's your name? Hello, welcome to our real virtual characters demo. Would you like to find out more? I'm a virtual character, programmed to respond to you with realistic body language.
Play video starting at :14:40 and follow transcript14:40
There we are, we have our first interactive virtual character. The animation could be better if I was doing this properly, I would have taken much more time to select and copy the animations, get some good facial animation, get all the features and parameters just right, it takes a lot of tweaking. Obviously, I could create a much more complex state machine to have much more complex behaviors. But as it is, it's the first sign that we can create actually compelling experience, which is an encounter in virtual reality with a virtual character, all that using basic unity and a head-mounted display.

By now, you have learned a lot about facial animation. Simple emotion or communicative acts, gaze, blinking, lip-sync. When you want to animate an avatar

talking, you might also add some simple head movement to synchronize with the speech to make it more realistic. Here is an example of something I did in 2007 when I started doing facial animation. It's really quite basic and doesn't really look great, but it's a simple idea to show you how everything could work together. There are a lot of things you could add to this video. Her eyes look a bit unnatural, so you may want to add some animations there. You could also put more a head animation or may her move her body a little bit more. Overall, because the model is quite photo realistic, ideally, we could use motion capture data to animate this rather than key frame by hand. However, there is something else I did in this video which might have had some psychological effect on you unnoticed. As she was making the apology, her face was turning red. She was blushing out of embarrassment. As humans, most of us would blush when we feel embarrassed and anxious, and people who blush more often are considered to be shy. Blushing is a very interesting expression as it is unique to human beings. Animals don't blush. In 1872, Charles Darwin refers to blushing as the most human of all expressions. What puzzles the psychologists about blushing is that, we don't quite understand why we blush. Or what is the social function blushing serves? It's not an easy question to answer because first of all, psychologists believe that the effect of blushing is subconscious. This means that we could be reacting to it without being aware of it. Secondly, is something quite difficult to test in real life. We can't really find someone who can blush on-demand. Not even the best actors. But with virtual characters, this is easy. We can blend a dark red texture with a facial texture of the character, and change the blending parameters over time to create a blushing virtual character, like I did in the video you saw. So I ran some experiments with this method and found that, first of all, unless you knew it in advance, otherwise for most people blushing is not noticeable. So if you didn't notice the blushing in the video, don't worry, you are completely normal. Secondly, although most of us don't consciously notice blushing, it does have a subconscious effect on our attitude towards the person blushes. We turn to build a stronger rapport with those who blush. In virtual reality, we would find the blushing virtual character more real as compared to the non-blushing ones. Similar to blushing, there are other subtle facial features you can add to the avatars to make them more expressive and human-like. For instance, instead of blushing, you can make their face go paler. Maybe when the avatar fuse frightened, or you can add tears in their eyes when they're having an emotional reaction. The size of our pupils is also something else you can animate. For instance, pupil dilation often indicates surprise or interest. If you try adding some of these subtle facial features to your avatar, you might achieve some surprising results. Your participants might be amazed as most of these subtle features have an effect that operate at a subconscious level.